

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ
TRUYỀN THÔNG (SOICT)



BÁO CÁO DỰ ÁN LƯU TRỮ VÀ XỬ LÝ
DỮ LIỆU LỚN

Mã học phần: IT4931

Đề tài:

**Xây dựng hệ thống thu thập và phân tích
dữ liệu thị trường lao động từ TOPCV**

Giảng viên hướng dẫn: TS. Trần Việt Trung

Nhóm: 9

Sinh viên thực hiện:
Phạm Anh Khôi - 20230043
Trần Tuấn Minh - 20230051
Nguyễn Phúc Huy Hoàng - 20235087
Nguyễn Bá Đức Anh - 20235004
Trần Gia Định - 20235036

Hà Nội, Tháng 6/2026

Mục lục

LỜI CẢM ƠN	7
LỜI MỞ ĐẦU	8
PHÂN CÔNG CÔNG VIỆC	10
1 GIỚI THIỆU TỔNG QUAN	13
1.1 Bối cảnh và động lực dự án	13
1.2 Mục tiêu dự án	14
1.3 Phạm vi dự án và công nghệ sử dụng	15
2 KIẾN TRÚC HỆ THỐNG	18
2.1 Tổng quan Lambda Architecture	18
2.2 Mô hình triển khai	19
2.2.1 Tổng quan mô hình triển khai	19
2.2.2 Phân chia namespace	19
2.2.3 Cấu hình triển khai thực nghiệm	21
2.3 Cấu trúc mã nguồn	23
2.3.1 Thư mục <code>apps/</code>	24
2.3.2 Thư mục <code>configs/</code>	26
2.3.3 Thư mục <code>data/</code>	26
2.3.4 Thư mục <code>infra/</code>	27
2.3.5 Thư mục <code>shared/</code>	27
2.3.6 Thư mục <code>docs/</code> , <code>scripts/</code> và <code>tests/</code>	27
2.3.7 Các file cấu hình ở thư mục gốc	28
3 THIẾT KẾ DỮ LIỆU: KIẾN TRÚC MEDALLION	29
3.1 Tổng quan 4 lớp dữ liệu	29
3.2 Lớp dữ liệu thô (Raw Layer)	30
3.3 Lớp dữ liệu đồng (Bronze Layer)	31
3.4 Lớp dữ liệu bạc (Silver Layer)	34
3.5 Lớp dữ liệu vàng (Gold Layer)	36
4 ĐƯỜNG ống XỬ LÝ DỮ LIỆU (DATA PIPELINE)	41
4.1 Quy trình xử lý dữ liệu tổng quát	41
4.2 Thu thập dữ liệu (Ingestion & Kafka Producer)	43
4.2.1 Xác định nguồn và phạm vi crawl	44
4.2.2 Trích xuất và tạo bản ghi JSONL	44

4.2.3	Thực hiện nạp dữ liệu vào HDFS cho batch layer	44
4.2.4	Thực hiện publish dữ liệu vào Kafka cho speed layer	45
4.2.5	Kiểm tra lỗi và đảm bảo chất lượng đầu vào	46
4.3	Batch Processing Layer (Apache Spark & HDFS)	46
4.3.1	Tổng quan luồng xử lý Batch Pipeline	47
4.3.2	Huấn luyện mô hình dự báo lương (<code>train_salary_model.py</code>) . .	48
4.3.3	Nạp dữ liệu vào Elasticsearch (<code>gold_to_elasticsearch.py</code>) . . .	48
4.3.4	Điều phối tự động bằng Apache Airflow	48
4.4	Speed Processing Layer (Spark Structured Streaming)	50
4.4.1	Tổng quan luồng xử lý realtime	50
4.4.2	Kafka topics và vai trò của từng luồng dữ liệu	51
4.4.3	Schema đầu vào và kiểm tra chất lượng dữ liệu	51
4.4.4	Chuẩn hóa realtime và tạo schema sạch	52
4.4.5	Tích hợp mô hình dự báo lương trong luồng realtime	52
4.4.6	Ghi dữ liệu ra Kafka và Elasticsearch	53
4.4.7	Tổng hợp trạng thái theo thời gian	53
4.4.8	Checkpoint, trigger và khả năng phục hồi	54
4.5	Điều phối luồng công việc với Airflow	54
4.5.1	Điều phối Batch Pipeline	55
4.5.2	Điều phối Speed Layer Bootstrap	57
4.5.3	Điều phối Real Crawler cho Speed Layer	58
4.5.4	Cơ chế giám sát và xử lý lỗi	59
5	MÔ HÌNH HỌC MÁY: DỰ ĐOÁN LƯƠNG	60
5.1	Bài toán và cấu trúc mô hình	60
5.2	Trích xuất đặc trưng	61
5.3	Tích hợp model	62
6	TRỰC QUAN HÓA DỮ LIỆU	65
6.1	Lưu trữ dữ liệu trên Elasticsearch	65
6.1.1	Batch index	65
6.1.2	Realtime indexes	66
6.1.3	Vai trò của Elasticsearch trong hệ thống	67
6.2	Trực quan hóa dữ liệu sử dụng Kibana Dashboard	67
6.2.1	Dashboard cho batch layer	67
6.2.2	Dashboard cho speed layer	70
6.2.3	Quy trình import và sử dụng dashboard	70
7	BÀI HỌC KINH NGHIỆM	71
7.1	Bài học 1: Crawler cần có cơ chế fallback khi cấu trúc website thay đổi .	71
7.1.1	Mô tả vấn đề	71
7.1.2	Cách tiếp cận đã thử	71
7.1.3	Giải pháp cuối cùng	72
7.1.4	Điểm rút ra	72
7.2	Bài học 2: Chống anti-bot cần cân bằng giữa tốc độ và độ ổn định	72
7.2.1	Mô tả vấn đề	72
7.2.2	Cách tiếp cận đã thử	72
7.2.3	Giải pháp cuối cùng	72

7.2.4	Điểm rút ra	73
7.3	Bài học 3: Tránh trùng lặp dữ liệu phải xét cả định danh và sự thay đổi nội dung	73
7.3.1	Mô tả vấn đề	73
7.3.2	Cách tiếp cận đã thử	73
7.3.3	Giải pháp cuối cùng	73
7.3.4	Điểm rút ra	74
7.4	Bài học 4: Chất lượng dữ liệu cần được kiểm soát ngay từ tầng raw . . .	74
7.4.1	Mô tả vấn đề	74
7.4.2	Cách tiếp cận đã thử	74
7.4.3	Giải pháp cuối cùng	74
7.4.4	Điểm rút ra	74
7.5	Bài học 5: Xử lý dữ liệu với Spark cần tối ưu theo luồng dữ liệu và giới hạn tài nguyên thực tế	75
7.5.1	Mô tả vấn đề	75
7.5.2	Cách tiếp cận đã thử	75
7.5.3	Giải pháp cuối cùng	75
7.5.4	Điểm rút ra	75
7.6	Bài học 6: Xử lý luồng cần thiết kế rõ về checkpoint, cửa sổ thời gian và trạng thái	76
7.6.1	Mô tả vấn đề	76
7.6.2	Cách tiếp cận đã thử	76
7.6.3	Giải pháp cuối cùng	76
7.6.4	Điểm rút ra	76
7.7	Bài học 7: Lưu trữ dữ liệu cần phân tầng theo giá trị sử dụng và vòng đời dữ liệu	76
7.7.1	Mô tả vấn đề	76
7.7.2	Cách tiếp cận đã thử	77
7.7.3	Giải pháp cuối cùng	77
7.7.4	Điểm rút ra	77
7.8	Bài học 8: Tích hợp hệ thống phân tán cần chuẩn hóa service discovery và cơ chế lỗi	77
7.8.1	Mô tả vấn đề	77
7.8.2	Cách tiếp cận đã thử	77
7.8.3	Giải pháp cuối cùng	77
7.8.4	Điểm rút ra	78
7.9	Bài học 9: Tối ưu hiệu năng phải bắt đầu từ việc xác định đúng điểm nghẽn	78
7.9.1	Mô tả vấn đề	78
7.9.2	Cách tiếp cận đã thử	78
7.9.3	Giải pháp cuối cùng	78
7.9.4	Điểm rút ra	78
7.10	Bài học 10: Giám sát và gỡ lỗi cần log có cấu trúc, metrics và khả năng truy vết theo pipeline	79
7.10.1	Mô tả vấn đề	79
7.10.2	Cách tiếp cận đã thử	79
7.10.3	Giải pháp cuối cùng	79
7.10.4	Điểm rút ra	79

7.11	Bài học 11: Mở rộng hệ thống cần lập kế hoạch ngang, dọc và chi phí tài nguyên	79
7.11.1	Mô tả vấn đề	79
7.11.2	Cách tiếp cận đã thử	79
7.11.3	Giải pháp cuối cùng	80
7.11.4	Điểm rút ra	80
7.12	Bài học 12: Chất lượng dữ liệu và kiểm thử phải được đặt ngang hàng với code xử lý	80
7.12.1	Mô tả vấn đề	80
7.12.2	Cách tiếp cận đã thử	80
7.12.3	Giải pháp cuối cùng	80
7.12.4	Điểm rút ra	80
7.13	Bài học 13: Bảo mật và quản trị dữ liệu cần được chuẩn bị ngay cả trong môi trường học thuật	81
7.13.1	Mô tả vấn đề	81
7.13.2	Cách tiếp cận đã thử	81
7.13.3	Giải pháp cuối cùng	81
7.13.4	Điểm rút ra	81
7.14	Bài học 14: Chịu lỗi cần kết hợp phục hồi tự động, checkpoint và khả năng tái xử lý	81
7.14.1	Mô tả vấn đề	81
7.14.2	Cách tiếp cận đã thử	81
7.14.3	Giải pháp cuối cùng	82
7.14.4	Điểm rút ra	82
	TÀI LIỆU THAM KHẢO	83

Danh sách bảng

1	Phân công công việc trong nhóm	11
1.1	Các công nghệ chính sử dụng trong dự án	16
2.1	Các namespace chính trong mô hình triển khai	20
3.1	Đặc tả cấu trúc cấp ngoài cùng (Root Schema v1.0)	30
3.2	Đặc tả cấu trúc khối payload thô	32
3.3	Đặc tả cấu trúc khối Quality Flags	33
3.4	Đặc tả các trường tạo mới trong bảng dữ liệu Bronze	34
3.5	Đặc tả các trường tạo mới tại lớp Silver	37
3.6	Đặc tả các trường dữ liệu mới xuất hiện tại tầng Gold	40
4.1	So sánh đặc tính kỹ thuật giữa Batch Layer và Speed Layer	43
4.2	Tổng quan các Spark Job trong Batch Pipeline	47
4.3	Cấu hình kết nối Elasticsearch trong Batch Pipeline	49
4.4	Cấu hình timeout của từng task trong Airflow DAG	49
4.5	Cấu hình Spark Session dùng chung cho tất cả Batch Job	50
4.6	Các Kafka topic chính trong Speed Layer	51
4.7	Các realtime aggregation trong Speed Layer	53
4.8	Một số cấu hình runtime quan trọng của Speed Layer	54
5.1	Quy tắc tạo nhãn huấn luyện cho mô hình dự đoán lương	61
5.2	Các đặc trưng sử dụng cho mô hình dự đoán lương	62
5.3	Các trường lương sau khi tích hợp mô hình	63
6.1	Các Elasticsearch index của speed layer	66

Danh sách hình vẽ

2.1	Cấu hình node của cụm Kubernetes/Minikube thực nghiệm.	21
2.2	Các workload và container được cấu hình trong hệ thống Kubernetes. . .	22
2.3	Các container image được sử dụng bởi các workload của hệ thống.	23
3.1	Luồng di chuyển và xử lý dữ liệu qua các tầng Medallion	29
4.1	Kiến trúc Lambda tổng quát của hệ thống Big Data Job Market	41
4.2	Luồng dữ liệu tuần tự của Batch Processing Pipeline	47
4.3	Luồng dữ liệu của Speed Processing Layer	51
4.4	Tái sử dụng mô hình dự báo lương từ Batch Layer cho Speed Layer . . .	53
4.5	Danh sách các DAG Airflow được triển khai trong hệ thống, gồm DAG điều phối batch pipeline, DAG khởi động speed layer và DAG chạy real crawler producer định kỳ.	55
4.6	Graph View của DAG <code>job_market_batch_pipeline</code> , thể hiện thứ tự điều phối các bước trong batch pipeline từ kiểm tra hạ tầng, crawl dữ liệu, xử lý Raw–Bronze–Silver–Gold đến ghi dữ liệu sang Elasticsearch.	56
6.1	Danh sách các Elasticsearch index của hệ thống, bao gồm index batch <code>gold-jobs-flat</code> và các index realtime phục vụ trực quan hóa gần thời gian thực.	67
6.2	Dashboard tổng quan thị trường tuyển dụng, thể hiện quy mô dữ liệu, phân bố nhu cầu tuyển dụng theo thành phố, nhóm ngành và kỹ năng được nhà tuyển dụng yêu cầu nhiều nhất.	68
6.3	Dashboard phân tích lương từ các tin tuyển dụng có lương công khai, thể hiện phân bố khoảng lương phổ biến, nhóm ngành có lương trung vị cao và mặt bằng lương theo thành phố.	69
6.4	Minh họa kết quả làm giàu dữ liệu lương bằng mô hình học máy, trong đó các tin tuyển dụng có lương gốc là “Thoả thuận” được bổ sung mức lương ước lượng tham khảo ở lớp Gold.	69

LỜI CẢM ƠN

Trước hết, chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến thầy Trần Việt Trung với vai trò giảng viên bộ môn Lưu trữ và xử lý dữ liệu lớn. Để hoàn thành project và bài báo cáo này một cách trọn vẹn, bên cạnh sự nỗ lực của các thành viên trong nhóm, chúng em đã nhận được sự định hướng và giúp đỡ vô cùng quý báu từ phía thầy. Trong suốt quá trình học tập tại trường, những kiến thức nền tảng vững chắc cùng sự hướng dẫn tận tình từ thầy đã là hành trang quý báu giúp chúng em tự tin nghiên cứu và thực hiện dự án này.

Do giới hạn về mặt thời gian cũng như kinh nghiệm thực tế của các thành viên trong nhóm, dự án này chắc chắn không tránh khỏi những thiếu sót thiếu khách quan. Nhóm chúng em rất mong sẽ tiếp tục nhận được những lời nhận xét, góp ý quý báu của thầy để nội dung báo cáo được hoàn thiện hơn, đồng thời giúp chúng em tích lũy thêm những hành trang vững chắc cho các đề án lớn tiếp theo.

LỜI MỞ ĐẦU

Trong kỷ nguyên số hóa và sự chuyển dịch không ngừng của nền kinh tế, thị trường lao động tại Việt Nam đang chứng kiến những biến động liên tục, đặc biệt là sự thay đổi nhanh chóng về nhu cầu kỹ năng và các tiêu chuẩn đãi ngộ. Mặc dù thông tin tuyển dụng hiện nay rất phong phú và dễ tiếp cận thông qua các nền tảng lớn như TopCV, nhưng dữ liệu này thường ở dạng phân tán, thiếu cấu trúc đồng nhất và khó có thể tổng hợp trực tiếp để đưa ra các phân tích có giá trị vĩ mô. Việc biến khối lượng dữ liệu thô khổng lồ này thành những tri thức sâu sắc hay nắm bắt xu hướng công nghệ theo thời gian thực là một thách thức lớn, đòi hỏi một hệ thống kỹ thuật đủ mạnh mẽ, tự động và linh hoạt.

Xuất phát từ bài toán thực tiễn đó, dự án **Big Data Job Market** được nghiên cứu và phát triển nhằm mục tiêu thu thập, lưu trữ, tinh chế và phân tích dữ liệu thị trường lao động. Điểm nhấn kỹ thuật của dự án nằm ở việc áp dụng Lambda Architecture kết hợp cùng mô hình dữ liệu Medallion. Sự kết hợp này mang lại những lợi ích vượt trội cho hệ thống:

- **Đảm bảo tính toàn vẹn và độ trễ thấp:** Lớp Batch Layer xử lý khối lượng lớn dữ liệu lịch sử một cách chính xác, trong khi Speed Layer (với Kafka và Spark Streaming) cập nhật dữ liệu gần thời gian thực (real-time). Điều này giúp hệ thống vừa có cái nhìn tổng quan dài hạn, vừa bắt kịp những biến động tức thời của thị trường.
- **Quản trị chất lượng dữ liệu khắt khe:** Việc luân chuyển dữ liệu qua 4 lớp Raw - Bronze - Silver - Gold của mô hình Medallion giúp chuẩn hóa hoàn toàn các thông tin phi cấu trúc, loại bỏ dữ liệu rác, bóc tách chính xác các trường phức tạp (như JSON-LD) và giải quyết triệt để bài toán trùng lặp (deduplication) với nhiều phiên bản cập nhật của cùng một tin tuyển dụng.
- **Khép kín vòng lặp giá trị bằng Machine Learning:** Thông qua việc tích hợp mô hình học máy Spark ML GBTRegressor tại lớp dữ liệu Gold, hệ thống giải quyết được một trong những điểm mù lớn nhất của thị trường việc làm: các tin tuyển dụng không công khai mức lương. Mô hình tận dụng các đặc trưng quan trọng để dự báo dải lương. Nhờ đó, trải nghiệm tìm kiếm, trích lọc và trực quan hóa dữ liệu trên Dashboard (Kibana/Elasticsearch) trở nên liền mạch, mang lại giá trị tham khảo thiết thực cho cả ứng viên lẫn nhà tuyển dụng.

Để trình bày toàn diện về quá trình nghiên cứu, thiết kế và triển khai hệ thống, báo cáo này được cấu trúc một cách bài bản, chia thành các phần chính như sau:

- **Phần 1:** Tổng quan dự án và Kiến trúc hệ thống. Trình bày các mục tiêu cốt lõi, công nghệ sử dụng (Tech Stack) và sơ đồ tổng thể theo Lambda Architecture.

- **Phần 2:** Thiết kế dữ liệu theo kiến trúc Medallion. Đi sâu vào Data Contract và vòng đời biến đổi của dữ liệu qua 4 phân lớp (Raw, Bronze, Silver, Gold), cùng các quy tắc chuẩn hóa nghiệp vụ.
- **Phần 3:** Xử lý luồng dữ liệu (ETL & Streaming). Chi tiết hóa các job tại Batch Layer và cơ chế hoạt động của Speed Layer trong việc xử lý dữ liệu gần thời gian thực.
- **Phần 4:** Ứng dụng Machine Learning, áp dụng mô hình GBTRegressor trong việc dự báo mức lương.
- **Phần 5:** Triển khai hạ tầng và vận hành. Đề cập đến quá trình đóng gói và điều phối hệ thống trên môi trường Kubernetes bằng Apache Airflow.

Thông qua báo cáo này, chúng em hy vọng sẽ cung cấp một góc nhìn toàn diện về cách các hệ thống dữ liệu lớn được thiết kế và vận hành để giải quyết một bài toán thiết thực trong xã hội.

PHÂN CÔNG CÔNG VIỆC

Trong quá trình thực hiện dự án **Big Data Job Market**, nhóm đã phân chia công việc theo từng thành phần chính của hệ thống nhằm đảm bảo mỗi thành viên phụ trách một mảng rõ ràng, phù hợp với kiến trúc tổng thể của pipeline. Việc phân công được thực hiện dựa trên các tầng chức năng của hệ thống, bao gồm thu thập dữ liệu, xử lý batch, xử lý streaming, học máy, điều phối workflow và trực quan hóa dữ liệu.

Bảng 1 trình bày chi tiết phân công công việc của các thành viên trong nhóm.

Bảng 1: Phân công công việc trong nhóm

Thành viên	Mã sinh viên	Nội dung phụ trách
Phạm Anh Khôi	20230043	Phụ trách Speed Layer : thiết kế và triển khai luồng xử lý dữ liệu gần thời gian thực với Kafka và Spark Structured Streaming; xây dựng logic đọc dữ liệu từ topic <code>jobs_raw</code> , kiểm tra schema, chuẩn hóa dữ liệu realtime, ghi dữ liệu hợp lệ sang <code>jobs_clean</code> , ghi dữ liệu lỗi sang <code>jobs_dead_letter</code> và tạo các chỉ số realtime phục vụ Elasticsearch/Kibana.
Nguyễn Phúc Huy Hoàng	20235087	Phụ trách Thu thập dữ liệu và Kafka Producer : xây dựng crawler thu thập dữ liệu tuyển dụng từ TopCV, trích xuất thông tin chi tiết của tin tuyển dụng, ghi dữ liệu thô ra định dạng JSONL; triển khai producer đọc JSONL và publish dữ liệu vào Kafka topic <code>jobs_raw</code> để phục vụ speed layer.
Trần Gia Định	20235036	Phụ trách Batch Layer và kiến trúc Medallion : thiết kế luồng xử lý dữ liệu theo các lớp Raw, Bronze, Silver và Gold; triển khai các Spark batch job chuyển đổi dữ liệu từ Raw sang Bronze, Bronze sang Silver và Silver sang Gold; chuẩn hóa dữ liệu nghiệp vụ, xử lý trùng lặp và tổ chức dữ liệu trên HDFS.
Nguyễn Bá Đức Anh	20235004	Phụ trách Trực quan hóa dữ liệu và điều phối workflow với Airflow : triển khai Elasticsearch/Kibana phục vụ lưu trữ, truy vấn và dashboard; xây dựng các biểu đồ phân tích dữ liệu batch và realtime; thiết kế DAG Airflow để điều phối batch pipeline, speed layer bootstrap và real crawler producer trên Kubernetes.
Trần Tuấn Minh	20230051	Phụ trách Mô hình học máy : xây dựng bài toán dự đoán lương, thiết kế đặc trưng đầu vào, huấn luyện mô hình Spark ML sử dụng <code>GBRegressor</code> ; tích hợp mô hình vào batch layer và speed layer để bổ sung các trường lương dự đoán cho những tin tuyển dụng chưa công khai mức lương.

Mặc dù mỗi thành viên có một phần việc chính, các thành phần trong hệ thống có mối liên hệ chặt chẽ với nhau nên nhóm vẫn phối hợp trong quá trình tích hợp, kiểm thử và hoàn thiện báo cáo. Cụ thể, dữ liệu từ crawler và Kafka producer là đầu vào cho speed layer; dữ liệu Raw/Bronze/Silver/Gold là nền tảng cho batch analytics và mô hình học máy; kết quả batch và realtime đều được ghi sang Elasticsearch để Kibana trực quan hóa; Airflow đóng vai trò điều phối toàn bộ quy trình chạy pipeline. Do đó, quá trình phát triển được thực hiện theo hướng vừa phân chia trách nhiệm rõ ràng, vừa phối hợp kiểm tra liên tục để đảm bảo hệ thống hoạt động thống nhất.

Chương 1

GIỚI THIỆU TỔNG QUAN

1.1 Bối cảnh và động lực dự án

Trong bối cảnh chuyển đổi số diễn ra mạnh mẽ, thị trường lao động tại Việt Nam ngày càng phụ thuộc nhiều hơn vào các nền tảng tuyển dụng trực tuyến. Các website như TopCV, VietnamWorks, LinkedIn hay Indeed không chỉ đóng vai trò là nơi kết nối giữa ứng viên và nhà tuyển dụng, mà còn trở thành nguồn dữ liệu quan trọng phản ánh nhu cầu nhân lực, xu hướng kỹ năng, mặt bằng lương và sự phân bố việc làm theo khu vực, ngành nghề, cấp bậc kinh nghiệm. Đối với sinh viên, người lao động, doanh nghiệp và đơn vị đào tạo, việc khai thác hiệu quả nguồn dữ liệu này có thể hỗ trợ ra quyết định trong nhiều bài toán thực tế như định hướng học tập, lựa chọn nghề nghiệp, xây dựng chiến lược tuyển dụng và điều chỉnh chương trình đào tạo.

Tuy nhiên, dữ liệu tuyển dụng trên các nền tảng web thường tồn tại ở dạng bán cấu trúc hoặc phi cấu trúc. Một tin tuyển dụng có thể bao gồm tiêu đề, tên công ty, địa điểm, mức lương, kinh nghiệm, kỹ năng, mô tả công việc, yêu cầu ứng viên, quyền lợi và thông tin metadata khác. Các trường này không phải lúc nào cũng được trình bày thống nhất; nhiều tin thiếu mức lương, có địa điểm ghi theo nhiều cách khác nhau, kỹ năng nằm lẫn trong phần mô tả tự do, hoặc cùng một tin có thể được cập nhật nhiều lần theo thời gian. Vì vậy, nếu chỉ thu thập dữ liệu thủ công hoặc xử lý bằng các script đơn lẻ, rất khó xây dựng được một tập dữ liệu đủ sạch, có khả năng mở rộng và có thể phục vụ phân tích dài hạn.

Dự án **Big Data Job Market** được xây dựng từ nhu cầu giải quyết bài toán trên bằng cách áp dụng các nguyên lý của hệ thống dữ liệu lớn. Thay vì chỉ dừng lại ở việc crawl dữ liệu và lưu file, dự án hướng tới xây dựng một pipeline hoàn chỉnh, bao gồm thu thập dữ liệu, lưu trữ phân tầng, xử lý batch, xử lý streaming, kiểm soát chất lượng dữ liệu, huấn luyện mô hình học máy, lập chỉ mục tìm kiếm và trực quan hóa. Nguồn dữ liệu chính được sử dụng trong phạm vi dự án là các tin tuyển dụng từ TopCV, một nền tảng có lượng thông tin việc làm phong phú và phù hợp để phân tích xu hướng thị trường lao động tại Việt Nam.

Động lực kỹ thuật của dự án là kiểm chứng khả năng kết hợp nhiều công nghệ trong hệ sinh thái Big Data như HDFS, Kafka, Apache Spark, Spark Structured Streaming, Elasticsearch, Kibana, Airflow, Docker và Kubernetes. Các công nghệ này được triển khai theo kiến trúc Lambda Architecture nhằm đáp ứng đồng thời hai yêu cầu: xử lý chính xác dữ liệu lịch sử bằng batch layer và cập nhật thông tin gần thời gian thực bằng speed layer. Bên cạnh đó, mô hình Medallion Architecture được áp dụng để tổ chức dữ liệu theo các lớp Raw, Bronze, Silver và Gold, giúp quá trình làm sạch, chuẩn hóa và khai

thác dữ liệu trở nên có kiểm soát hơn.

1.2 Mục tiêu dự án

Mục tiêu tổng quát của dự án là xây dựng một hệ thống dữ liệu lớn có khả năng thu thập, lưu trữ, xử lý, phân tích và trực quan hóa dữ liệu thị trường lao động từ TopCV. Hệ thống không chỉ phục vụ mục tiêu học thuật trong học phần Lưu trữ và xử lý dữ liệu lớn, mà còn mô phỏng một bài toán thực tế trong doanh nghiệp: biến dữ liệu thô từ web thành dữ liệu phân tích có giá trị.

Về mặt nghiệp vụ, dự án hướng tới các mục tiêu sau:

- Thu thập dữ liệu tin tuyển dụng từ TopCV, bao gồm các thông tin như tiêu đề công việc, công ty, địa điểm, mức lương, kinh nghiệm, kỹ năng, mô tả công việc và thời điểm cập nhật.
- Chuẩn hóa dữ liệu tuyển dụng để giảm sự khác biệt về định dạng giữa các tin, đặc biệt ở các trường như lương, địa điểm, kỹ năng và kinh nghiệm.
- Xây dựng các tập dữ liệu phân tích phục vụ việc quan sát xu hướng thị trường lao động, ví dụ số lượng việc làm theo thời gian, nhóm kỹ năng phổ biến, phân bố địa điểm, ngành nghề và mức lương.
- Hỗ trợ tìm kiếm và trực quan hóa dữ liệu thông qua Elasticsearch và Kibana, giúp người dùng có thể theo dõi kết quả xử lý dưới dạng dashboard.
- Thử nghiệm bài toán dự đoán mức lương cho các tin tuyển dụng không công khai lương hoặc có thông tin lương không đầy đủ, từ đó tăng giá trị khai thác của dữ liệu.

Về mặt kỹ thuật, dự án đặt ra các mục tiêu chính sau:

- Thiết kế pipeline theo **Lambda Architecture**, kết hợp batch layer cho dữ liệu lịch sử và speed layer cho dữ liệu gần thời gian thực.
- Tổ chức dữ liệu theo mô hình **Medallion Architecture** với các lớp Raw, Bronze, Silver và Gold để đảm bảo dữ liệu được xử lý có thứ tự, có khả năng truy vết và dễ mở rộng.
- Sử dụng **HDFS** làm hệ thống lưu trữ phân tán cho batch pipeline, đảm bảo dữ liệu thô và dữ liệu đã xử lý được lưu theo từng tầng.
- Sử dụng **Kafka** làm message broker cho speed layer, giúp tách biệt quá trình sinh dữ liệu, xử lý streaming và ghi kết quả.
- Sử dụng **Apache Spark** và **PySpark** để xử lý dữ liệu batch, làm sạch dữ liệu, chuẩn hóa schema, tạo dữ liệu Gold và huấn luyện mô hình Spark ML.
- Sử dụng **Spark Structured Streaming** để xử lý luồng dữ liệu realtime, kiểm tra schema, loại bỏ bản ghi lỗi, ghi dữ liệu hợp lệ và tạo các bảng tổng hợp theo thời gian.

- Sử dụng **Elasticsearch** làm serving layer cho dữ liệu đã xử lý, đồng thời sử dụng **Kibana** để trực quan hóa và phân tích.
- Sử dụng **Apache Airflow** để điều phối pipeline, kiểm tra phụ thuộc giữa các bước và tự động hóa quá trình chạy batch hoặc speed pipeline.
- Triển khai hệ thống trên môi trường **Docker** và **Kubernetes/Minikube**, giúp các thành phần hạ tầng được đóng gói, cô lập và có khả năng quản lý tập trung.

1.3 Phạm vi dự án và công nghệ sử dụng

Trong phạm vi báo cáo này, hệ thống tập trung vào dữ liệu tuyển dụng từ TopCV. Dữ liệu được thu thập phục vụ mục tiêu nghiên cứu, thử nghiệm pipeline và phân tích xu hướng thị trường lao động. Dự án chưa đặt mục tiêu xây dựng một sản phẩm thương mại hoàn chỉnh cho người dùng cuối, mà tập trung vào thiết kế kiến trúc, triển khai pipeline dữ liệu lớn, đảm bảo chất lượng dữ liệu và chứng minh khả năng vận hành của hệ thống trên môi trường cục bộ.

Phạm vi chức năng của dự án bao gồm các nhóm công việc chính:

- **Thu thập dữ liệu:** xây dựng crawler để lấy dữ liệu tuyển dụng từ TopCV; hỗ trợ cả chế độ batch crawler và speed crawler ghi dữ liệu ra JSONL.
- **Lưu trữ dữ liệu:** tổ chức dữ liệu trong HDFS theo các lớp Raw, Bronze, Silver và Gold; dữ liệu thô được giữ lại để phục vụ tái xử lý khi logic làm sạch thay đổi.
- **Xử lý batch:** triển khai các Spark job chuyển dữ liệu từ Raw sang Bronze, Bronze sang Silver, huấn luyện mô hình dự đoán lương, tạo Gold layer và ghi dữ liệu Gold sang Elasticsearch.
- **Xử lý streaming:** sử dụng Kafka và Spark Structured Streaming để xử lý dữ liệu gần thời gian thực, phân tách bản ghi hợp lệ và bản ghi lỗi, đồng thời sinh các chỉ số realtime.
- **Học máy:** sử dụng Spark ML để huấn luyện mô hình dự đoán mức lương dựa trên các đặc trưng đã chuẩn hóa từ dữ liệu tuyển dụng.
- **Serving và trực quan hóa:** ghi dữ liệu phân tích sang Elasticsearch và xây dựng dashboard trên Kibana để quan sát kết quả.
- **Điều phối và triển khai:** sử dụng Airflow để điều phối workflow; sử dụng Docker và Kubernetes/Minikube để triển khai các thành phần hạ tầng.

Các công nghệ chính được sử dụng trong dự án được tóm tắt trong Bảng 1.1.

Bảng 1.1: Các công nghệ chính sử dụng trong dự án

Nhóm công nghệ	Công nghệ	Vai trò trong hệ thống
Thu thập dữ liệu	Python, crawler, JSONL	Thu thập dữ liệu tuyển dụng từ TopCV, chuẩn bị dữ liệu đầu vào cho batch và speed pipeline.
Lưu trữ phân tán	HDFS	Lưu dữ liệu theo các lớp Raw, Bronze, Silver, Gold và lưu artifact của mô hình học máy.
Message broker	Kafka, Strimzi Kafka	Truyền dữ liệu realtime qua các topic như <code>jobs_raw</code> , <code>jobs_clean</code> , <code>jobs_dead_letter</code> .
Xử lý dữ liệu	Apache Spark, PySpark	Xử lý batch ETL, chuẩn hóa dữ liệu, tạo dữ liệu phân tích và ghi kết quả sang serving layer.
Xử lý streaming	Spark Structured Streaming	Xử lý dữ liệu gần thời gian thực, kiểm tra schema, deduplication, checkpoint và tạo realtime indexes.
Học máy	Spark ML, GBTRegressor	Huấn luyện mô hình dự đoán mức lương cho tin tuyển dụng.
Serving layer	Elasticsearch	Lưu trữ dữ liệu đã xử lý để phục vụ tìm kiếm, truy vấn và phân tích.
Trực quan hóa	Kibana	Xây dựng dashboard theo dõi xu hướng việc làm, kỹ năng, lương và dữ liệu realtime.
Điều phối workflow	Apache Airflow	Lên lịch, điều phối, kiểm tra trạng thái và quản lý phụ thuộc giữa các bước pipeline.
Triển khai hạ tầng	Docker, Kubernetes, Minikube	Đóng gói và triển khai các thành phần hệ thống trong môi trường container hóa.

Nhìn chung, Chương 1 trình bày bối cảnh hình thành đề tài, mục tiêu nghiệp vụ và mục tiêu kỹ thuật của hệ thống, đồng thời xác định phạm vi triển khai cũng như các công nghệ chủ đạo. Các chương tiếp theo sẽ đi sâu vào kiến trúc tổng thể, thiết kế dữ liệu, pipeline batch, pipeline streaming, mô hình học máy, serving layer, triển khai hạ tầng và những bài học kinh nghiệm rút ra trong quá trình thực hiện dự án.

Chương 2

KIẾN TRÚC HỆ THỐNG

2.1 Tổng quan Lambda Architecture

Hệ thống Big Data Job Market được thiết kế theo Lambda Architecture nhằm kết hợp khả năng xử lý chính xác dữ liệu lịch sử với khả năng cập nhật dữ liệu gần thời gian thực. Kiến trúc của hệ thống gồm ba thành phần chính: Batch Layer, Speed Layer và Serving Layer.

Batch Layer chịu trách nhiệm lưu trữ và xử lý dữ liệu tuyển dụng theo từng đợt. Dữ liệu được crawler thu thập từ TopCV, lưu vào HDFS và lần lượt được xử lý qua các lớp Raw, Bronze, Silver và Gold của kiến trúc Medallion. Luồng batch ưu tiên tính đầy đủ, khả năng tái xử lý và độ chính xác của dữ liệu lịch sử.

Luồng xử lý batch của hệ thống được tổ chức như sau:

```
TopCV → Batch Crawler → HDFS Raw → Raw to Bronze → Bronze to Silver  
→ Train Salary Model → Silver to Gold → Gold to Elasticsearch →  
Kibana
```

Speed Layer chịu trách nhiệm xử lý các tin tuyển dụng mới hoặc vừa được cập nhật với độ trễ thấp. Crawler ở chế độ speed ghi dữ liệu ra file JSONL, producer publish từng bản ghi vào Kafka và Spark Structured Streaming thực hiện kiểm tra schema, chuẩn hóa, loại bỏ trùng lặp và tạo các chỉ số gần thời gian thực.

Luồng xử lý speed layer được tổ chức như sau:

```
TopCV Speed Crawler → JSONL Producer → Kafka jobs_raw  
→ Spark Structured Streaming → Kafka jobs_clean/jobs_dead_letter  
→ Elasticsearch realtime indexes → Kibana
```

Serving Layer sử dụng Elasticsearch để lưu trữ dữ liệu đã xử lý từ cả batch layer và speed layer. Kibana kết nối với Elasticsearch để cung cấp các dashboard phân tích thị trường lao động, bao gồm nhu cầu tuyển dụng, kỹ năng, địa điểm, mức lương và các chỉ số cập nhật gần thời gian thực.

Việc kết hợp hai luồng xử lý giúp hệ thống vừa duy trì được tập dữ liệu lịch sử đầy đủ và có thể tái xử lý, vừa phản ánh nhanh các biến động mới của thị trường tuyển dụng.

2.2 Mô hình triển khai

2.2.1 Tổng quan mô hình triển khai

Dự án được triển khai theo hướng container hóa trên Kubernetes sử dụng Minikube. Các thành phần của hệ thống được đóng gói thành container và phân tách theo từng nhóm chức năng, bao gồm HDFS, Kafka, Spark, Elasticsearch, Kibana và Airflow.

HDFS được sử dụng làm hệ thống lưu trữ phân tán cho dữ liệu batch và artifact của mô hình học máy. Kafka đóng vai trò message broker cho speed layer. Apache Spark thực hiện các tác vụ xử lý batch, Spark Structured Streaming và huấn luyện mô hình dự đoán lương. Elasticsearch đảm nhiệm vai trò serving layer, trong khi Kibana cung cấp giao diện trực quan hóa dữ liệu.

Airflow không trực tiếp xử lý dữ liệu mà thực hiện điều phối workflow. Với batch pipeline, Airflow kiểm tra trạng thái HDFS và Elasticsearch, sau đó tạo các Kubernetes Job từ những CronJob template đã được cấu hình trong namespace **spark**. Các task được thực hiện tuần tự từ crawl dữ liệu, xử lý Raw–Bronze–Silver–Gold, huấn luyện mô hình đến ghi dữ liệu sang Elasticsearch.

Đối với speed layer, Airflow kiểm tra trạng thái Kafka và Elasticsearch, đảm bảo các Kafka topic cần thiết đã tồn tại, khởi động Spark Structured Streaming và điều phối real crawler producer. Cách triển khai này giúp các thành phần được quản lý tập trung, dễ theo dõi trạng thái, log và tài nguyên trên Kubernetes.

2.2.2 Phân chia namespace

Hệ thống được triển khai trên Kubernetes/Minikube và được chia thành các namespace theo từng nhóm chức năng. Các namespace chính gồm **hdfs**, **kafka**, **spark**, **search** và **airflow**. Trong đó, namespace **hdfs** được tạo trong manifest triển khai HDFS, còn các namespace còn lại được định nghĩa trong nhóm manifest khởi tạo namespace của hệ thống.

Việc phân chia namespace giúp tách biệt các thành phần hạ tầng, dễ theo dõi trạng thái pod, service, persistent volume, log và quyền truy cập. Đồng thời, cách tổ chức này cũng giúp Airflow dễ điều phối các job xử lý dữ liệu thông qua Kubernetes.

Bảng 2.1: Các namespace chính trong mô hình triển khai

Namespace	Thành phần chính	Vai trò
hdfs	HDFS NameNode, HDFS DataNode, Service, PersistentVolumeClaim	Lưu trữ dữ liệu phân tán cho batch layer, bao gồm các lớp Raw, Bronze, Silver, Gold và thư mục lưu mô hình dự đoán lương. Namespace này được khởi tạo trực tiếp trong manifest HDFS.
kafka	Strimzi Kafka cluster, KafkaNodePool, KafkaTopic	Đóng vai trò message broker cho speed layer. Kafka cluster có tên my-cluster ; các topic chính gồm jobs_raw , jobs_clean và jobs_dead_letter .
spark	Spark batch CronJob, Spark streaming Job, Spark ML training CronJob, RBAC, checkpoint PVC	Thực hiện xử lý dữ liệu phân tán cho cả batch layer và speed layer. Namespace này chứa các job ETL như Raw to Bronze, Bronze to Silver, Silver to Gold, Gold to Elasticsearch, job huấn luyện mô hình dự đoán lương và job Spark Structured Streaming.
search	Elasticsearch StatefulSet, Elasticsearch Service, Kibana Deployment, Kibana Service	Lưu trữ dữ liệu đã xử lý và phục vụ phân tích. Elasticsearch lưu batch index gold-jobs-flat và các realtime index; Kibana kết nối tới Elasticsearch để xây dựng dashboard trực quan hóa dữ liệu thị trường lao động.
airflow	Airflow Webserver, Airflow Scheduler, PostgreSQL metadata database, ServiceAccount, RBAC, DAGs	Điều phối các pipeline xử lý dữ liệu. Airflow sử dụng LocalExecutor , lưu metadata trong PostgreSQL, chạy các DAG batch và speed layer, đồng thời có quyền tạo, theo dõi và xóa Kubernetes Job trong namespace spark , cũng như kiểm tra Kafka resource trong namespace kafka .

2.2.3 Cấu hình triển khai thực nghiệm

Hệ thống được triển khai trên cụm Kubernetes single-node sử dụng Minikube. Cụm gồm một node có tên `job-market`, đồng thời đảm nhiệm vai trò control plane và thực thi các workload ứng dụng. Node sử dụng Kubernetes `v1.35.1`, hệ điều hành Debian GNU/Linux 12 và container runtime Docker `29.2.1`.

Node được cấp 8 CPU, khoảng 10,7 GiB bộ nhớ và giới hạn tối đa 110 pod. Do toàn bộ dịch vụ hạ tầng và các job xử lý dữ liệu cùng được lập lịch trên một node, cấu hình này phù hợp với mục tiêu phát triển, thử nghiệm cục bộ và trình diễn pipeline. Tuy nhiên, mô hình single-node chưa cung cấp khả năng chịu lỗi ở cấp độ node và không phản ánh đầy đủ khả năng mở rộng của một cụm Kubernetes nhiều node.

```
Node       : job-market
Kubernetes : v1.35.1
OS         : Debian GNU/Linux 12 (bookworm)
Kernel     : 6.6.87.2-microsoft-standard-WSL2
Runtime    : docker://29.2.1
CPU        : 8
Memory     : 11217112Ki
MaxPods    : 110
```

Hình 2.1: Cấu hình node của cụm Kubernetes/Minikube thực nghiệm.

Các thành phần của hệ thống được tách thành nhiều workload độc lập trong các namespace `airflow`, `hdfs`, `kafka`, `search` và `spark`. Tùy theo đặc điểm vận hành, chúng được triển khai dưới dạng Deployment, StatefulSet, CronJob hoặc Job.

Các dịch vụ hạ tầng như Airflow, HDFS, Kafka, Elasticsearch và Kibana được duy trì lâu dài. Các bước xử lý batch được khai báo dưới dạng CronJob template và được Airflow tạo thành Kubernetes Job khi pipeline được kích hoạt. Vì vậy, các container trong danh sách cấu hình không nhất thiết chạy đồng thời tại cùng một thời điểm.

Namespace	Type	Workload	Container
airflow	Deployment	airflow-postgres	postgres
airflow	Deployment	airflow-scheduler	airflow-scheduler
airflow	Deployment	airflow-webserver	airflow-webserver
hdfs	StatefulSet	hdfs-datanode	datanode
hdfs	StatefulSet	hdfs-namenode	namenode
kafka	Deployment	my-cluster-entity-operator	topic-operator
kafka	Deployment	my-cluster-entity-operator	user-operator
kafka	Deployment	strimzi-cluster-operator	strimzi-cluster-operator
kafka	StrimziPod	my-cluster-dual-role-0	kafka
search	Deployment	kibana	kibana
search	StatefulSet	elasticsearch	elasticsearch
spark	CronJob	batch-etl-bronze-to-silver	spark-submit
spark	CronJob	batch-etl-crawl-jobs	batch-crawler
spark	CronJob	batch-etl-gold-to-elasticsearch	spark-submit
spark	CronJob	batch-etl-raw-to-bronze	spark-submit
spark	CronJob	batch-etl-silver-to-gold	spark-submit
spark	CronJob	batch-etl-train-salary-model	spark-submit
spark	Job	speed-real-crawler-producer	speed-crawler-producer
spark	Job	speed-stream-es-submit	spark-submit

PS F:\IT4931- Lưu trữ và xử lý dữ liệu lớn\bigdata-job-market> █

Hình 2.2: Các workload và container được cấu hình trong hệ thống Kubernetes.

Các workload trong hệ thống sử dụng các container image được đóng gói theo từng nhóm chức năng. Airflow sử dụng image tùy chỉnh `job-market-airflow:2.9.3`; HDFS sử dụng bộ image Hadoop 3.2.1; Kafka được triển khai bằng Strimzi với Kafka 4.1.0; Elasticsearch và Kibana cùng sử dụng phiên bản 8.13.4.

Các tác vụ xử lý batch và speed layer sử dụng chung image tùy chỉnh `spark-job-market:latest`. Image này chứa mã nguồn crawler, các Spark batch job, Spark Structured Streaming và các thư viện phụ thuộc của dự án. Việc sử dụng chung image giúp đảm bảo các bước xử lý được thực thi trong cùng một môi trường phần mềm.

Namespace	Workload	Image
airflow	airflow-postgres	postgres:15-alpine
airflow	airflow-scheduler	job-market-airflow:2.9.3
airflow	airflow-webserver	job-market-airflow:2.9.3
hdfs	hdfs-datanode	bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8
hdfs	hdfs-namenode	bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8
kafka	my-cluster-dual-role-0	quay.io/strimzi/kafka:1.0.0-kafka-4.1.0
kafka	my-cluster-entity-operator	quay.io/strimzi/operator:1.0.0
kafka	strimzi-cluster-operator	quay.io/strimzi/operator:1.0.0
search	elasticsearch	docker.elastic.co/elasticsearch/elasticsearch:8.13.4
search	kibana	docker.elastic.co/kibana/kibana:8.13.4
spark	batch-etl-bronze-to-silver	spark-job-market:latest
spark	batch-etl-crawl-jobs	spark-job-market:latest
spark	batch-etl-gold-to-elasticsearch	spark-job-market:latest
spark	batch-etl-raw-to-bronze	spark-job-market:latest
spark	batch-etl-silver-to-gold	spark-job-market:latest
spark	batch-etl-train-salary-model	spark-job-market:latest
spark	speed-real-crawler-producer	spark-job-market:latest
spark	speed-stream-es-submit	spark-job-market:latest

PS F:\IT4931- Lưu trữ và xử lý dữ liệu lớn\bigdata-job-market> █

Hình 2.3: Các container image được sử dụng bởi các workload của hệ thống.

2.3 Cấu trúc mã nguồn

Repository của dự án có tên `bigdata-job-market`. Mã nguồn được tổ chức theo hướng tách biệt giữa các tầng chức năng chính của hệ thống Big Data, bao gồm tầng thu thập dữ liệu, tầng xử lý batch, tầng xử lý streaming, tầng học máy, tầng triển khai hạ tầng và các thành phần dùng chung. Cách tổ chức này giúp nhóm phát triển độc lập từng module, đồng thời phản ánh trực tiếp kiến trúc Lambda Architecture của hệ thống.

Cấu trúc thư mục chính của repository như sau:

```
bigdata-job-market/
|-- apps/
|   |-- batch/
|   |   '-- jobs/
|   |       |-- raw_to_bronze.py
|   |       |-- bronze_to_silver.py
|   |       |-- train_salary_model.py
|   |       |-- silver_to_gold.py
|   |       '-- gold_to_elasticsearch.py
|   |-- ingestion/
|   |   |-- run_crawler.py
|   |   |-- run_crawler_to_hdfs.py
|   |   '-- topcv_crawler.py
|   |-- producer/
```



```

|   |   '-- crawler_jsonl_producer.py
|   |
|   |-- stream_etl/
|   |   |-- stream_main.py
|   |   |-- transform.py
|   |   |-- normalizers.py
|   |   |-- schemas/
|   |   |-- sinks/
|   |   '-- stateful_jobs/
|   |
|   '-- ml/
|       '-- salary_prediction.py
|
|-- configs/
|
|-- data/
|   |-- raw/
|   |-- bronze/
|   |-- silver/
|   '-- gold/
|
|-- infra/
|   |-- airflow/
|   |-- docker-compose/
|   |-- hdfs/
|   |-- kafka/
|   |-- kibana/
|   |-- namespaces/
|   |-- producer/
|   |-- search/
|   '-- spark/
|
|-- shared/
|   '-- quality/
|       '-- streaming_quality_rules.py
|
|-- docs/
|-- scripts/
|-- tests/
|-- Makefile
|-- pyproject.toml
|-- uv.lock
'-- README.md

```

2.3.1 Thư mục apps/

Thư mục `apps/` chứa mã nguồn chính của hệ thống. Đây là nơi triển khai các thành phần nghiệp vụ như crawler, Spark batch job, Spark streaming job, Kafka producer, mô

hình học máy và các module dùng chung ở tầng ứng dụng.

Thư mục apps/ingestion/

Thư mục `apps/ingestion/` phụ trách quá trình thu thập dữ liệu từ TopCV. Trong đó, `topcv_crawler.py` là module chứa logic crawl chính, bao gồm quét danh sách tin tuyển dụng, lấy đường dẫn chi tiết, parse nội dung HTML, trích xuất thông tin việc làm và ghi kết quả ra file JSONL.

File `run_crawler.py` đóng vai trò entrypoint dòng lệnh cho crawler. File này cho phép chạy crawler theo hai chế độ `batch` và `speed`. Khi chạy, `run_crawler.py` gọi logic crawl trong `topcv_crawler.py` để thực hiện quá trình thu thập dữ liệu.

Với batch pipeline trên Kubernetes, `run_crawler_to_hdfs.py` được sử dụng làm entrypoint. File này chạy crawler ở chế độ batch, sau đó copy file JSONL đầu ra lên HDFS tại Raw layer. Luồng thu thập dữ liệu batch được mô tả như sau:

```
run_crawler_to_hdfs.py → run_crawler.py → topcv_crawler.py → JSONL local  
                        → HDFS Raw
```

Với speed layer, `run_crawler.py` chạy ở chế độ speed để ghi dữ liệu crawl được ra file JSONL. Sau đó, producer trong `apps/producer/` đọc file JSONL này và publish từng bản ghi vào Kafka topic `jobs_raw`.

Thư mục apps/producer/

Thư mục `apps/producer/` chứa Kafka producer phục vụ speed layer. Trong luồng triển khai chính, `crawler_jsonl_producer.py` được dùng để đọc file JSONL do crawler sinh ra và publish từng bản ghi vào Kafka topic `jobs_raw`. Cách tổ chức này giúp tách crawler và producer thành hai tiến trình độc lập: crawler chịu trách nhiệm lấy dữ liệu từ TopCV, còn producer chịu trách nhiệm đưa dữ liệu vào Kafka.

Thư mục apps/batch/jobs/

Thư mục `apps/batch/jobs/` chứa các Spark job xử lý batch. Đây là phần lõi của batch layer, chịu trách nhiệm xử lý dữ liệu theo kiến trúc Medallion từ Raw layer đến Bronze, Silver và Gold layer.

Các job chính gồm:

- `raw_to_bronze.py`: đọc dữ liệu thô từ HDFS Raw layer, chuẩn hóa cấu trúc ban đầu và ghi sang Bronze layer dưới định dạng Parquet.
- `bronze_to_silver.py`: làm sạch dữ liệu, chuẩn hóa các trường như mức lương, địa điểm, kinh nghiệm, kỹ năng và loại bỏ bản ghi lỗi hoặc trùng lặp.
- `train_salary_model.py`: huấn luyện mô hình dự đoán lương dựa trên dữ liệu đã được chuẩn hóa ở Silver layer.
- `silver_to_gold.py`: tạo Gold layer dạng denormalized, bổ sung các trường phục vụ phân tích, dashboard và dự đoán lương.
- `gold_to_elasticsearch.py`: ghi dữ liệu Gold sang Elasticsearch index `gold-jobs-flat`.

Luồng batch chính của hệ thống là:

Raw → Bronze → Silver → Train Salary Model → Gold → Elasticsearch

Thư mục apps/stream_etl/

Thư mục apps/stream_etl/ chứa mã nguồn của speed layer. File trung tâm là `stream_main.py`, thực hiện đọc dữ liệu từ Kafka topic `jobs_raw`, validate schema, chuẩn hóa dữ liệu, ghi dữ liệu hợp lệ sang topic `jobs_clean`, ghi dữ liệu lỗi sang topic `jobs_dead_letter` và ghi kết quả realtime sang Elasticsearch.

Các thành phần chính gồm:

- `stream_main.py`: entrypoint chính của Spark Structured Streaming job.
- `transform.py`: parse dữ liệu, validate bản ghi và tách dữ liệu hợp lệ/lỗi.
- `normalizers.py`: chuẩn hóa địa điểm, mức lương, kỹ năng và kinh nghiệm.
- `schemas/`: định nghĩa schema cho dữ liệu đầu vào.
- `sinks/`: chứa logic ghi dữ liệu sang Kafka và Elasticsearch.
- `stateful_jobs/`: chứa các job tổng hợp realtime theo cửa sổ thời gian.

Một số bảng tổng hợp realtime được ghi sang Elasticsearch gồm thống kê số lượng việc làm theo 10 phút, kỹ năng phổ biến theo giờ, top kỹ năng theo giờ và phân nhóm mức lương theo giờ.

Thư mục apps/ml/

Thư mục apps/ml/ chứa mã nguồn phục vụ bài toán dự đoán lương, trong đó file chính là `salary_prediction.py`. Module này được sử dụng trong quá trình huấn luyện mô hình ở batch layer và được tái sử dụng trong các bước xử lý cần bổ sung thông tin lương dự đoán.

2.3.2 Thư mục configs/

Thư mục `configs/` chứa các cấu hình dùng chung cho hệ thống, bao gồm đường dẫn HDFS, cấu hình logging, biến môi trường mẫu và các dependency cần thiết. Một số đường dẫn quan trọng được cấu hình tại đây gồm Raw path, Bronze path, Silver path, Gold path và đường dẫn lưu mô hình dự đoán lương.

Việc tập trung cấu hình vào một thư mục riêng giúp quá trình triển khai dễ bảo trì hơn, tránh việc hard-code tham số trong nhiều file xử lý khác nhau.

2.3.3 Thư mục data/

Thư mục `data/` mô tả các lớp dữ liệu trong kiến trúc Medallion. Trong môi trường triển khai, dữ liệu chính được lưu trên HDFS; tuy nhiên, thư mục `data/` vẫn được dùng để lưu data contract, dữ liệu mẫu và tài liệu mô tả định dạng dữ liệu.

Các thư mục con gồm:

- `data/raw/`: mô tả dữ liệu thô thu thập từ TopCV.
- `data/bronze/`: mô tả dữ liệu đã được chuẩn hóa cấu trúc ban đầu.
- `data/silver/`: mô tả dữ liệu đã được làm sạch và chuẩn hóa nghiệp vụ.
- `data/gold/`: mô tả dữ liệu đã được tổng hợp, làm giàu và sẵn sàng phục vụ phân tích.

2.3.4 Thư mục `infra/`

Thư mục `infra/` chứa toàn bộ cấu hình triển khai hệ thống trên Kubernetes/Minikube. Đây là thư mục dùng để khởi tạo các thành phần hạ tầng như HDFS, Kafka, Spark, Elasticsearch, Kibana và Airflow.

Các nhóm manifest chính gồm:

- `infra/hdfs/`: triển khai HDFS NameNode, DataNode, Service và PersistentVolumeClaim.
- `infra/kafka/`: triển khai Kafka cluster bằng Strimzi và các topic `jobs_raw`, `jobs_clean`, `jobs_dead_letter`.
- `infra/spark/`: chứa Dockerfile, RBAC, các Spark batch CronJob, job huấn luyện mô hình và Spark Structured Streaming job.
- `infra/search/`: triển khai Elasticsearch và các service liên quan.
- `infra/kibana/`: lưu các saved objects phục vụ dashboard Kibana.
- `infra/airflow/`: triển khai Airflow Webserver, Scheduler, PostgreSQL metadata database, RBAC và các DAG điều phối pipeline.
- `infra/producer/`: chứa manifest triển khai real crawler producer cho speed layer.

2.3.5 Thư mục `shared/`

Thư mục `shared/` chứa rule kiểm tra chất lượng dữ liệu dùng trong speed layer. Trong đó, `shared/quality/streaming_quality_rules.py` được sử dụng để gắn nhãn lỗi cho các bản ghi Kafka không hợp lệ trước khi dữ liệu được ghi sang topic `jobs_clean` hoặc `jobs_dead_letter`. Thành phần này giúp tách riêng logic kiểm tra chất lượng dữ liệu khỏi logic streaming chính trong `apps/stream_etl/`.

2.3.6 Thư mục `docs/`, `scripts/` và `tests/`

Thư mục `docs/` chứa tài liệu hướng dẫn triển khai và vận hành hệ thống. Thư mục `scripts/` chứa các script hỗ trợ chạy pipeline, kiểm tra trạng thái service và thao tác với HDFS/Kafka/Spark. Thư mục `tests/` chứa các test phục vụ kiểm tra một số thành phần quan trọng của pipeline.

2.3.7 Các file cấu hình ở thư mục gốc

Ở thư mục gốc của repository có một số file quan trọng:

- `README.md`: mô tả tổng quan project, kiến trúc hệ thống, data flow và cách chạy các thành phần chính.
- `Makefile`: gom các lệnh triển khai và vận hành thường dùng.
- `pyproject.toml`: cấu hình project Python và dependency.
- `uv.lock`: khóa phiên bản dependency để đảm bảo môi trường cài đặt ổn định và tái lập được.

Nhìn chung, cấu trúc repository phản ánh rõ kiến trúc hệ thống Big Data. Thư mục `apps/` chứa logic xử lý chính, `infra/` chứa cấu hình triển khai, `data/` mô tả các lớp dữ liệu, `shared/` chứa logic kiểm tra chất lượng dữ liệu dùng chung, còn `docs/`, `scripts/` và `tests/` hỗ trợ vận hành và kiểm thử hệ thống.

Chương 3

THIẾT KẾ DỮ LIỆU: KIẾN TRÚC MEDALLION

Để xây dựng một hệ thống phân tích dữ liệu thị trường lao động từ nền tảng tuyển dụng TopCV có độ tin cậy cao, khả năng chịu lỗi tốt và sẵn sàng mở rộng, hệ thống áp dụng **Kiến trúc Medallion** trong tầng Batch Layer thuộc kiến trúc Lambda.

Kiến trúc Medallion tổ chức dữ liệu qua các lớp xử lý tuần tự từ thô đến tinh lọc bao gồm: Raw, Bronze, Silver, và Gold. Việc phân lớp này giúp bảo toàn dữ liệu gốc, tách biệt các giai đoạn làm sạch, chuẩn hóa và tối ưu hóa dữ liệu phục vụ cho các mục đích nghiệp vụ khác nhau.

Hình 3.1 minh họa luồng dữ liệu chính trong kiến trúc Medallion, từ bước thu thập dữ liệu thô đến khi tạo lớp dữ liệu phục vụ phân tích và trực quan hóa.



Hình 3.1: Luồng di chuyển và xử lý dữ liệu qua các tầng Medallion

3.1 Tổng quan 4 lớp dữ liệu

Mỗi lớp trong kiến trúc Medallion chịu trách nhiệm giải quyết một bài toán cụ thể trong chuỗi cung ứng dữ liệu của dự án, cụ thể như sau:

- **Lớp dữ liệu thô (Raw):** Lưu trữ nguyên trạng dữ liệu thu được từ crawler dưới định dạng bán cấu trúc JSONL. Lớp này đảm bảo tính bất biến, không cho phép chỉnh sửa dữ liệu gốc để phục vụ mục đích tính toán lại khi có sự thay đổi về thuật toán xử lý downstream.
- **Lớp dữ liệu đồng (Bronze):** Là bước đầu tiên chuyển đổi sang định dạng cột Parquet có cấu trúc rõ ràng. Dữ liệu được trích xuất từ payload thô, chuyển đổi các trường thời gian cơ bản, và làm phẳng các cấu trúc lồng nhau đơn giản mà không làm thay đổi ngữ nghĩa nghiệp vụ của dữ liệu gốc.
- **Lớp dữ liệu bạc (Silver):** Đảm nhận vai trò chuẩn hóa nghiệp vụ toàn doanh nghiệp. Tại đây dữ liệu lương, vị trí địa lý, kỹ năng và thời gian làm việc được đồng bộ hóa về một định dạng chuẩn chung. Đồng thời, chính sách loại bỏ trùng lặp

ng nghiệp vụ được áp dụng để đảm bảo mỗi công việc duy nhất chỉ tồn tại một bản ghi cập nhật mới nhất.

- **Lớp dữ liệu vàng (Gold):** Là tầng phục vụ nghiệp vụ tối ưu cho việc truy vấn của ứng dụng API và hiển thị Dashboard trực quan. Dữ liệu được phi chuẩn hóa hoàn toàn bằng cách tích hợp cả dữ liệu tin tuyển dụng và dữ liệu doanh nghiệp nhằm tăng tốc độ truy vấn thay vì tối ưu lưu trữ, đồng thời được làm giàu thông qua kết quả dự đoán từ mô hình Học máy.

3.2 Lớp dữ liệu thô (Raw Layer)

Dữ liệu thô thu được từ crawler được đẩy trực tiếp lên HDFS tại thư mục `/raw/jobs/ingest_date=YYYY-MM-DD/`. Mỗi dòng trong tệp JSONL đại diện cho một tài liệu tin tuyển dụng hoàn chỉnh, phản ánh chính xác cấu trúc dữ liệu trả về từ website TopCV.

Cấu trúc được phân rã thành ba phần chính: cấp root dữ liệu, khối dữ liệu nghiệp vụ `payload`, và khối kiểm soát chất lượng `quality_flags`.

Cấu trúc cấp ngoài cùng (Root Level)

Cấp ngoài cùng chứa các trường metadata mang tính kỹ thuật của hệ thống thu thập, giúp quản lý nguồn gốc, phiên bản và thời điểm cào dữ liệu.

Bảng 3.1: Đặc tả cấu trúc cấp ngoài cùng (Root Schema v1.0)

Tên trường	Kiểu dữ liệu	Bắt buộc	Mô tả chi tiết
<code>source</code>	String	Có	Tên nguồn thu thập dữ liệu (Mặc định: "topcv").
<code>source_url</code>	String	Có	URL gốc của bài đăng tuyển dụng trên TopCV.
<code>normalized_source_url</code>	String	Có	URL đã được chuẩn hóa.
<code>crawl_version</code>	Integer	Có	Phiên bản của crawler.
<code>ingest_ts</code>	Long (Unix ms)	Có	Thời gian lúc crawler bóc tách dữ liệu thành công.
<code>event_ts</code>	Long (Unix ms)	Không	Thời điểm đăng bài viết lấy từ cấu trúc JSON-LD.
<code>job_id</code>	String	Có	Khóa chính duy nhất, bằng SHA256(<code>source</code> + <code>normalized_source_url</code>).
<code>hash_content</code>	String	Có	SHA256 nội dung cốt lõi của bài tuyển dụng để phát hiện cập nhật.
<code>payload</code>	Object	Có	Khối đối tượng chứa toàn bộ dữ liệu nghiệp vụ chính.
<code>quality_flags</code>	Object	Có	Khối đối tượng chứa các cờ boolean đánh giá chất lượng.

Cấu trúc khối payload

Khối payload chứa thông tin chi tiết về công việc và công ty tuyển dụng đã được cào về từ cấu trúc trang HTML hoặc thẻ script JSON-LD của bài tuyển dụng. Cụ thể thông tin các trường nằm trong khối payload được mô tả trong Bảng 3.2.

Cấu trúc khối Quality Flags

Nhằm phục vụ việc giám sát chất lượng dữ liệu, hệ thống định nghĩa các cờ kiểm định tự động dạng Boolean cho mỗi bản ghi thu thập được, thông tin cụ thể được mô tả ở Bảng 3.3.

3.3 Lớp dữ liệu đồng (Bronze Layer)

Lớp Bronze là kết quả đầu tiên của quá trình biến đổi dữ liệu thô bán cấu trúc thành cấu trúc cột lưu trữ tối ưu dưới định dạng Parquet. Tập tin được ghi vào đường dẫn HDFS: `/bronze/jobs/ingest_date=YYYY-MM-DD/`. Thông tin chi tiết về các trường được tạo thêm trong lớp dữ liệu Bronze (so với Raw) được mô tả tại Bảng 3.4.

Nguyên tắc thiết kế

- **Không áp dụng các quy tắc nghiệp vụ:** Lớp Bronze tuân thủ nguyên tắc không can thiệp vào ngữ nghĩa nghiệp vụ. Dữ liệu giữ nguyên giá trị thô và chỉ thay đổi cấu trúc kỹ thuật (kiểu dữ liệu, định dạng file).
- **Lưu trữ định dạng kiểu tối ưu:** Ép kiểu các trường thời gian dạng Unix ms về định dạng Timestamp chuẩn của Spark.
- **Giữ nguyên tên gốc:** Tên các trường kế thừa từ Raw được bảo toàn 100%, không đổi tên.
- **Không parse lồng phức tạp:** Khối `json_ld` gốc được trích xuất thành một chuỗi JSON thô (`StringType`) thay vì parse thành cấu trúc nested struct phức tạp ngay tại đây.

Các chuyển đổi kỹ thuật chính từ Raw lên Bronze

1. **Làm phẳng thông tin doanh nghiệp:** Trường lồng `company_details` dạng Object được bóc tách và làm phẳng trực tiếp thành ba cột độc lập tại Bronze: `company_scale`, `company_field`, và `company_address`.
2. **Chuyển đổi kiểu dữ liệu thời gian:** Các trường thời gian dạng Unix Epoch Millisecond (`ingest_ts`, `event_ts`, `deadline`) được cast sang định dạng Spark `TimestampType`.
3. **monthOfExperience được giữ ở dạng String:** Vì trong dữ liệu thô, trường này chứa cả số lượng tháng (Integer) lẫn các chuỗi ký tự tự do khác (như "Không yêu cầu"), hệ thống đọc toàn bộ dưới kiểu `StringType` để tránh mất mát dữ liệu do ép kiểu lỗi.

Bảng 3.2: Đặc tả cấu trúc khối payload thô

Tên trường	Kiểu dữ liệu	Mô tả chi tiết
title	String	Tiêu đề của vị trí tuyển dụng.
company_name	String	Tên của doanh nghiệp tuyển dụng.
company_details	Object	Đối tượng chứa thông tin quy mô, lĩnh vực, địa chỉ của công ty.
company_details.scale	String	Quy mô nhân sự của công ty.
company_details.field	String	Lĩnh vực hoạt động của công ty.
company_details.address	String	Địa chỉ chi tiết của công ty.
salary	String	Chuỗi mô tả lương (Ví dụ: "10 - 15 Triệu", "Thỏa thuận").
location	Array[String]	Danh sách các địa điểm làm việc.
monthOfExperience	String	Số tháng kinh nghiệm (Ví dụ: 3 hoặc "Không yêu cầu").
deadline	Long (Unix ms)	Hạn nộp hồ sơ của ứng viên.
occupationalCategory	String	Cấp bậc công việc (Ví dụ: "Nhân viên", "Trưởng phòng").
education	String	Yêu cầu học vấn (Ví dụ: "Đại học trở lên").
employmentType	String	Hình thức làm việc (Ví dụ: "FULL_TIME").
openings	Integer	Số lượng tuyển dụng (Ví dụ: 1).
description	String	Mô tả công việc dạng text, các mục phân cách bằng \textbackslashn.
requirements	String	Yêu cầu ứng viên dạng text, các mục phân cách bằng \textbackslashn.
benefits	String	Quyền lợi ứng viên dạng text, các mục phân cách bằng \textbackslashn.
income	Array[String]	Mảng chứa các thông tin về lương (Ví dụ: thông tin lương, phụ cấp/thu nhập thêm,...).
schedule	String	Thời gian làm việc (Ví dụ: "Thứ 2 - Thứ 6 (từ 8:00 đến 17:00)").
skillsNeeded	Array[String]	Danh sách các kỹ năng bắt buộc đối với ứng viên.
skillsShouldHave	Array[String]	Danh sách các kỹ năng được ưu tiên hoặc nên có.
specialty	Array[String]	Chuyên môn công việc (Ví dụ: ["SoftwareEngineer", "Backend"]).
extra_inf	Object	Các trường thông tin tùy chỉnh khác.
meta_tags	Object	Cung cấp thông tin mô tả về trang web cho trình duyệt.
json_ld	Object	Khối dữ liệu JSON-LD gốc của TopCV.
pageText	String	Toàn bộ văn bản thô của trang.

Bảng 3.3: Đặc tả cấu trúc khối Quality Flags

Tên trường	Kiểu dữ liệu	Mô tả khi giá trị mang cờ true
has_json_ld	Boolean	Trang tuyển dụng chứa thẻ schema chuẩn cấu trúc JSON-LD.
has_page_text	Boolean	Có thu được nội dung văn bản thô, tránh lỗi tải trang trống.
has_structured_company_name_conflict	Boolean	Tên công ty ở JSON-LD khác với tên ở HTML.
has_valid_posting_date	Boolean	Trường <code>event_ts</code> parse thành công, không bị null.
has_valid_deadline	Boolean	Trường <code>deadline</code> parse thành công, không bị null.
has_salary_info	Boolean	Bóc tách thành công thông tin lương trong tin tuyển dụng.
has_location_info	Boolean	Bóc tách thành công thông tin địa điểm làm việc.
has_experience_info	Boolean	Bóc tách thành công thông tin về kinh nghiệm làm việc.
has_requirements	Boolean	Mảng <code>requirements</code> có dữ liệu.
has_description	Boolean	Mảng <code>description</code> có dữ liệu.
has_benefits	Boolean	Mảng <code>benefits</code> có dữ liệu.
has_skills_info	Boolean	Có chứa thông tin về kỹ năng yêu cầu của ứng viên.
has_education_info	Boolean	Có thông tin yêu cầu học vấn.
has_specialty	Boolean	Trường <code>specialty</code> có dữ liệu.
has_schedule	Boolean	Trường <code>schedule</code> có dữ liệu.
has_employment_type	Boolean	Có thông tin hình thức làm việc.
has_income	Boolean	Trường <code>income</code> có dữ liệu.
has_extra_info	Boolean	Có thêm thông tin tùy chỉnh thêm.

4. **Kết hợp thông tin kỹ năng (Skills):** Các kỹ năng bắt buộc (`skillsNeeded`) và ưu tiên (`skillsShouldHave`) được gom lại thành một trường mảng duy nhất tên là `skills` bằng hàm `array_distinct(array_union(col1,col2))`.
5. **Tính toán các chỉ số văn bản (Count Metrics):** Nhằm phục vụ kiểm tra chất lượng dữ liệu, hệ thống tự động sinh ra các trường đo lường như `description_count` (số lượng dòng mô tả công việc phân cách bởi `\textbackslashn`), `requirements_count`, `benefits_count`, và các trường đếm kích thước mảng (`skills_count`, `specialty_count`).
6. **Bổ sung metadata kỹ thuật:** Thêm trường `crawl_domain` trích xuất host của URL ("`www.topcv.vn`") và phiên bản bản ghi `record_version`.

Chính sách loại bỏ trùng lặp tại Bronze

Trong quá trình ETL nạp dữ liệu hàng ngày, nếu hệ thống phát hiện các sự kiện trùng lặp, cơ chế deduplication của Bronze sẽ được áp dụng:

- **Quy tắc dedup:** Dedup theo cặp khoá (`job_id`, `hash_content`, `ingest_date`).
- **Nguyên tắc xử lý:** Nếu có nhiều bản ghi trùng cả `job_id` và `hash_content` (nghĩa là bài đăng tuyển dụng hoàn toàn giống nhau về nội dung), Spark sẽ chỉ giữ lại 1 bản ghi duy nhất có thời điểm thu thập `ingest_ts` mới nhất.
- **Quản lý phiên bản thay đổi:** Nếu trùng `job_id` nhưng có `hash_content` khác biệt (bài tuyển dụng có sự thay đổi nội dung), hệ thống không loại bỏ mà tăng chỉ số phiên bản `record_version` của bản ghi.

Bảng 3.4: Đặc tả các trường tạo mới trong bảng dữ liệu Bronze

Tên trường	Kiểu dữ liệu	Mô tả chi tiết
<code>record_version</code>	Integer	Tăng khi <code>hash_content</code> đổi với cùng <code>job_id</code> . Mặc định 1.
<code>is_deleted</code>	Boolean	Trường phát hiện một URL tuyển dụng không còn tồn tại. Mặc định <code>false</code> .
<code>crawl_domain</code>	String	Tên miền nguồn thu thập, được trích xuất tự động từ <code>source_url</code> . Mặc định <code>false</code> .
<code>description_count</code>	Integer	Đếm số lượng dòng trong mô tả công việc, phân cách bởi dấu xuống dòng <code>\n</code> .
<code>requirements_count</code>	Integer	Đếm số lượng dòng trong yêu cầu công việc, phân cách bởi dấu xuống dòng <code>\n</code> .
<code>benefits_count</code>	Integer	Đếm số lượng dòng trong lợi ích công việc, phân cách bởi dấu xuống dòng <code>\n</code> .
<code>income_count</code>	Integer	Đếm số lượng phần tử trong mảng <code>income</code> .
<code>skills_count</code>	Integer	Đếm số lượng phần tử trong mảng <code>skills</code> .
<code>specialty_count</code>	Integer	Đếm số lượng chuyên môn được gán thẻ trong tin tuyển dụng.

3.4 Lớp dữ liệu bạc (Silver Layer)

Lớp Bạc (Silver) là trung tâm của quá trình làm sạch dữ liệu, chuẩn hóa nghiệp vụ chung của doanh nghiệp và loại bỏ trùng lặp tuyệt đối. Dữ liệu Silver được lưu trữ dưới

định dạng Parquet phân vùng theo `ingest_date` trên HDFS tại thư mục `/silver/jobs/`. Thông tin về các trường được tạo mới từ lớp Silver (so với lớp Bronze) được mô tả trong Bảng 3.5.

Nguyên tắc thiết kế và chế độ ghi

Dữ liệu Silver được ghi với chế độ `overwrite` kết hợp cấu hình Spark:

```
spark.sql.sources.partitionOverwriteMode = dynamic
```

Cơ chế này cho phép hệ thống chỉ cập nhật và ghi đè các phân vùng (`ingest_date`) có sự thay đổi dữ liệu trong ngày mà không gây ảnh hưởng đến dữ liệu lịch sử của các phân vùng khác trên HDFS.

Các bước chuẩn hóa nghiệp vụ cốt lõi

1. **Phân tích cú pháp JSON-LD:** Sử dụng hàm `get_json_object(col("json_ld"), "$.path")` để trích xuất các thuộc tính trong cấu trúc JSON-LD. Điều này giúp hệ thống lấy được các thông tin chính thức gồm:

- `company_url`: Website chính thức của công ty.
- `company_logo`: Logo công ty tuyển dụng.
- `work_country`: Mã quốc gia nơi làm việc.
- `job_location_type`: Kiểu địa điểm làm việc (Ví dụ: "TELECOMMUTE").
- `job_id_platform` Mã định danh công việc (job) nội bộ của TopCV.

Các ký tự ký hiệu URL lồng nhau (\/) đều được hệ thống tự động unescape để trả về định dạng URL chuẩn.

2. **Chuẩn hóa lương về VNĐ/Tháng:** Nhằm tạo cơ sở cho việc phân tích và dự báo tài chính lao động, hệ thống chuẩn hóa tất cả các định dạng lương về VNĐ/tháng theo quy trình hai bước:

- *Bước 1 (Ưu tiên)*: Đọc thông tin có cấu trúc về mức lương min (`salary_min`), max (`salary_max`), loại tiền tệ (`salary_currency`), và đơn vị chu kỳ lương (`salary_unit`) trích xuất trực tiếp từ khối JSON-LD.
- *Bước 2 (Fallback)*: Nếu các trường lương trong JSON-LD bị trống (null), hệ thống áp dụng các biểu thức chính quy (Regex) để phân tích chuỗi văn bản thô `salary` (Ví dụ: "10 - 15 Triệu" chuyển thành min = 10,000,000 và max = 15,000,000 VNĐ).
- *Bước 3 (Quy đổi tỷ giá)*: Toàn bộ lương có đơn vị ngoại tệ được quy đổi sang VNĐ bằng hằng số tỷ giá quy định trong code là `USD_TO_VND = 25_000`.

Kết quả sinh ra hai trường `salary_min_vnd` và `salary_max_vnd`. Ngoài ra, cờ hiệu `salary_is_negotiable` sẽ được set là `true` nếu chuỗi lương thô chứa từ khóa "Thỏa thuận" hoặc cả hai cận lương min/max đều null.

3. **Chuẩn hóa địa điểm:** Trường địa điểm thô thường chứa chuỗi phức tạp bao gồm cả tỉnh/thành phố và địa chỉ chi nhánh. Hệ thống chuẩn hóa bằng cách:

- Đếm tổng số địa điểm chi nhánh ứng tuyển bằng trường `location_count`.
 - Sử dụng hàm chuỗi của Spark để cắt chuỗi tại ký tự hai chấm (:) đầu tiên. Phần phía trước ký tự là `city` (thành phố trung tâm), phần phía sau là `address` (địa chỉ cụ thể). Kết quả được lưu dưới dạng một mảng các struct: `location_detail: Array<Struct<city, address>`.
 - Xác định cờ `has_remote` có giá trị `true` nếu thuộc tính `job_location_type` trong JSON-LD mang giá trị bằng `"TELECOMMUTE"`.
4. **Chuẩn hóa yêu cầu kinh nghiệm:** Trường `experience_required` dạng Boolean sẽ trả về `false` nếu trường kinh nghiệm chứa từ khóa "Thỏa thuận" hoặc "Không yêu cầu", ngược lại được gán giá trị mặc định là `true`.

Chính sách loại bỏ trùng lặp tại Silver

Khác với lớp Bronze cho phép giữ lại các phiên bản thay đổi của cùng một công việc, lớp Silver áp dụng chính sách loại bỏ trùng lặp nghiệp vụ tuyệt đối dựa trên khoá duy nhất: `job_id`. Hệ thống chỉ giữ lại bản ghi có thứ tự xếp hạng bằng 1 (nghĩa là bản ghi có `record_version` cao nhất - phiên bản mới nhất được cập nhật của công việc đó). Điều này đảm bảo tính duy nhất của dữ liệu phục vụ cho các thuật toán phân tích downstream.

3.5 Lớp dữ liệu vàng (Gold Layer)

Lớp Vàng (Gold) đại diện cho tầng lưu trữ sẵn sàng phục vụ báo cáo doanh nghiệp và truy vấn ứng dụng. Dữ liệu Gold được tổ chức dưới mô hình phi chuẩn hóa để tạo thành một bảng duy nhất có tên `job_market_index`.

Dữ liệu được ghi đồng thời xuống HDFS Parquet tại thư mục `/gold/jobs/job_market_index/` và được đồng bộ đẩy trực tiếp vào hệ thống cơ sở dữ liệu Elasticsearch để tạo lập chỉ mục tìm kiếm phục vụ Serving Layer.

Các phương pháp xử lý dữ liệu khi chuyển từ Silver sang Gold

Tiến trình ETL chuyển đổi dữ liệu từ tầng Silver lên Gold (được triển khai trong tệp mã nguồn `silver_to_gold.py`) áp dụng 5 phương pháp xử lý và làm sạch sau đây:

1. **Phân tích cú pháp và phân loại lịch làm việc (parse_schedule):** Trường lịch trình làm việc `schedule` gốc từ crawler chứa dữ liệu văn bản tự do phức tạp. Hệ thống áp dụng một quy trình phân tích luật dựa trên tiêu đề (Header-first rule-based approach) qua 3 bước:
 - *Bước 1: Chuẩn hóa văn bản.* Loại bỏ các cặp dấu ngoặc mảng JSON thô (`[ "... ]`), unescape các dấu nhảy kép trùng lặp và các ký tự xuống dòng ẩn (`\n`). Lấy dòng đầu tiên của văn bản làm tiêu đề lịch trình (Header).
 - *Bước 2: Phân loại theo tiêu đề.* Sử dụng biểu thức chính quy (Regex) khớp mẫu dòng đầu tiên để phát hiện ngày làm việc tiêu chuẩn: "Thứ 2 - Chủ nhật" phân loại vào nhóm T2-CN, "Thứ 2 - Thứ 7" vào nhóm T2-T7, "Thứ 2 - Thứ 6" vào nhóm T2-T6.

Bảng 3.5: Đặc tả các trường tạo mới tại lớp Silver

Tên trường	Kiểu dữ liệu	Mô tả chi tiết
company_url	String	URL chính thức của doanh nghiệp trên TopCV.
company_logo	String	Logo doanh nghiệp.
work_country	String	Mã quốc gia nơi làm việc.
job_location_type	String	Hình thức làm việc.
salary_currency	String	Đơn vị tiền tệ của mức lương mà công việc cung cấp.
salary_min	Double	Mức lương tối thiểu được mô tả trong tin tuyển dụng.
salary_max	Double	Mức lương tối đa được mô tả trong tin tuyển dụng.
salary_unit	String	Đơn vị thanh toán lương (Ví dụ: "MONTH", "YEAR").
job_id_platform	String	Mã định danh tin tuyển dụng nội bộ của TopCV.
salary_min_vnd	Long	Cận dưới mức lương quy đổi về VND/tháng (mặc định <code>null</code> nếu thỏa thuận).
salary_max_vnd	Long	Cận trên mức lương quy đổi về VND/tháng (mặc định <code>null</code> nếu thỏa thuận).
location_detail	Array[Struct]	Mảng cấu trúc tách rõ tỉnh/thành phố (<code>city</code>) và địa chỉ (<code>address</code>).
location_count	Integer	Số lượng chi nhánh làm việc.
has_remote	Boolean	Cờ đánh dấu công việc có cho phép làm việc từ xa hay không.
salary_is_negotiable	Boolean	Đánh dấu bài tuyển dụng thuộc dạng lương thỏa thuận.
experience_required	Boolean	Đánh dấu tin tuyển dụng có yêu cầu kinh nghiệm bắt buộc.

- *Bước 3: Nhận diện tín hiệu làm việc Thứ Bảy (Saturday Work Signals).* Hệ thống quét toàn bộ phần thân văn bản còn lại (Body) để tìm các dấu hiệu làm thêm Thứ Bảy (Ví dụ: “sáng thứ 7”, “làm t7”, “thứ 7 cách tuần”, “saturday”). Nếu phát hiện có tín hiệu làm Thứ Bảy mặc dù tiêu đề là từ Thứ 2 đến Thứ 6, hệ thống tự động hiệu chỉnh kết quả phân loại từ T2-T6 lên T2-T7.
- *Xử lý Fallback:* Các trường hợp lịch xoay ca (chứa từ khóa “xoay ca”, “theo ca”) được gán vào nhóm Other. Các công việc làm tại nhà, tự do (chứa từ khóa “linh hoạt”, “flexible”) được gán vào nhóm Flexible.

Từ kết quả của `schedule_type`, hệ thống suy diễn thuộc tính logic `is_weekend_free` mang giá trị `true` nếu thuộc nhóm T2-T6 hoặc Flexible, ngược lại nhận giá trị `false`.

2. **Làm sạch mảng kỹ năng (clean_skills):** Trong quá trình bóc tách HTML từ TopCV, do giao diện chứa các nút tương tác ẩn/hiện, mảng kỹ năng `skills` cào được bị lẫn các nhãn định hướng giao diện của trang web như “thu gọn” hoặc “xem thêm”. Tầng Gold áp dụng bộ lọc mảng của Spark SQL (`F.expr("filter(...)")`) để loại bỏ triệt để các chuỗi này (không phân biệt chữ hoa thường sau khi đã trim khoảng trắng), trả lại mảng kỹ năng nghiệp vụ sạch.

3. **Chuẩn hóa và đồng bộ hóa trường thông tin (normalize_gold_fields):** Phương pháp này thực hiện đồng bộ cấu trúc nghiệp vụ để phục vụ lập chỉ mục:

- *Đổi tên liên kết:* Đổi tên trường `job_id_platform` ở lớp Silver thành `company_id` ở lớp Gold, thiết lập định danh duy nhất cho doanh nghiệp.
- *Chuẩn hóa kiểu kinh nghiệm:* Ép kiểu trường kinh nghiệm `monthOfExperience` từ chuỗi sang số nguyên (`IntegerType`) bằng hàm `try_cast`. Các chuỗi chữ như “Không yêu cầu” được chuyển đổi thành giá trị `null`, biểu thị vị trí tuyển dụng không có giới hạn cứng về số tháng kinh nghiệm tối thiểu.
- *Tính toán trạng thái hoạt động (is_active):* Tự động gán bằng `false` nếu thời gian hạn nộp hồ sơ (`deadline`) nhỏ hơn thời gian hiện tại của hệ thống, ngược lại gán bằng `true` (mặc định bằng `true` nếu không đăng ký hạn `deadline`).
- *Xử lý giá trị trống:* Chuyển đổi các giá trị `null` của cờ `has_remote` thành `false` mặc định để đảm bảo Elasticsearch lập chỉ mục đầy đủ, tránh hiện tượng ẩn trường thông tin trên giao diện Kibana.

4. **Làm giàu dữ liệu lương bằng mô hình học máy (enrich_salary_predictions):**

Thị trường tuyển dụng Việt Nam có một tỉ lệ lớn tin đăng ở dạng “Thỏa thuận” mà không công bố mức lương cụ thể, khiến người dùng không thể so sánh và lọc công việc theo thu nhập. Để giải quyết vấn đề này, hệ thống tích hợp bước dự báo lương bằng Spark ML trực tiếp vào luồng ETL Silver → Gold.

Điều kiện kích hoạt dự báo: Mô hình chỉ được phép dự báo và ghi đè lương khi cả hai cận `salary_min_vnd` và `salary_max_vnd` đều bằng `null`. Bất kỳ giá trị lương thật nào đã parse được (dù chỉ một cận) đều được ưu tiên bảo toàn, không bị dự báo ghi đè.

Thuật toán: Mô hình `GBRegressor` (Gradient Boosted Tree Regressor) của Spark ML được chọn vì khả năng xử lý phi tuyến tính và tương tác phức tạp giữa các

đặc trưng — phù hợp với bài toán dự báo lương nơi mức lương không tăng tuyến tính theo số năm kinh nghiệm mà phụ thuộc nhiều vào ngành, kỹ năng chuyên sâu và quy mô công ty. Nhân huấn luyện là $\log(\text{mid_salary_vnd})$, tức logarithm của lương trung bình thật, giúp giảm thiểu ảnh hưởng của các ngoại lệ (outliers) trong phân phối lương.

Quy tắc điền lương hiển thị (Serving Rules): Sau khi mô hình tạo ra một lương trung bình dự đoán $\hat{m}$, hệ thống tính toán khoảng lương hiển thị theo quy tắc cố định: cận dưới bằng $0.9 \times \hat{m}$ và cận trên bằng $1.1 \times \hat{m}$. Các trường `salary_display_*` được điền theo thứ tự ưu tiên:

- Nếu tin có đủ cả hai cận lương thật: giữ nguyên, không dự báo.
- Nếu tin chỉ có lương tối thiểu thật: bảo toàn lương tối thiểu thật, điền lương tối đa bằng dự đoán.
- Nếu tin chỉ có lương tối đa thật: bảo toàn lương tối đa thật, điền lương tối thiểu bằng dự đoán.
- Nếu hoàn toàn không có lương thật: điền toàn bộ từ kết quả dự báo.
- Nếu mô hình lỗi hoặc chưa được huấn luyện: các trường `salary_display_*` được gán `null` để đảm bảo tính ổn định schema cho Elasticsearch.

Giá trị `salary_source` ghi lại nguồn gốc lương, cho phép Kibana Dashboard phân biệt và thống kê riêng biệt tỉ lệ lương thật so với lương dự báo trên toàn bộ thị trường.

5. **Lọc lựa chọn cột dữ liệu tối ưu (select_gold_columns):** Để tối ưu hóa không gian lưu trữ trên HDFS và giảm tải dung lượng đánh chỉ mục truyền tải qua mạng sang Elasticsearch, hệ thống loại bỏ hoàn toàn các cột metadata kỹ thuật trung gian ở các bước trước (như `json_ld`, `pageText`, `hash_content`, `quality_flags`, `record_version`, `is_deleted`, `crawl_domain` và các trường đếm dòng `*_count` của Bronze). Chỉ lựa chọn các trường nghiệp vụ thực tế phục vụ trực tiếp cho Dashboard và API.

Các trường dữ liệu mới xuất hiện tại tầng Gold (so với Silver)

Bảng 3.6 liệt kê chi tiết các trường thông tin mới được sinh ra trong tiến trình ETL từ Silver lên Gold, phục vụ trực tiếp cho tìm kiếm và phân tích:

Bảng 3.6: Đặc tả các trường dữ liệu mới xuất hiện tại tầng Gold

Tên trường	Kiểu dữ liệu	Mô tả chi tiết
company_id	String	ID định danh công ty, đổi tên từ trường job_id_platform của Silver.
is_active	Boolean	Trạng thái hoạt động của tin tuyển dụng; nhận false nếu đã quá hạn deadline.
schedule_type	String	Phân loại lịch làm việc: T2-T6, T2-T7, T2-CN, Flexible hoặc Other.
is_weekend_free	Boolean	Đánh dấu công việc được nghỉ Thứ Bảy và Chủ Nhật.
salary_prediction_applied	Boolean	Nhận diện tin tuyển dụng đã áp dụng mức lương dự báo từ mô hình học máy.
salary_display_min_vnd	Long	Lương tối thiểu dùng hiển thị trên UI.
salary_display_max_vnd	Long	Lương tối đa dùng hiển thị trên UI.
salary_display_avg_vnd	Long	Lương trung bình dùng cho sắp xếp hoặc vẽ biểu đồ xu hướng.
salary_source	String	Phân loại nguồn lương: parsed_range , predicted , unknown , v.v.
salary_predicted_min_vnd	Long	Cận dưới mức lương dự báo bằng Spark ML.
salary_predicted_max_vnd	Long	Cận trên mức lương dự báo bằng Spark ML.
salary_predicted_avg_vnd	Long	Lương trung bình dự đoán trực tiếp từ mô hình học máy.
salary_prediction_model_version	String	Phiên bản mô hình Spark ML được sử dụng.

Chương 4

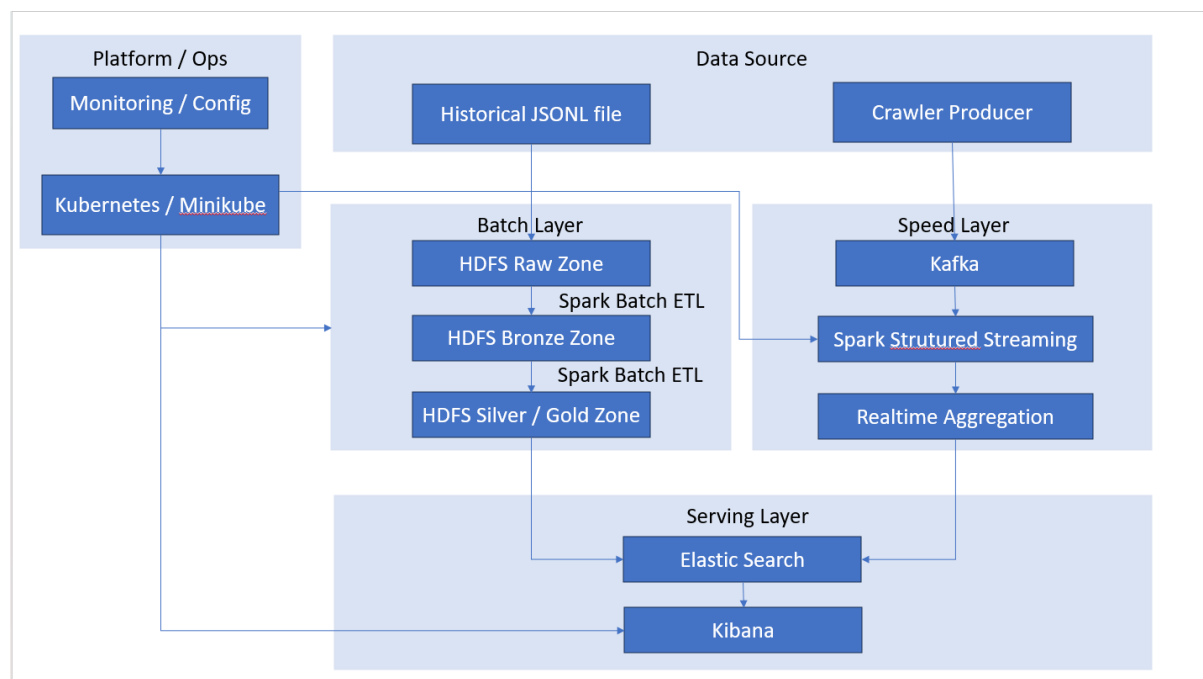
ĐƯỜNG ống XỬ LÝ DỮ LIỆU (DATA PIPELINE)

4.1 Quy trình xử lý dữ liệu tổng quát

Để đáp ứng đồng thời hai yêu cầu cốt lõi của hệ thống phân tích thị trường lao động: (1) tính chính xác, đầy đủ của dữ liệu lịch sử phục vụ các báo cáo chuyên sâu và (2) tính thời sự, nhanh chóng của các tin tuyển dụng mới phát hành, hệ thống được thiết kế theo mô hình kiến trúc **Lambda Architecture**.

Quy trình xử lý dữ liệu tổng quát phân chia hệ thống thành ba tầng chức năng chính (Batch Layer, Speed Layer, Serving Layer) chạy trên nền tảng hạ tầng điều phối thống nhất.

Hình 4.1 mô tả chi tiết kiến trúc Lambda tổng quát và luồng luân chuyển dữ liệu từ nguồn thu thập cho đến khi hiển thị trên các bảng điều khiển trực quan.



Hình 4.1: Kiến trúc Lambda tổng quát của hệ thống Big Data Job Market

Các tầng chức năng trong hệ thống

Quy trình hoạt động tuần tự và song song của các tầng chức năng được mô tả chi tiết dưới đây:

- 1. Data Source (Nguồn dữ liệu):** Dữ liệu đầu vào của hệ thống được chia làm hai luồng độc lập:
 - *Luồng lịch sử (Historical JSONL files):* Các tệp tin lưu trữ thô ở định dạng JSONL chứa lịch sử dữ liệu tuyển dụng đã được cào từ trước. Luồng này được đẩy trực tiếp vào hệ thống HDFS để phục vụ cho các tiến trình xử lý lô.
 - *Luồng thời gian thực (Realtime Crawlers):* Bộ thu thập dữ liệu (Crawler Producer) hoạt động liên tục, bóc tách thông tin công việc mới trên nền tảng TopCV và đẩy các thông điệp thô vào hàng đợi thông điệp Apache Kafka.
- 2. Batch Layer (Tầng xử lý theo lô):** Chịu trách nhiệm lưu trữ bền vững và tính toán chính xác tuyệt đối trên toàn bộ tập dữ liệu lịch sử theo mô hình kiến trúc Medallion 4 lớp:
 - *HDFS Raw Zone:* Nơi tiếp nhận và lưu trữ nguyên bản các tệp JSONL từ luồng lịch sử, đảm bảo dữ liệu không bị sửa đổi.
 - *HDFS Bronze Zone:* Spark Batch Job thực hiện làm phẳng cấu trúc JSON, ép kiểu sơ bộ, gán nhãn thời gian thu thập và lưu trữ dưới định dạng Parquet (phân vùng theo ngày).
 - *HDFS Silver/Gold Zone:* Dữ liệu đi qua các bước chuẩn hóa sâu (địa điểm, kinh nghiệm, kỹ năng), xử lý trùng lặp và làm giàu thông tin lương bằng mô hình Machine Learning (GBTRegressor). Kết quả được tổ chức thành các bảng phân tích tối ưu (Gold Layer) lưu trữ trên HDFS.
- 3. Speed Layer (Tầng xử lý thời gian thực):** Được thiết kế để tối thiểu hóa độ trễ, xử lý tức thời dòng dữ liệu biến động liên tục:
 - *Hàng đợi Apache Kafka:* Nhận và đệm thông điệp thô từ các Crawler.
 - *Spark Structured Streaming:* Tiêu thụ dữ liệu từ Kafka topic `jobs_raw` theo chu kỳ micro-batch (mỗi 10 giây). Tiến trình thực hiện phân tích cấu trúc, lọc trùng lặp cục bộ dựa trên cơ chế Watermarking (60 phút) và gán nhãn/dự báo lương thời gian thực bằng cách sử dụng trực tiếp mô hình ML đã được huấn luyện sẵn từ Batch Layer (`SALARY_MODEL_PATH`).
 - *Realtime Aggregation (Tích lũy thời gian thực):* Thực hiện các phép tính toán thống kê trượt (slide window) để cho ra các chỉ số tức thời như: Số lượng công việc mới xuất hiện mỗi 10 phút, Tần suất xuất hiện của các kỹ năng hot hàng giờ, và Phân bố dải lương tuyển dụng hàng giờ.
- 4. Serving Layer (Tầng dịch vụ):** Đóng vai trò điểm truy vấn nhất của hệ thống phân tích. Tầng này kết hợp kết quả từ cả hai luồng xử lý:
 - *Elasticsearch:* Nhận dữ liệu cập nhật theo chu kỳ ngày từ Gold Layer của Batch và dữ liệu cập nhật liên tục (độ trễ dưới 10 giây) từ luồng Realtime của Speed Layer. Các bản ghi trùng lặp được ghi đè dựa trên định danh duy nhất (`job_id`) để đảm bảo tính nhất quán của dữ liệu.

- *Kibana*: Kết nối trực tiếp với Elasticsearch để cung cấp giao diện dashboard trực quan hóa dữ liệu theo thời gian thực và lịch sử, giúp người phân tích theo dõi biến động thị trường việc làm một cách toàn diện.

5. **Platform / Ops (Nền tảng vận hành và Quản trị)**: Toàn bộ hệ thống được container hóa và điều phối trên cụm Kubernetes (Minikube).

- *Kubernetes*: Đảm bảo khả năng tự phục hồi (self-healing), tự động mở rộng và quản lý vòng đời của tất cả các container dịch vụ (HDFS Namenode/Datanode, Kafka Brokers, Spark Master/Workers, Elasticsearch cluster, Kibana, và Apache Airflow).
- *Monitoring / Config*: Cung cấp khả năng giám sát tập trung các chỉ số tài nguyên và sức khỏe hệ thống, kết hợp cấu hình đồng bộ cho toàn bộ chu kỳ sống của ứng dụng.

So sánh vai trò của Batch Layer và Speed Layer

Để thấy rõ sự tương trợ giữa hai luồng xử lý trong kiến trúc Lambda, bảng 4.1 tóm tắt sự khác biệt về mặt chức năng và công nghệ áp dụng:

Bảng 4.1: So sánh đặc tính kỹ thuật giữa Batch Layer và Speed Layer

Đặc tính	Batch Layer	Speed Layer
Độ trễ (Latency)	Cao (Theo chu kỳ ngày - Daily)	Thấp (Thời gian thực - 10 giây)
Độ chính xác	Tuyệt đối (Xử lý toàn bộ dữ liệu lịch sử)	Tương đối (Phục vụ phân tích xu hướng tức thời)
Công nghệ cốt lõi	Apache Spark Batch, HDFS, Airflow	Apache Kafka, Spark Structured Streaming
Đầu vào dữ liệu	Tệp JSONL lịch sử tích lũy trên HDFS	Message stream từ Crawler gửi tới Kafka
Xử lý ML	Huấn luyện lại mô hình (Offline Training)	Dự báo lương tức thời (Real-time Inference)
Loại bỏ trùng lặp	Toàn cục (Global deduplication)	Cục bộ (Watermark & sliding window)

4.2 Thu thập dữ liệu (Ingestion & Kafka Producer)

Phần thu thập dữ liệu được thực hiện nhằm đưa dữ liệu tuyển dụng từ TopCV vào hai luồng xử lý của hệ thống: batch layer và speed layer. Thay vì chỉ crawl dữ liệu rồi lưu thủ công, hệ thống xây dựng một quy trình có thể chạy lặp lại, gồm các bước: xác định danh sách trang cần crawl, lấy chi tiết từng tin tuyển dụng, ghi dữ liệu thô ra JSONL, sau đó nạp dữ liệu vào HDFS hoặc Kafka tùy theo mục đích xử lý.

Quy trình thực hiện tổng quát được chia thành hai nhánh:

TopCV → Crawler → JSONL → HDFS Raw → Batch Processing

TopCV → Speed Crawler → JSONL → Kafka jobs_raw → Streaming Processing

4.2.1 Xác định nguồn và phạm vi crawl

Bước đầu tiên là xác định nguồn dữ liệu cần thu thập trên TopCV. Hệ thống xuất phát từ các trang danh sách việc làm, sau đó duyệt qua từng trang kết quả để lấy URL chi tiết của các tin tuyển dụng. Với mỗi URL chi tiết, crawler gửi request tới trang tương ứng và phân tích HTML để trích xuất nội dung tuyển dụng.

Để tránh phụ thuộc quá nhiều vào giao diện hiển thị của website, quá trình crawl được tổ chức theo hai mức dữ liệu. Mức thứ nhất là dữ liệu danh sách, dùng để lấy các đường dẫn chi tiết và một số thông tin ngắn nếu có. Mức thứ hai là dữ liệu chi tiết, dùng để lấy đầy đủ các trường quan trọng như tiêu đề công việc, tên công ty, địa điểm, mức lương, kinh nghiệm, mô tả công việc, yêu cầu ứng viên, quyền lợi và kỹ năng liên quan.

Khi thực hiện crawl, hệ thống cần giới hạn số trang hoặc số tin tuyển dụng theo từng lần chạy. Việc giới hạn này giúp kiểm soát thời gian chạy, tránh tạo tải quá lớn lên nguồn dữ liệu và giúp pipeline dễ kiểm thử. Trong batch layer, số lượng trang crawl thường lớn hơn để thu thập dữ liệu lịch sử. Trong speed layer, số lượng trang crawl nhỏ hơn và tập trung vào các tin mới hoặc vừa cập nhật.

4.2.2 Trích xuất và tạo bản ghi JSONL

Sau khi lấy được nội dung HTML của trang chi tiết, crawler tiến hành parse dữ liệu và tạo một bản ghi tuyển dụng. Mỗi bản ghi được biểu diễn dưới dạng JSON, trong đó mỗi trường mô tả một thuộc tính của tin tuyển dụng. Các trường được giữ ở dạng gần với dữ liệu gốc để đảm bảo Raw layer phản ánh đúng nguồn thu thập ban đầu.

Một bản ghi đầu ra của bước crawl thường gồm các nhóm thông tin sau:

- Thông tin định danh: URL gốc của tin tuyển dụng, thời điểm crawl và mã định danh nếu có.
- Thông tin công việc: tiêu đề, vị trí, mô tả công việc, yêu cầu ứng viên và quyền lợi.
- Thông tin công ty: tên công ty và các thông tin liên quan có thể trích xuất được.
- Thông tin phân tích: địa điểm, mức lương, kinh nghiệm, kỹ năng và hình thức làm việc.

Dữ liệu sau khi trích xuất được ghi ra file JSONL. Trong định dạng này, mỗi dòng là một object JSON độc lập tương ứng với một tin tuyển dụng. Cách ghi theo từng dòng giúp crawler có thể ghi dữ liệu dần trong quá trình chạy, không cần chờ thu thập xong toàn bộ dữ liệu mới lưu file. Nếu quá trình crawl bị gián đoạn, các bản ghi đã ghi trước đó vẫn có thể được sử dụng hoặc kiểm tra lại.

Ở bước ingestion, hệ thống chưa thực hiện chuẩn hóa nghiệp vụ sâu. Ví dụ, chuỗi lương như “10–20 triệu” hoặc “Thỏa thuận” vẫn được giữ gần với dạng gốc. Các bước như quy đổi lương sang VND, chuẩn hóa địa điểm, tách kỹ năng và loại bỏ bản ghi lỗi được thực hiện ở các tầng xử lý sau. Cách làm này giúp dữ liệu thô có khả năng tái xử lý khi logic ETL thay đổi.

4.2.3 Thực hiện nạp dữ liệu vào HDFS cho batch layer

Đối với batch layer, sau khi crawler tạo xong file JSONL, hệ thống đưa file này vào HDFS Raw layer. Đây là nơi lưu trữ đầu vào cho các Spark batch job. Quy trình thực hiện gồm các bước chính:

1. Chạy crawler ở chế độ batch để thu thập dữ liệu từ TopCV và tạo file JSONL cục bộ trong container hoặc môi trường chạy job.
2. Kiểm tra file đầu ra, đảm bảo file tồn tại, không rỗng và mỗi dòng có thể parse được thành JSON hợp lệ.
3. Tạo thư mục Raw trên HDFS theo ngày ingest hoặc theo mã lần chạy để dễ truy vết dữ liệu.
4. Copy file JSONL từ môi trường chạy crawler lên HDFS Raw layer.
5. Kiểm tra lại trên HDFS để xác nhận file đã được ghi thành công trước khi chuyển sang bước xử lý Raw to Bronze.

Luồng thực hiện batch ingestion có thể mô tả như sau:

Run Batch Crawler → Validate JSONL → Create HDFS Raw Path → Put JSONL to HDFS → Verify HDFS File

Trong triển khai trên Kubernetes, bước này được đóng gói thành một Kubernetes Job để Airflow có thể gọi và giám sát. Khi job chạy thành công, dữ liệu trong HDFS Raw layer trở thành đầu vào cho Spark job `raw_to_bronze.py`. Nếu job thất bại, Airflow ghi nhận trạng thái lỗi và người vận hành có thể kiểm tra log của pod crawler để xác định nguyên nhân, ví dụ lỗi kết nối tới TopCV, lỗi ghi file, hoặc lỗi kết nối tới HDFS.

Thiết kế này giúp batch pipeline có tính lặp lại. Mỗi lần chạy pipeline đều tạo ra một tập dữ liệu Raw riêng, nhờ đó có thể so sánh kết quả giữa các lần crawl, chạy lại ETL từ một mốc dữ liệu cụ thể hoặc phục hồi pipeline mà không cần thu thập lại dữ liệu từ đầu.

4.2.4 Thực hiện publish dữ liệu vào Kafka cho speed layer

Đối với speed layer, mục tiêu không phải lưu dữ liệu lịch sử trước mà là đưa dữ liệu mới vào luồng xử lý realtime càng sớm càng tốt. Vì vậy, sau khi speed crawler tạo file JSONL, Kafka producer đọc file này và gửi từng bản ghi vào topic `jobs_raw`.

Quy trình thực hiện gồm các bước:

1. Chạy crawler ở chế độ speed để lấy một lượng nhỏ tin tuyển dụng mới hoặc vừa cập nhật.
2. Ghi kết quả crawl ra file JSONL trung gian.
3. Producer mở file JSONL và đọc lần lượt từng dòng.
4. Với mỗi dòng, producer kiểm tra JSON cơ bản và gửi nội dung bản ghi vào Kafka topic `jobs_raw`.
5. Sau khi gửi xong, producer ghi log số lượng bản ghi đã publish thành công và số bản ghi bị lỗi nếu có.

Luồng thực hiện speed ingestion được mô tả như sau:

Run Speed Crawler → Write JSONL → Read Line by Line → Publish to jobs_raw → Spark Streaming Consume

Kafka đóng vai trò buffer giữa crawler và Spark Structured Streaming. Nhờ Kafka, crawler không cần phụ thuộc trực tiếp vào tốc độ xử lý của Spark. Nếu Spark streaming job đang restart hoặc xử lý chậm, dữ liệu vẫn được giữ trong topic và có thể được đọc lại dựa trên offset. Điều này giúp speed layer ổn định hơn so với việc crawler ghi trực tiếp vào Elasticsearch hoặc gọi trực tiếp logic xử lý streaming.

Trước khi producer chạy, hệ thống cần đảm bảo Kafka cluster đã sẵn sàng và các topic cần thiết đã tồn tại, đặc biệt là `jobs_raw`, `jobs_clean` và `jobs_dead_letter`. Sau khi producer publish dữ liệu vào `jobs_raw`, Spark Structured Streaming sẽ đọc dữ liệu từ topic này, validate schema, chuẩn hóa bản ghi và ghi kết quả sang các đích tiếp theo.

4.2.5 Kiểm tra lỗi và đảm bảo chất lượng đầu vào

Trong quá trình thực hiện ingestion, hệ thống cần kiểm tra lỗi ở từng điểm chuyển tiếp dữ liệu. Với crawler, cần kiểm tra số lượng URL thu được, số request thành công, số trang chi tiết parse được và số bản ghi được ghi ra JSONL. Với batch ingestion, cần kiểm tra file trên HDFS có tồn tại và có kích thước hợp lệ. Với speed ingestion, cần kiểm tra số lượng message được publish vào Kafka và log lỗi của producer.

Một số lỗi thường gặp trong bước thu thập dữ liệu gồm:

- Trang nguồn thay đổi cấu trúc HTML khiến crawler không trích xuất được trường cần thiết.
- Request tới TopCV bị timeout hoặc bị giới hạn tần suất.
- File JSONL sinh ra rỗng hoặc có dòng JSON không hợp lệ.
- Không kết nối được tới HDFS khi chạy batch ingestion.
- Không kết nối được tới Kafka hoặc topic `jobs_raw` chưa được tạo khi chạy speed ingestion.

Để hạn chế ảnh hưởng của các lỗi này, ingestion chỉ thực hiện kiểm tra cơ bản và ghi log rõ ràng, còn việc phân loại bản ghi hợp lệ hay lỗi nghiệp vụ được giao cho các tầng xử lý sau. Trong speed layer, các bản ghi không đạt yêu cầu sau khi Spark Structured Streaming kiểm tra sẽ được chuyển sang topic `jobs_dead_letter`. Trong batch layer, các bản ghi lỗi hoặc thiếu trường quan trọng có thể được loại bỏ hoặc đánh dấu trong quá trình chuyển từ Bronze sang Silver.

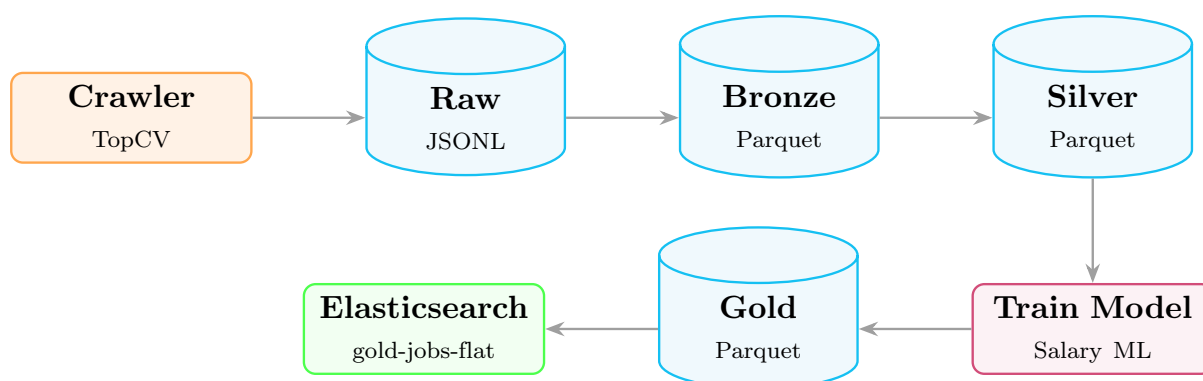
Tóm lại, cách thực hiện ingestion của hệ thống gồm bốn bước chính: crawl dữ liệu từ TopCV, ghi dữ liệu thô ra JSONL, nạp JSONL vào HDFS cho batch layer hoặc publish vào Kafka cho speed layer, sau đó kiểm tra log và trạng thái đầu ra. Cách tổ chức này giúp cùng một nguồn dữ liệu có thể phục vụ đồng thời hai nhu cầu: xử lý lịch sử theo lô và xử lý dữ liệu mới gần thời gian thực.

4.3 Batch Processing Layer (Apache Spark & HDFS)

Tầng xử lý theo lô (Batch Processing Layer) là thành phần trung tâm của kiến trúc Lambda trong hệ thống phân tích thị trường lao động TopCV. Tầng này chịu trách nhiệm xử lý toàn bộ lịch sử dữ liệu thu thập được, đảm bảo tính chính xác và đầy đủ cho các kết quả phân tích nghiệp vụ theo chu kỳ ngày. Hệ thống sử dụng Apache Spark 4.1.1 làm engine tính toán phân tán và HDFS (Hadoop Distributed File System) làm hệ thống lưu trữ dữ liệu bền vững, toàn bộ triển khai trên nền tảng Minikube.

4.3.1 Tổng quan luồng xử lý Batch Pipeline

Toàn bộ luồng xử lý dữ liệu của Batch Layer được tổ chức thành một chuỗi tuyến tính, được điều phối hoàn toàn tự động bởi Apache Airflow:



Hình 4.2: Luồng dữ liệu tuần tự của Batch Processing Pipeline

Bảng 4.2 tóm tắt các Spark Job trong pipeline cùng với chế độ ghi và nguồn/đích dữ liệu HDFS tương ứng. Chi tiết về các phép biến đổi dữ liệu nghiệp vụ được thực hiện bên trong mỗi job đã được trình bày ở **Mục 3**.

Bảng 4.2: Tổng quan các Spark Job trong Batch Pipeline

Bước	Tên Job	Nguồn	Đích	Chế độ ghi
1	raw_to_bronze.py	/raw/	/bronze/	Append
2	bronze_to_silver.py	/bronze/	/silver/	Overwrite
3	train_salary_model.py	/silver/	/models/	Overwrite
4	silver_to_gold.py	/silver/	/gold/	Overwrite
5	gold_to_elasticsearch.py	/gold/	ES Index	Overwrite

Chế độ chạy Incremental và Full load

Các Spark Job trong pipeline (trừ `train_salary_model.py` và `gold_to_elasticsearch.py`) đều hỗ trợ hai chế độ chạy linh hoạt thông qua tham số dòng lệnh `-date`:

- **Incremental** (`-date YYYY-MM-DD`): Chỉ đọc và ghi một phân vùng `ingest_date` cụ thể. Chế độ này được Airflow sử dụng trong các lần chạy lại tự động (retry) hoặc khi cần xử lý lại dữ liệu của một ngày cụ thể mà không ảnh hưởng đến toàn bộ kho dữ liệu.
- **Full load** (không tham số `-date`): Đọc và xử lý toàn bộ các phân vùng đang có trong HDFS. Chế độ này phù hợp khi cần tái xây dựng hoàn toàn một tầng dữ liệu sau khi nâng cấp logic nghiệp vụ hoặc sau khi mô hình ML được cải tiến.

Riêng `train_salary_model.py` luôn đọc toàn bộ Silver (không phân biệt ngày) để đảm bảo mô hình được huấn luyện trên tập dữ liệu lớn nhất có thể. `gold_to_elasticsearch.py` luôn nạp lại toàn bộ Gold Parquet mỗi lần chạy.

4.3.2 Huấn luyện mô hình dự báo lương (`train_salary_model.py`)

Job `train_salary_model.py` được thực thi mỗi ngày sau khi bước Bronze → Silver hoàn thành, đảm bảo mô hình liên tục được cập nhật khi có thêm dữ liệu lương công khai mới. Khác với các ETL job khác chỉ xử lý dữ liệu theo ngày, job này luôn đọc toàn bộ tầng Silver để mô hình học được từ tất cả lịch sử tích lũy.

Quy trình huấn luyện.

1. Đọc toàn bộ Silver Parquet từ HDFS.
2. Lọc chỉ giữ những bản ghi có ít nhất một cận lương công khai (`salary_min_vnd` hoặc `salary_max_vnd` khác `null`) — đây là tập huấn luyện. Các bản ghi lương thỏa thuận là chính xác tập mà mô hình cần dự báo, vì vậy chúng bị loại khỏi tập huấn luyện để tránh rò rỉ thông tin (data leakage).
3. Chia tập 80/20 với `seed=42` để đảm bảo tính tái lập. Nếu tập test rỗng (dữ liệu quá ít), hệ thống tự động dùng toàn bộ tập huấn luyện làm tập đánh giá để tránh lỗi dừng pipeline.
4. Fit Pipeline Spark ML (`GBRegressor`). Chi tiết đặc trưng và kiến trúc mô hình được trình bày ở Mục 3.5 và Mục 5.
5. Đánh giá mô hình trên tập test bằng hai chỉ số: RMSE và MAE trên không gian log-lương.

Lưu mô hình và ghi lại chỉ số. PipelineModel được ghi vào HDFS tại `/models/salary_prediction/latest` với chế độ `overwrite`, thay thế hoàn toàn model của ngày hôm trước. Song song đó, một tệp JSON chứa các chỉ số đánh giá (RMSE, MAE, số dòng huấn luyện, tên model family, timestamp) được lưu tại `/models/salary_prediction/metrics/latest` — đây là artifact kiểm toán (audit trail) giúp truy vết hiệu suất mô hình theo thời gian mà không cần lưu toàn bộ lịch sử model.

4.3.3 Nạp dữ liệu vào Elasticsearch (`gold_to_elasticsearch.py`)

Bước cuối cùng của pipeline đọc toàn bộ Gold Parquet từ HDFS để ghi vào chỉ mục Elasticsearch `gold-jobs-flat`. Trường `job_id` được cấu hình làm Document ID (`es.mapping.id`) để Elasticsearch thực hiện `upsert` — bản ghi đã tồn tại sẽ được cập nhật thay vì tạo thêm bản sao.

4.3.4 Điều phối tự động bằng Apache Airflow

Toàn bộ pipeline được điều phối bởi Apache Airflow DAG có tên `job_market_batch_pipeline`, chạy theo lịch cố định lúc 2:00 AM hàng ngày theo múi giờ Asia/Ho Chi Minh. DAG được thiết kế theo mô hình Airflow-as-Orchestrator: toàn bộ định nghĩa runtime Spark (image Docker, tham số `spark-submit`, tài nguyên CPU/RAM) nằm trong các Kubernetes CronJob ở namespace `spark` — Airflow chỉ đảm nhiệm vai trò kích hoạt đúng thứ tự và giám sát trạng thái hoàn thành.

Bảng 4.3: Cấu hình kết nối Elasticsearch trong Batch Pipeline

Tham số cấu hình	Giá trị và ý nghĩa
<code>es.nodes</code>	<code>elasticsearch.search.svc</code> — DNS nội bộ Kubernetes trỏ đến service Elasticsearch.
<code>es.port</code>	9200 — Cổng REST API mặc định của Elasticsearch.
<code>es.resource</code>	<code>gold-jobs-flat</code> — Tên chỉ mục đích đến trên Elasticsearch.
<code>es.mapping.id</code>	<code>job_id</code> — Document ID để upsert, đảm bảo không tạo bản ghi trùng.
<code>es.nodes.wan.only</code>	<code>false</code> — Cho phép Spark Executor giao tiếp trực tiếp với ES data node qua IP nội bộ cluster, thay vì chỉ qua coordinator node.
Write mode	<code>overwrite</code> — Ghi đè toàn bộ chỉ mục mỗi lần chạy batch.

Cơ chế kích hoạt Spark Job qua Kubernetes. Mỗi task Airflow thực thi lệnh bash gồm 3 giai đoạn:

1. **Tạo Kubernetes Job:** Dùng lệnh `kubectl create job -from=cronjob/<name>` để tạo một Kubernetes Job chạy một lần (one-off Job) từ CronJob đã được định nghĩa sẵn, với tên job mang theo ngày và số thứ tự retry của Airflow để tránh trùng tên.
2. **Chờ pod khởi động:** Airflow polling kiểm tra cho đến khi tìm thấy submit pod của job đó (tối đa 5 phút).
3. **Chờ hoàn thành:** Dùng `kubectl wait -for=condition=complete job/<name> -timeout=<N>s`. Nếu job thất bại hoặc quá timeout, Airflow tự động in toàn bộ log của submit pod và Spark Driver Pod trước khi đánh dấu task Failed, hỗ trợ debug nhanh mà không cần vào Kubernetes thủ công.

Bảng 4.4: Cấu hình timeout của từng task trong Airflow DAG

Task Airflow	CronJob Kubernetes	Timeout
<code>crawl_jobs</code>	<code>batch-etl-crawl-jobs</code>	108,000 giây (30 giờ) — Crawl toàn bộ TopCV có thể kéo dài nhiều giờ.
<code>raw_to_bronze</code>	<code>batch-etl-raw-to-bronze</code>	2,400 giây (40 phút).
<code>bronze_to_silver</code>	<code>batch-etl-bronze-to-silver</code>	2,400 giây (40 phút).
<code>train_salary_model</code>	<code>batch-etl-train-salary-model</code>	2,400 giây (40 phút).
<code>silver_to_gold</code>	<code>batch-etl-silver-to-gold</code>	2,400 giây (40 phút).
<code>gold_to_elasticsearch</code>	<code>batch-etl-gold-to-elasticsearch</code>	2,400 giây (40 phút).

Kiểm tra sức khỏe hạ tầng trước khi chạy. DAG bắt đầu bằng hai task song song kiểm tra tình trạng sẵn sàng của hạ tầng trước khi cho phép crawler khởi động:

- `check_hdfs`: Dùng `nc -z hdfs-namenode.hdfs.svc.cluster.local 9000` xác nhận HDFS NameNode đang lắng nghe trên cổng RPC 9000.
- `check_elasticsearch`: Gọi HTTP tới http://elasticsearch.search.svc.cluster.local:9200/_cluster/health xác nhận Elasticsearch đang hoạt động.

Nếu một trong hai bước kiểm tra thất bại, toàn bộ pipeline dừng ngay lập tức, tránh tình trạng Spark Job khởi chạy rồi thất bại ở giữa do hạ tầng chưa sẵn sàng.

Cấu hình Spark Session

Tất cả các Spark Job trong Batch Pipeline sử dụng cấu hình Spark Session thống nhất:

Bảng 4.5: Cấu hình Spark Session dùng chung cho tất cả Batch Job

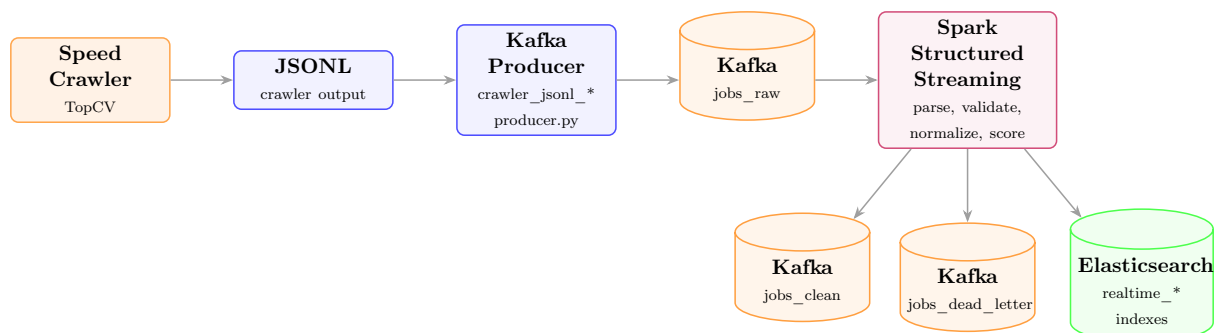
Tham số cấu hình	Giá trị và ý nghĩa
<code>spark.sql.parquet.compression.codec</code>	<code>snappy</code> — Nén Snappy cân bằng giữa tốc độ đọc/ghi và dung lượng lưu trữ.
<code>spark.sql.shuffle.partitions</code>	200 — Số partition shuffle, phù hợp với môi trường Minikube tài nguyên giới hạn.
<code>spark.sql.sources.partitionOverWriteMode</code>	<code>dynamic</code> — Chỉ ghi đè partition đang xử lý, bảo vệ dữ liệu lịch sử các ngày khác.

4.4 Speed Processing Layer (Spark Structured Streaming)

Tầng xử lý luồng (Speed Processing Layer) được xây dựng nhằm bổ sung khả năng cập nhật dữ liệu gần thời gian thực cho hệ thống Lambda Architecture. Khác với Batch Layer xử lý toàn bộ dữ liệu lịch sử theo chu kỳ, Speed Layer tập trung vào các bản ghi mới hoặc vừa được cập nhật từ TopCV, đưa chúng qua Kafka và Spark Structured Streaming để tạo dữ liệu sạch, dữ liệu lỗi và các chỉ số tổng hợp phục vụ dashboard realtime trên Kibana. Trong triển khai hiện tại, endpoint chính của tầng này là `apps/stream_etl/stream_main.py`.

4.4.1 Tổng quan luồng xử lý realtime

Luồng xử lý realtime bắt đầu từ speed crawler. Crawler ghi dữ liệu đầu ra dưới dạng JSONL, sau đó `crawler_jsonl_producer.py` đọc từng dòng, chuẩn hóa nhẹ cấu trúc event và publish vào Kafka topic `jobs_raw`. Spark Structured Streaming đọc topic này theo cơ chế micro-batch, parse JSON theo schema đã định nghĩa, kiểm tra chất lượng bản ghi, chuẩn hóa các trường nghiệp vụ và ghi kết quả sang nhiều đích khác nhau.



Hình 4.3: Luồng dữ liệu của Speed Processing Layer

Thiết kế này tách rời tốc độ thu thập dữ liệu và tốc độ xử lý dữ liệu. Kafka đóng vai trò bộ đệm trung gian, giúp crawler không phụ thuộc trực tiếp vào trạng thái của Spark Streaming hoặc Elasticsearch. Khi Spark job khởi động lại, hệ thống có thể tiếp tục đọc dữ liệu từ Kafka dựa trên offset và checkpoint thay vì mất dữ liệu đã publish.

4.4.2 Kafka topics và vai trò của từng luồng dữ liệu

Speed Layer sử dụng ba topic Kafka chính. Topic `jobs_raw` là đầu vào của Spark Structured Streaming, chứa các event thô được producer gửi lên từ file JSONL. Sau khi xử lý, các bản ghi hợp lệ được ghi sang topic `jobs_clean`, còn các bản ghi không đạt yêu cầu được ghi sang topic `jobs_dead_letter`. Cách tách topic này giúp hệ thống vừa có dữ liệu sạch để tiêu thụ tiếp, vừa bảo toàn được thông tin lỗi để phục vụ debug và kiểm toán chất lượng dữ liệu.

Bảng 4.6: Các Kafka topic chính trong Speed Layer

Topic	Vai trò	Nội dung chính
<code>jobs_raw</code>	Input stream	Event thô từ crawler/producer, gồm metadata, payload và <code>quality_flags</code> .
<code>jobs_clean</code>	Clean output	Bản ghi đã parse, validate, chuẩn hóa, deduplicate và có schema gần với Gold Layer.
<code>jobs_dead_letter</code>	Error output	Bản ghi lỗi kèm <code>error_reason</code> , Kafka offset, partition và timestamp để truy vết.

4.4.3 Schema đầu vào và kiểm tra chất lượng dữ liệu

Schema đầu vào của Speed Layer được định nghĩa trong `apps/stream_etl/schemas/raw_job_schema.py`. Mỗi event gồm hai nhóm thông tin chính: metadata của quá trình crawl và phần `payload` chứa nội dung tin tuyển dụng. Metadata bao gồm các trường như `source`, `source_url`, `ingest_ts`, `event_ts`, `job_id` và `hash_content`. Phần `payload` chứa các trường nghiệp vụ như tiêu đề, công ty, lương, địa điểm, kinh nghiệm, hạn nộp, mô tả, yêu cầu, phúc lợi, kỹ năng và hình thức làm việc.

Khi Spark đọc dữ liệu từ `jobs_raw`, hàm `parse_raw_kafka` chuyển key/value dạng bytes của Kafka thành chuỗi, parse JSON bằng `from_json` và giữ lại các thông tin Kafka như topic, partition, offset và timestamp. Sau đó, `validate_raw_jobs` gọi bộ quy tắc

chất lượng dữ liệu dùng chung để sinh cột `error_reason`. Nếu `error_reason` bằng `null`, bản ghi được xem là hợp lệ; ngược lại, bản ghi được chuyển sang dead-letter stream.

4.4.4 Chuẩn hóa realtime và tạo schema sạch

Các bản ghi hợp lệ được xử lý bởi hàm `build_clean_jobs`. Mục tiêu của bước này là tạo một schema realtime có tên trường gần với Gold Layer để dữ liệu batch và realtime có thể được truy vấn, trực quan hóa và so sánh theo cùng một logic nghiệp vụ. Các phép biến đổi quan trọng gồm:

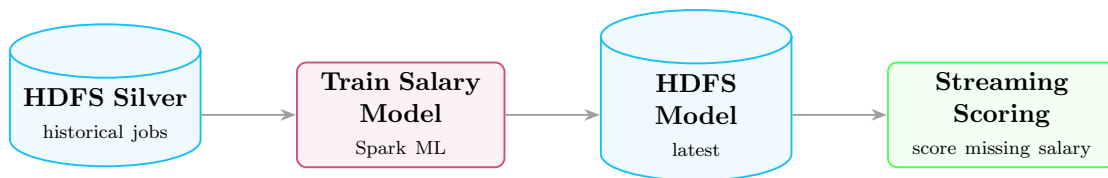
- Chuẩn hóa thời gian: chuyển `event_ts`, `ingest_ts`, `stream_ingest_ts` từ epoch milliseconds sang timestamp; tạo `event_time`, `ingest_time`, `stream_ingest_time` và `ingest_date`.
- Chuẩn hóa lương: parse chuỗi lương thô để tạo `salary_min_million`, `salary_max_million`, `salary_avg_million`, `salary_min_vnd`, `salary_max_vnd`, `salary_currency`, `salary_unit` và `salary_bin`.
- Chuẩn hóa địa điểm và hình thức làm việc: tạo mảng `location`, `location_detail`, `primary_city`, `location_count` và cờ `has_remote`.
- Chuẩn hóa kỹ năng và ngành nghề: hợp nhất `skillsNeeded`, `skillsShouldHave` thành `skills`; giữ `specialty`; tính `skills_count` và `specialty_count`.
- Chuẩn hóa lịch làm việc: tái sử dụng logic `parse_schedule` để tạo `schedule_type` và `is_weekend_free`.
- Tạo các đặc trưng đếm như `benefits_count`, `requirements_count`, `description_count`, hỗ trợ phân tích chất lượng nội dung tin tuyển dụng.

Sau khi chuẩn hóa, Spark áp dụng watermark trên `event_time` với ngưỡng 60 phút và loại bỏ trùng lặp theo cặp `job_id`, `hash_content`. Cơ chế này giúp giảm trùng lặp trong luồng realtime, đồng thời vẫn cho phép hệ thống xử lý các event đến muộn trong một khoảng thời gian hợp lý.

4.4.5 Tích hợp mô hình dự báo lương trong luồng realtime

Một điểm quan trọng của Speed Layer là khả năng tái sử dụng mô hình dự báo lương đã được huấn luyện ở Batch Layer. Khi biến môi trường `ENABLE_SALARY_PREDICTION` được bật, `stream_main.py` nạp `PipelineModel` từ đường dẫn `SALARY_MODEL_PATH`, mặc định là `hdfs://hdfs-namenode.hdfs.svc:9000/models/salary_prediction/latest`. Mô hình được load một lần khi streaming job khởi động, sau đó được dùng để transform từng micro-batch.

Dữ liệu realtime được score chủ yếu trong trường hợp bản ghi không có `salary_min_vnd` và `salary_max_vnd`, tức các tin tuyển dụng có lương thỏa thuận hoặc không công khai mức lương. Các đặc trưng đầu vào như kinh nghiệm, địa điểm, công ty và trạng thái remote được chuẩn hóa theo cùng một logic với batch layer. Nhờ đó, kết quả dự báo ở realtime giữ được tính nhất quán với dữ liệu Gold.



Hình 4.4: Tái sử dụng mô hình dự báo lương từ Batch Layer cho Speed Layer

4.4.6 Ghi dữ liệu ra Kafka và Elasticsearch

Sau bước chuẩn hóa, Spark Structured Streaming khởi chạy đồng thời nhiều streaming query. Hai query bắt buộc ghi dữ liệu sang Kafka: `jobs_clean` cho bản ghi hợp lệ và `jobs_dead_letter` cho bản ghi lỗi. Ngoài ra, nếu `WRITE_ELASTICSEARCH=true`, hệ thống ghi dữ liệu chi tiết realtime sang Elasticsearch thông qua `foreachBatch`. Cách ghi theo `foreachBatch` phù hợp với Elasticsearch vì mỗi micro-batch có thể được xử lý như một DataFrame tĩnh, để cấu hình bulk write, mapping và document id.

Các Elasticsearch index realtime được hệ thống sử dụng gồm:

- `realtime_jobs_v1`: lưu bản ghi job realtime đã chuẩn hóa.
- `realtime_job_counts_10m_v1`: số lượng job theo cửa sổ 10 phút.
- `realtime_skill_counts_hourly_v1`: số lần xuất hiện của kỹ năng theo giờ.
- `realtime_top_skills_hourly_v1`: nhóm kỹ năng nổi bật theo giờ.
- `realtime_salary_bins_hourly_v1`: phân bố dải lương theo giờ.

4.4.7 Tổng hợp trạng thái theo thời gian

Bên cạnh việc ghi dữ liệu chi tiết, Speed Layer còn tạo các bảng tổng hợp realtime để phục vụ dashboard. Các aggregation này được xây dựng từ clean stream trước bước ghi Elasticsearch và sử dụng output mode `update`. Hiện tại hệ thống triển khai ba nhóm tổng hợp chính:

Bảng 4.7: Các realtime aggregation trong Speed Layer

Module	Index đích	Ý nghĩa nghiệp vụ
<code>jobs_per_10m.py</code>	<code>realtime_job_counts_10m_v1</code>	Theo dõi số lượng tin tuyển dụng mới trong từng cửa sổ 10 phút, phục vụ biểu đồ tốc độ xuất hiện job realtime.
<code>top_skills_hourly.py</code>	<code>realtime_skill_counts_hourly_v1</code> <code>realtime_top_skills_hourly_v1</code>	Đếm tần suất kỹ năng và xác định nhóm kỹ năng nổi bật theo từng giờ, hỗ trợ nhận diện xu hướng kỹ năng nóng.
<code>salary_bins_realtime.py</code>	<code>realtime_salary_bins_hourly_v1</code>	Theo dõi phân bố tin tuyển dụng theo các dải lương trong từng cửa sổ 1 giờ.

Các chỉ số này được tính sẵn ở Spark thay vì để Kibana tự tổng hợp trên toàn bộ dữ liệu thô. Cách làm này giảm tải cho Elasticsearch, giúp dashboard realtime phản hồi nhanh hơn và có cấu trúc dữ liệu ổn định hơn.

4.4.8 Checkpoint, trigger và khả năng phục hồi

Streaming job sử dụng checkpoint riêng cho từng output query, nằm dưới thư mục CHECKPOINT_DIR, mặc định là /checkpoints/speed. Trong môi trường Kubernetes, checkpoint được gắn với PVC speed-checkpoint-pvc. Mỗi query có checkpoint location riêng, ví dụ jobs_clean, jobs_dead_letter, jobs_realtime_es, jobs_per_10m, top_skills_hourly và salary_bins_hourly. Nhờ checkpoint, Spark có thể khôi phục offset Kafka và trạng thái aggregation sau khi driver restart.

Chu kỳ xử lý micro-batch được điều khiển bởi biến TRIGGER_SECONDS, mặc định là 10 giây. Giá trị này cân bằng giữa độ trễ realtime và chi phí xử lý trong môi trường Minikube tài nguyên giới hạn. Khi cần chạy lại demo từ đầu hoặc Kafka topic được tạo lại, checkpoint cần được xóa để tránh Spark giữ offset cũ không còn hợp lệ.

Bảng 4.8: Một số cấu hình runtime quan trọng của Speed Layer

Biến cấu hình	Ý nghĩa
KAFKA_BOOTSTRAP_SERVERS	Kafka bootstrap server dùng để Spark Structured Streaming đọc/ghi dữ liệu Kafka trong môi trường local hoặc Kubernetes.
RAW_TOPIC CLEAN_TOPIC DEAD_LETTER_TOPIC	Tên topic đầu vào, topic đầu ra sạch và topic dead-letter cho các bản ghi lỗi.
STARTING_OFFSETS	Offset bắt đầu khi streaming query chưa có checkpoint; mặc định là <code>earliest</code> .
TRIGGER_SECONDS	Chu kỳ chạy micro-batch của Spark Structured Streaming.
CHECKPOINT_DIR	Thư mục checkpoint gốc cho các streaming query, giúp khôi phục Kafka offset và trạng thái aggregation sau restart.
WRITE_ELASTICSEARCH	Bật hoặc tắt việc ghi dữ liệu realtime và các bảng tổng hợp sang Elasticsearch.
ENABLE_JOBS_PER_10M ENABLE_TOP_SKILLS_HOURLY ENABLE_SALARY_BINS_HOURLY	Bật hoặc tắt từng nhóm aggregation realtime: số lượng job theo 10 phút, kỹ năng theo giờ và phân bố lương theo giờ.
ENABLE_SALARY_PREDICTION SALARY_MODEL_PATH	Bật hoặc tắt salary scoring realtime và cấu hình vị trí mô hình dự báo lương trên HDFS.

4.5 Điều phối luồng công việc với Airflow

Airflow được sử dụng để điều phối các pipeline xử lý dữ liệu trong hệ thống. Trong project, Airflow không trực tiếp thực hiện xử lý dữ liệu mà đóng vai trò quản lý lịch chạy, kiểm tra trạng thái các service, tạo Kubernetes Job và theo dõi kết quả thực thi của từng bước trong pipeline. Các tác vụ xử lý dữ liệu chính vẫn được thực hiện bởi crawler, Spark batch job, Spark Structured Streaming job, Kafka và Elasticsearch.

Airflow được triển khai trong namespace `airflow`, gồm các thành phần chính: Airflow Webserver, Airflow Scheduler, PostgreSQL metadata database, ServiceAccount, RBAC và volume lưu log. Project sử dụng `LocalExecutor`; các task trong DAG chủ yếu dùng `BashOperator` để gọi lệnh `kubect1`. Nhờ đó, Airflow có thể tạo job trong namespace `spark`, kiểm tra Kafka topic trong namespace `kafka`, kiểm tra Elasticsearch và giám sát trạng thái pod/job trong Kubernetes.

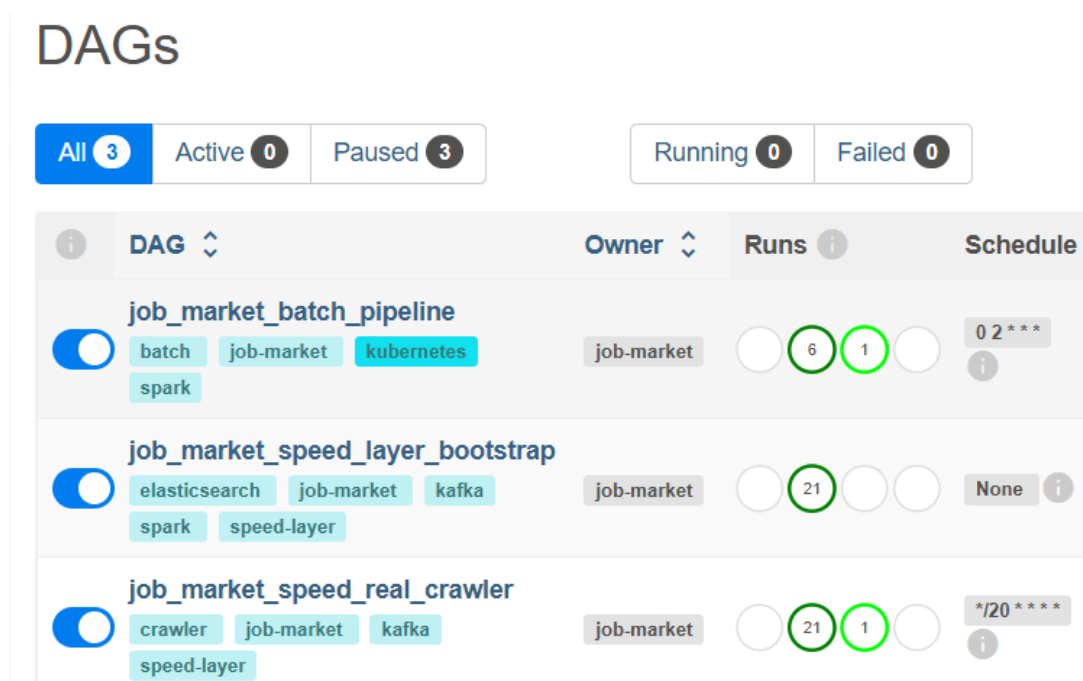
Các DAG của hệ thống được đặt trong thư mục:

`infra/airflow/dags/`

Các DAG chính gồm:

- `job_market_batch_pipeline`: điều phối batch pipeline từ bước crawl dữ liệu đến ghi dữ liệu Gold sang Elasticsearch.
- `job_market_speed_layer_bootstrap`: khởi động speed layer, submit Spark Structured Streaming job và kiểm tra các realtime index.
- `job_market_speed_real_crawler`: chạy định kỳ real TopCV speed crawler producer để đưa dữ liệu mới vào Kafka.

Hình 4.5 minh họa giao diện Airflow sau khi triển khai, trong đó hệ thống nhận diện ba DAG chính phục vụ điều phối batch pipeline, khởi động speed layer và chạy real crawler producer.

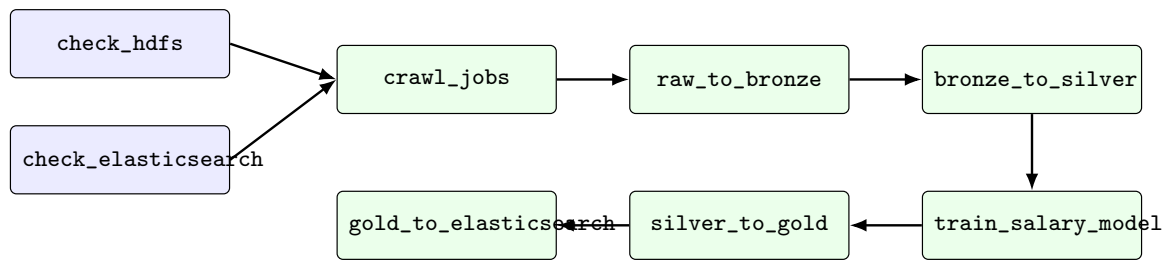


Hình 4.5: Danh sách các DAG Airflow được triển khai trong hệ thống, gồm DAG điều phối batch pipeline, DAG khởi động speed layer và DAG chạy real crawler producer định kỳ.

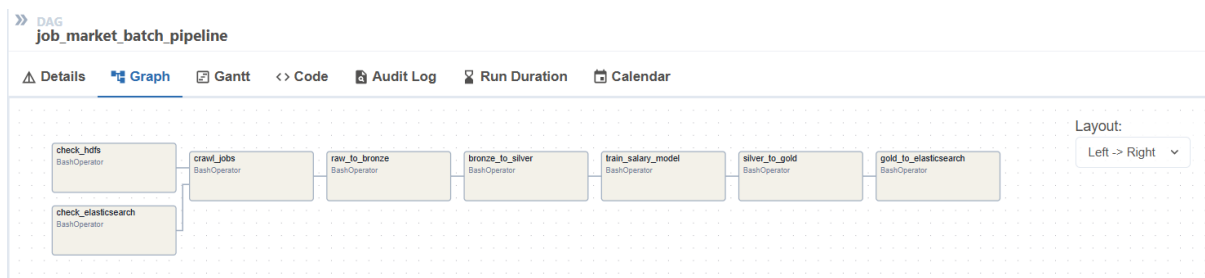
4.5.1 Điều phối Batch Pipeline

Batch pipeline được định nghĩa trong DAG `job_market_batch_pipeline`. DAG này có lịch chạy hằng ngày vào lúc 02:00 và chịu trách nhiệm điều phối toàn bộ luồng xử lý dữ liệu batch.

Hai task `check_hdfs` và `check_elasticsearch` được thực hiện song song. Task `crawl_jobs` chỉ bắt đầu khi cả hai bước kiểm tra hạ tầng đều hoàn thành thành công. Các task tiếp theo được thực hiện tuần tự theo luồng xử lý Raw–Bronze–Silver–Gold.



Hình 4.6 minh họa Graph View của DAG batch trong Airflow. Các task được tổ chức theo quan hệ phụ thuộc rõ ràng, bắt đầu từ kiểm tra trạng thái HDFS và Elasticsearch, sau đó lần lượt chạy crawler, các Spark batch job trong kiến trúc Medallion, huấn luyện mô hình dự đoán lương và cuối cùng ghi dữ liệu Gold sang Elasticsearch.



Hình 4.6: Graph View của DAG `job_market_batch_pipeline`, thể hiện thứ tự điều phối các bước trong batch pipeline từ kiểm tra hạ tầng, crawl dữ liệu, xử lý Raw–Bronze–Silver–Gold đến ghi dữ liệu sang Elasticsearch.

Các bước chính trong batch DAG gồm:

- **check_hdfs**: kiểm tra kết nối tới HDFS NameNode tại địa chỉ nội bộ của Kubernetes.
- **check_elasticsearch**: kiểm tra trạng thái Elasticsearch thông qua API `_cluster/health`.
- **crawl_jobs**: tạo Kubernetes Job từ CronJob `batch-etl-crawl-jobs`. Job này chạy crawler để thu thập dữ liệu TopCV và ghi dữ liệu thô vào HDFS Raw layer.
- **raw_to_bronze**: tạo Kubernetes Job từ CronJob `batch-etl-raw-to-bronze`. Job này chạy Spark để chuyển dữ liệu Raw sang Bronze.
- **bronze_to_silver**: tạo Kubernetes Job từ CronJob `batch-etl-bronze-to-silver`. Job này làm sạch, chuẩn hóa dữ liệu và tạo Silver layer.
- **train_salary_model**: tạo Kubernetes Job từ CronJob `batch-etl-train-salary-model`. Job này huấn luyện mô hình dự đoán lương dựa trên dữ liệu Silver.
- **silver_to_gold**: tạo Kubernetes Job từ CronJob `batch-etl-silver-to-gold`. Job này tạo Gold layer phục vụ phân tích và trực quan hóa.
- **gold_to_elasticsearch**: tạo Kubernetes Job từ CronJob `batch-etl-gold-to-elasticsearch`. Job này ghi dữ liệu Gold sang Elasticsearch index `gold-jobs-flat`.

Các CronJob trong namespace `spark` được đặt ở trạng thái `suspend: true`. Vì vậy, chúng không tự chạy theo lịch Kubernetes mà được Airflow sử dụng như các template. Khi cần chạy một bước xử lý, Airflow tạo một Kubernetes Job mới từ CronJob tương ứng bằng lệnh `kubectrl create job -from=cronjob/....`. Cách tổ chức này giúp định nghĩa runtime của Spark job được quản lý trong manifest Kubernetes, còn Airflow chỉ chịu trách nhiệm điều phối thứ tự chạy và giám sát kết quả.

DAG được cấu hình `retries=1`, do đó một task thất bại có thể được Airflow thử lại thêm một lần. Tên Kubernetes Job chứa ngày logic của DAG run và số lần thử, giúp tránh xung đột tên giữa các attempt.

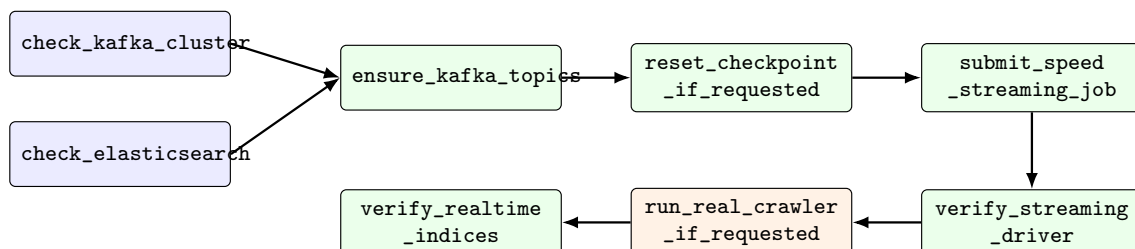
DAG có lịch `0 2 * * *` theo múi giờ `Asia/Ho_Chi_Minh` và sử dụng `catchup=False`. Vì vậy, Airflow không tự động tạo hàng loạt run bù cho các khoảng thời gian trong quá khứ khi DAG được kích hoạt lại.

Nếu một Spark job thất bại hoặc quá thời gian chờ, Airflow sẽ lấy thông tin pod, log của submit pod, log của driver pod và các Kubernetes event gần nhất để hỗ trợ debug. Nhờ đó, quá trình vận hành pipeline dễ theo dõi hơn so với việc chạy thủ công từng job.

4.5.2 Điều phối Speed Layer Bootstrap

DAG `job_market_speed_layer_bootstrap` được sử dụng để khởi động speed layer. DAG này không chạy theo lịch định kỳ mà được trigger thủ công khi cần khởi động hoặc kiểm thử luồng streaming.

Hai task `check_kafka_cluster` và `check_elasticsearch` được thực hiện song song. Sau khi cả Kafka và Elasticsearch đều sẵn sàng, DAG kiểm tra các Kafka topic, xử lý yêu cầu đặt lại checkpoint, khởi động hoặc sử dụng lại streaming driver và thực hiện các bước kiểm tra tiếp theo.



Màu cam ở task `run_real_crawler_if_requested` thể hiện đây là bước phụ thuộc tham số, không phải lúc nào cũng chạy crawler thật.

Các nhiệm vụ chính của DAG này gồm:

- Kiểm tra Kafka cluster và các Kafka resource trong namespace `kafka`.
- Kiểm tra trạng thái Elasticsearch.
- Tạo hoặc cập nhật các Kafka topic cần thiết thông qua manifest `jobs-topics.yaml`.
- Tùy chọn xóa checkpoint của Spark Structured Streaming khi cần chạy demo lại từ đầu.
- Submit Spark Structured Streaming job từ manifest `speed-stream-es-job.yaml`.
- Kiểm tra Spark streaming driver đã ở trạng thái `Running`.

- Tùy chọn chạy real TopCV speed crawler producer.
- Kiểm tra các realtime Elasticsearch index đã được tạo.

DAG này có hai tham số quan trọng:

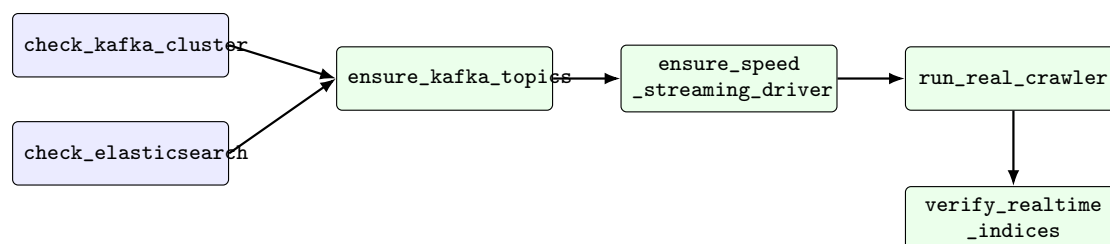
- **reset_checkpoint**: nếu được bật, Airflow sẽ chạy job dọn checkpoint để Spark Structured Streaming có thể xử lý lại dữ liệu từ đầu.
- **run_real_crawler**: nếu được bật, Airflow sẽ chạy real crawler producer để thu thập dữ liệu TopCV và publish vào Kafka.

Spark Structured Streaming là job chạy dài hạn, vì vậy Airflow chỉ submit job và kiểm tra driver pod đã chạy thành công. Airflow không chờ streaming job kết thúc, do bản chất streaming job được thiết kế để chạy liên tục.

4.5.3 Điều phối Real Crawler cho Speed Layer

DAG `job_market_speed_real_crawler` được dùng để chạy real TopCV speed crawler producer theo lịch định kỳ. DAG này có lịch chạy mỗi 20 phút, nhằm bổ sung dữ liệu mới hoặc dữ liệu vừa cập nhật từ TopCV vào Kafka.

Hai bước kiểm tra Kafka và Elasticsearch được thực hiện song song. Sau khi hạ tầng sẵn sàng, DAG bảo đảm các Kafka topic tồn tại, kiểm tra streaming driver, chạy crawler producer và xác minh dữ liệu đã xuất hiện trong các realtime index.



Trước khi chạy crawler, DAG kiểm tra Kafka, Elasticsearch và đảm bảo các Kafka topic đã tồn tại. Sau đó, DAG kiểm tra Spark Structured Streaming driver. Nếu streaming driver đang chạy, DAG sử dụng lại driver hiện có. Nếu không có driver đang chạy, Airflow submit lại job streaming từ manifest `speed-stream-es-job.yaml`.

Sau khi speed layer đã sẵn sàng, Airflow chạy Kubernetes Job `speed-real-crawler-producer`. Job này gồm hai tiến trình chính: crawler TopCV chạy ở chế độ speed để ghi dữ liệu ra file JSONL, và producer đọc file JSONL đó để publish từng bản ghi vào Kafka topic `jobs_raw`. Sau khi producer hoàn thành, Airflow kiểm tra các realtime Elasticsearch index như:

- `realtime_jobs_v1`
- `realtime_job_counts_10m_v1`
- `realtime_salary_bins_hourly_v1`
- `realtime_skill_counts_hourly_v1`
- `realtime_top_skills_hourly_v1`

Nhờ DAG này, speed layer có thể được duy trì tự động: Spark streaming job tiếp tục chạy để xử lý dữ liệu từ Kafka, còn crawler producer được kích hoạt định kỳ để đưa dữ liệu mới vào hệ thống.

4.5.4 Cơ chế giám sát và xử lý lỗi

Airflow đóng vai trò quan trọng trong việc giám sát pipeline. Với mỗi task, Airflow ghi lại trạng thái thực thi, thời gian chạy, log và lỗi nếu có. Khi một Kubernetes Job bị lỗi hoặc quá thời gian chờ, task tương ứng trong Airflow sẽ chuyển sang trạng thái **failed** do các lệnh trong **BashOperator** trả về mã lỗi khác 0. Log của task sẽ ghi lại các thông tin hỗ trợ debug như trạng thái Job/Pod và log container liên quan. Đối với các Spark batch job, DAG còn thu thập thêm thông tin chi tiết của pod và các Kubernetes event gần nhất để hỗ trợ xác định nguyên nhân lỗi.

Cơ chế giám sát của Airflow giúp xác định lỗi tại từng giai đoạn cụ thể trong pipeline. Khi một task như **raw_to_bronze** thất bại, log của Spark driver có thể được sử dụng để kiểm tra nguyên nhân liên quan đến quá trình đọc dữ liệu từ Raw layer hoặc ghi dữ liệu sang Bronze layer. Tương tự, nếu task **verify_realtime_indices** không thành công, nguyên nhân có thể đến từ việc Spark Structured Streaming chưa ghi dữ liệu sang Elasticsearch hoặc crawler producer chưa publish dữ liệu hợp lệ vào Kafka.

Việc sử dụng Airflow giúp hệ thống chuyển từ cách vận hành thủ công từng bước sang mô hình điều phối workflow tự động. Các bước xử lý được quản lý theo quan hệ phụ thuộc rõ ràng, có trạng thái thực thi cụ thể và có log hỗ trợ kiểm tra lỗi. Điều này giúp tăng tính ổn định và khả năng quan sát của pipeline trong môi trường Big Data gồm nhiều thành phần độc lập như HDFS, Kafka, Spark, Elasticsearch và Kibana.

Chương 5

MÔ HÌNH HỌC MÁY: DỰ ĐOÁN LƯƠNG

5.1 Bài toán và cấu trúc mô hình

Trong hệ thống phân tích thị trường việc làm, thông tin lương là một thuộc tính quan trọng để phục vụ lọc, sắp xếp, thống kê và trực quan hóa. Tuy nhiên, dữ liệu tuyển dụng thu thập từ thực tế không phải lúc nào cũng có mức lương rõ ràng. Một số tin công khai lương theo khoảng, ví dụ từ 10 đến 20 triệu đồng; một số tin chỉ có một cận như “trên 40 triệu” hoặc “tối 40 triệu”; nhiều tin khác chỉ ghi lương thỏa thuận. Nếu chỉ giữ chuỗi lương gốc, hệ thống khó thực hiện các phép phân tích định lượng như tính lương trung bình, phân bố lương theo kỹ năng hoặc lọc job theo mức lương.

Vì vậy, dự án xây dựng một mô hình học máy để dự đoán mức lương cho các tin tuyển dụng không có lương công khai rõ ràng. Đây là bài toán học máy có giám sát thuộc nhóm hồi quy. Dữ liệu huấn luyện được lấy từ các tin tuyển dụng đã parse được lương thật, còn dữ liệu cần dự đoán là các tin có lương bị ẩn hoặc chỉ ghi thỏa thuận.

Bài toán được mô tả như sau:

Đầu vào: X = thông tin mô tả tin tuyển dụng,

Nhãn: y = mức lương trung bình đã chuẩn hóa,

Mô hình: $f(X)$ = mức lương dự đoán cho tin chưa có lương rõ ràng.

Trong đó, đầu vào X gồm các nhóm thông tin như tiêu đề công việc, danh sách kỹ năng, số tháng kinh nghiệm yêu cầu, địa điểm làm việc, tên công ty và trạng thái làm việc từ xa. Đầu ra y là mức lương trung bình theo đơn vị VND/tháng. Nhãn huấn luyện được tạo từ hai trường đã chuẩn hóa là `salary_min_vnd` và `salary_max_vnd`. Quy tắc tạo nhãn được trình bày trong Bảng 5.1.

Mô hình không học trực tiếp trên giá trị lương gốc mà học trên biến đổi logarit:

$$\text{label_log_salary} = \log(1 + y)$$

Cách biến đổi này giúp giảm ảnh hưởng của các giá trị lương quá lớn và làm bài toán hồi quy ổn định hơn. Khi dự đoán, kết quả log được chuyển ngược về đơn vị VND bằng công thức:

$$\text{salary_predicted_avg_vnd} = \exp(\text{prediction_log_salary}) - 1$$

Bảng 5.1: Quy tắc tạo nhãn huấn luyện cho mô hình dự đoán lương

Trạng thái dữ liệu lương	Nhãn huấn luyện
Có cả <code>salary_min_vnd</code> và <code>salary_max_vnd</code>	$y = (\text{salary_min_vnd} + \text{salary_max_vnd})/2$
Chỉ có <code>salary_min_vnd</code>	$y = \text{salary_min_vnd}$
Chỉ có <code>salary_max_vnd</code>	$y = \text{salary_max_vnd}$
Không có cả hai trường	Không dùng để huấn luyện

Mô hình hiện tại sử dụng `GBRegressor` của Spark ML, tức Gradient-Boosted Tree Regressor. Thuật toán này phù hợp vì quan hệ giữa lương và các yếu tố tuyển dụng thường phi tuyến. Chẳng hạn, cùng kỹ năng Java nhưng mức lương của fresher và senior khác nhau đáng kể; cùng một mức kinh nghiệm nhưng công ty, địa điểm, hình thức remote và tập kỹ năng đi kèm cũng có thể tạo ra chênh lệch lớn. Mô hình được lưu với phiên bản `spark_ml_salary_gbt_v1` để các bước batch và streaming có thể ghi nhận rõ nguồn dự đoán.

Một nguyên tắc nghiệp vụ quan trọng là mô hình không ghi đè lương thật. Nếu tin tuyển dụng đã có ít nhất một trong hai trường `salary_min_vnd` hoặc `salary_max_vnd`, hệ thống giữ nguyên lương đã parse được. Mô hình chỉ được áp dụng khi cả hai trường này đều rỗng. Nhờ đó, dự đoán lương đóng vai trò làm giàu dữ liệu, không thay thế dữ liệu gốc.

5.2 Trích xuất đặc trưng

Feature engineering được thiết kế để batch layer và speed layer sử dụng cùng một chuẩn đặc trưng. Điều này giúp mô hình được huấn luyện từ dữ liệu lịch sử ở tầng Silver vẫn có thể được dùng để dự đoán cho dữ liệu realtime trong Spark Structured Streaming. Trong mã nguồn, logic tạo đặc trưng được gom vào module `salary_prediction.py`, còn job `train_salary_model.py`, `silver_to_gold.py` và `stream_main.py` cùng gọi lại các hàm dùng chung này.

Các đặc trưng chính của mô hình được mô tả trong Bảng 5.2.

Với nhóm đặc trưng dạng văn bản, tiêu đề công việc được chuẩn hóa thành chuỗi và tách token bằng `RegexTokenizer`. Sau đó, `CountVectorizer` chuyển tập token thành vector đặc trưng. Cách làm này giúp mô hình khai thác các từ khóa quan trọng trong tiêu đề như `senior`, `junior`, `backend`, `data`, `java` hoặc `devops`. Đối với trường `skills`, dữ liệu đã ở dạng danh sách kỹ năng nên được đưa trực tiếp vào `CountVectorizer`. Nhóm đặc trưng này giúp mô hình học được ảnh hưởng của kỹ năng và công nghệ tới mức lương.

Với nhóm đặc trưng phân loại, `location` và `company_name` được xử lý bằng `StringIndexer` và `OneHotEncoder`. `StringIndexer` chuyển từng giá trị phân loại thành chỉ số, còn `OneHotEncoder` biến chỉ số đó thành vector thưa để mô hình sử dụng. Các stage này được cấu hình để xử lý giá trị chưa từng gặp bằng cơ chế `handleInvalid`, nhờ đó pipeline có thể chạy ổn định khi dữ liệu realtime xuất hiện công ty hoặc địa điểm mới.

Với nhóm đặc trưng số và boolean, `monthOfExperience` được dùng như một đặc trưng số biểu diễn kinh nghiệm, còn `has_remote` được ép về dạng số để biểu diễn tin tuyển dụng có yếu tố remote hay không. Trước khi đưa vào mô hình, các giá trị thiếu được

Bảng 5.2: Các đặc trưng sử dụng cho mô hình dự đoán lương

Đặc trưng	Nguồn dữ liệu	Ý nghĩa
<code>title</code>	Tiêu đề tin tuyển dụng	Mô tả vị trí, cấp bậc hoặc công nghệ chính của công việc.
<code>skills</code>	Danh sách kỹ năng	Biểu diễn yêu cầu kỹ thuật và năng lực chuyên môn.
<code>monthOfExperience</code> hoặc <code>experience_months</code>	Số tháng kinh nghiệm	Phản ánh mức seniority của ứng viên mục tiêu.
<code>location</code> hoặc <code>location_raw</code>	Địa điểm làm việc	Phản ánh khác biệt thị trường lương theo khu vực.
<code>company_name</code>	Tên công ty	Giúp mô hình học khác biệt lương giữa các nhóm doanh nghiệp.
<code>has_remote</code>	Thông tin remote	Biểu diễn việc tin tuyển dụng có yếu tố làm việc từ xa hay không.

chuẩn hóa về mặc định phù hợp, giúp pipeline không bị lỗi khi một số trường trong dữ liệu crawl bị rỗng.

Sau khi xử lý riêng từng nhóm, các vector đặc trưng được ghép lại bằng `VectorAssembler`. Cấu trúc pipeline tổng quát như sau:

```

title → RegexTokenizer → CountVectorizer
skills → CountVectorizer
location, company_name → StringIndexer → OneHotEncoder
monthOfExperience, has_remote → đặc trưng số
↓
VectorAssembler → GBRegressor

```

Các tham số chính của `GBRegressor` gồm `maxIter = 60`, `maxDepth = 5`, `maxBins = 64`, `minInstancesPerNode = 5`, `stepSize = 0.05`, `subsamplingRate = 0.8` và `seed = 42`. Trong quá trình huấn luyện, dữ liệu có lương công khai được chia thành tập train và test theo tỷ lệ 80/20. Mô hình được đánh giá bằng RMSE và MAE trên thang log salary. Sau khi train xong, mô hình và metrics được lưu lại để phục vụ quá trình scoring ở cả batch layer và speed layer.

5.3 Tích hợp model

Mô hình dự đoán lương được tích hợp vào cả batch layer và speed layer, phù hợp với kiến trúc Lambda Architecture của hệ thống. Batch layer chịu trách nhiệm huấn luyện mô hình từ dữ liệu lịch sử và áp dụng mô hình khi tạo Gold layer. Speed layer không train lại mô hình mà chỉ nạp mô hình mới nhất đã được batch layer huấn luyện để dự đoán cho dữ liệu realtime.

Trong batch layer, luồng xử lý liên quan đến mô hình lương có thứ tự như sau:

```

Raw → Bronze → Silver → Train Salary Model → Silver to Gold → Gold to
Elasticsearch

```

Sau khi dữ liệu được chuẩn hóa đến tầng Silver, job `train_salary_model.py` đọc toàn bộ bảng Silver, lọc ra các bản ghi có lương công khai, tạo nhãn huấn luyện và train mô hình Spark ML. Mô hình được lưu tại đường dẫn:

```
hdfs://hdfs-namenode.hdfs.svc:9000/models/salary_prediction/latest
```

Metrics của mô hình được lưu tại:

```
hdfs://hdfs-namenode.hdfs.svc:9000/models/salary_prediction/metrics/latest
```

Sau bước train, job `silver_to_gold.py` nạp mô hình từ HDFS và áp dụng lên dữ liệu Silver. Nếu mô hình nạp thành công, hệ thống gọi hàm scoring để bổ sung các trường dự đoán lương. Nếu mô hình chưa tồn tại hoặc nạp lỗi, pipeline vẫn tiếp tục chạy và tạo các cột dự đoán với giá trị `null`; nhờ đó schema của Gold layer vẫn ổn định cho các bước phía sau.

Quy tắc scoring trong batch layer là chỉ dự đoán khi thỏa mãn đồng thời ba điều kiện: `salary_min_vnd` rỗng, `salary_max_vnd` rỗng và mô hình được nạp thành công. Với các bản ghi đã có lương thật, hệ thống không dùng mô hình để ghi đè. Kết quả sau scoring được mô tả trong Bảng 5.3.

Bảng 5.3: Các trường lương sau khi tích hợp mô hình

Trường	Ý nghĩa
<code>salary_predicted_avg_vnd</code>	Lương trung bình do mô hình dự đoán.
<code>salary_predicted_min_vnd</code>	Cận dưới dự đoán, xấp xỉ 90% lương trung bình dự đoán.
<code>salary_predicted_max_vnd</code>	Cận trên dự đoán, xấp xỉ 110% lương trung bình dự đoán.
<code>salary_display_avg_vnd</code>	Lương trung bình cuối cùng dùng cho sort, filter và chart.
<code>salary_display_min_vnd</code>	Lương min cuối cùng dùng cho hiển thị và lọc.
<code>salary_display_max_vnd</code>	Lương max cuối cùng dùng cho hiển thị và lọc.
<code>salary_prediction_applied</code>	Bằng <code>true</code> nếu bản ghi thực sự được dự đoán bằng mô hình.
<code>salary_source</code>	Nguồn gốc của lương hiển thị.
<code>salary_prediction_model_version</code>	Phiên bản mô hình đã dùng khi dự đoán.

Trường `salary_source` giúp phân biệt rõ nguồn của giá trị lương cuối cùng. Nếu có cả min và max thật, giá trị là `parsed_range`; nếu chỉ có min thật là `parsed_min_only`; nếu chỉ có max thật là `parsed_max_only`; nếu không có lương thật nhưng đã dự đoán bằng mô hình là `predicted`; nếu không có cả lương thật lẫn dự đoán là `unknown`. Nhóm trường `salary_display_*` là nhóm trường nên được ưu tiên sử dụng ở Elasticsearch, Kibana hoặc API, vì chúng đã tự chọn lương thật khi có dữ liệu thật và chỉ dùng lương dự đoán khi cần thiết.

Trong speed layer, Spark Structured Streaming đọc dữ liệu từ Kafka topic `jobs_raw`, parse raw event, kiểm tra schema, chuẩn hóa dữ liệu và tạo các cột tương thích với feature

contract của mô hình. Mô hình được nạp một lần khi streaming job khởi động, sau đó được dùng để score từng micro-batch realtime. Luồng tích hợp realtime có thể mô tả như sau:

```
Kafka jobs_raw → Parse raw event → Build clean jobs → Load salary model  
→ Score salary prediction → Kafka jobs_clean / Elasticsearch realtime  
index
```

Speed layer sử dụng hai biến môi trường quan trọng là `ENABLE_SALARY_PREDICTION` và `SALARY_MODEL_PATH`. Khi `ENABLE_SALARY_PREDICTION=true`, streaming job sẽ cố gắng nạp mô hình từ đường dẫn HDFS đã cấu hình. Nếu mô hình chưa tồn tại hoặc không nạp được, stream vẫn tiếp tục chạy, các trường `salary_predicted_*` nhận giá trị `null`, `salary_prediction_applied=false`, còn `salary_source` vẫn phản ánh lương parse được nếu có. Nếu batch layer train ra mô hình mới, streaming job cần được khởi động lại để nạp phiên bản mô hình mới nhất.

Tóm lại, mô hình dự đoán lương được tích hợp như một bước làm giàu dữ liệu ở cả batch và realtime. Thiết kế này giữ được tính chính xác của các mức lương thật đã crawl được, đồng thời bổ sung khả năng phân tích cho các tin tuyển dụng chưa công khai lương. Nhờ đó, các chức năng như lọc theo lương, sắp xếp theo lương, thống kê thị trường và trực quan hóa dashboard trở nên đầy đủ hơn.

Chương 6

TRỰC QUAN HÓA DỮ LIỆU

Sau khi dữ liệu được xử lý ở batch layer và speed layer, hệ thống sử dụng Elasticsearch làm tầng lưu trữ phục vụ truy vấn, phân tích và trực quan hóa. Dữ liệu batch sau khi được tổng hợp ở Gold layer được ghi vào Elasticsearch để phục vụ các biểu đồ phân tích thị trường lao động. Dữ liệu streaming sau khi được Spark Structured Streaming xử lý cũng được ghi vào các Elasticsearch index realtime để theo dõi xu hướng dữ liệu gần thời gian thực.

Kibana được sử dụng làm công cụ trực quan hóa dữ liệu. Các dashboard trong Kibana kết nối trực tiếp tới Elasticsearch, từ đó hiển thị các chỉ số như số lượng tin tuyển dụng, phân bố mức lương, kỹ năng phổ biến, xu hướng việc làm theo thời gian và các thống kê realtime từ speed layer.

6.1 Lưu trữ dữ liệu trên Elasticsearch

Elasticsearch đóng vai trò là tầng lưu trữ tìm kiếm và phân tích của hệ thống. Sau khi dữ liệu được xử lý bởi Spark, các bản ghi đã được chuẩn hóa sẽ được ghi vào Elasticsearch dưới dạng các index. Việc sử dụng Elasticsearch giúp hệ thống truy vấn dữ liệu nhanh, hỗ trợ lọc theo nhiều trường như vị trí công việc, công ty, địa điểm, mức lương, kỹ năng và thời gian cập nhật.

Trong project, Elasticsearch được triển khai trong namespace `search`. Thành phần này được cấu hình dưới dạng StatefulSet để đảm bảo dữ liệu index được lưu ổn định. Các service nội bộ trong Kubernetes cho phép Spark và Kibana kết nối tới Elasticsearch thông qua địa chỉ service trong cụm.

6.1.1 Batch index

Với batch layer, dữ liệu sau khi được xử lý qua các lớp Raw, Bronze, Silver và Gold sẽ được ghi sang Elasticsearch bởi job `gold_to_elasticsearch.py`. Index chính của batch layer là:

`gold-jobs-flat`

Index `gold-jobs-flat` lưu dữ liệu việc làm ở dạng đã được làm sạch, chuẩn hóa và làm giàu. Mỗi document trong index đại diện cho một tin tuyển dụng sau xử lý. Dữ liệu trong index này được sử dụng để xây dựng các dashboard phân tích tổng quan về thị trường lao động.

Một số nhóm thông tin chính trong batch index gồm:

- Thông tin công việc: tiêu đề, công ty, đường dẫn tin tuyển dụng.
- Thông tin địa điểm và hình thức làm việc.
- Thông tin mức lương đã được chuẩn hóa.
- Thông tin kinh nghiệm, kỹ năng và nhóm ngành.
- Các trường bổ sung phục vụ thống kê và trực quan hóa.

6.1.2 Realtime indexes

Với speed layer, Spark Structured Streaming đọc dữ liệu từ Kafka topic `jobs_raw`, kiểm tra schema, chuẩn hóa dữ liệu và ghi kết quả sang Elasticsearch. Hệ thống sử dụng nhiều index realtime để tách dữ liệu chi tiết và dữ liệu tổng hợp theo thời gian.

Các index realtime chính được mô tả trong Bảng 6.1.

Bảng 6.1: Các Elasticsearch index của speed layer

Index	Loại dữ liệu	Vai trò
<code>realtime_jobs_v1</code>	Job-level	Lưu từng tin tuyển dụng realtime đã được làm sạch, phục vụ kiểm tra dữ liệu mới và filter theo job.
<code>realtime_job_counts_10m_v1</code>	Aggregate	Lưu số lượng tin tuyển dụng theo cửa sổ thời gian 10 phút.
<code>realtime_skill_counts_hourly_v1</code>	Aggregate	Lưu số lượng tin tuyển dụng theo từng kỹ năng trong mỗi cửa sổ 1 giờ.
<code>realtime_top_skills_hourly_v1</code>	Aggregate	Lưu danh sách kỹ năng phổ biến nhất theo từng giờ.
<code>realtime_salary_bins_hourly_v1</code>	Aggregate	Lưu phân bố mức lương theo các khoảng lương trong từng cửa sổ 1 giờ.

Index `realtime_jobs_v1` được dùng để lưu dữ liệu chi tiết của từng tin tuyển dụng trong speed layer. Các index còn lại là các bảng tổng hợp phục vụ dashboard realtime. Việc tách riêng các index aggregate giúp Kibana hiển thị biểu đồ nhanh hơn, vì dữ liệu đã được Spark tính toán trước thay vì phải tổng hợp trực tiếp từ toàn bộ dữ liệu job-level.

Hình 6.1 minh họa danh sách các index thực tế trong Elasticsearch sau khi hệ thống chạy batch layer và speed layer. Index `gold-jobs-flat` chứa dữ liệu Gold của batch pipeline với 5424 document, tương ứng với số tin tuyển dụng duy nhất theo `job_id` sau khi gom các file Raw trên HDFS. Các index `realtime_*` được tạo bởi Spark Structured Streaming để lưu dữ liệu realtime chi tiết và các bảng tổng hợp phục vụ dashboard gần thời gian thực.

	health	status	index	pri	rep	docs.count
1	yellow	open	gold-jobs-flat	1	1	5424
2	yellow	open	realtime_job_counts_10m_v1	1	1	397
3	yellow	open	realtime_jobs_v1	1	1	669
4	yellow	open	realtime_salary_bins_hourly_v1	1	1	277
5	yellow	open	realtime_skill_counts_hourly_v1	1	1	4775
6	yellow	open	realtime_top_skills_hourly_v1	1	1	231

Hình 6.1: Danh sách các Elasticsearch index của hệ thống, bao gồm index batch `gold-jobs-flat` và các index realtime phục vụ trực quan hóa gần thời gian thực.

6.1.3 Vai trò của Elasticsearch trong hệ thống

Elasticsearch là điểm giao giữa tầng xử lý dữ liệu và tầng trực quan hóa. Batch layer ghi dữ liệu lịch sử đã xử lý vào index `gold-jobs-flat`, còn speed layer ghi dữ liệu realtime và các bảng metrics vào nhóm index `realtime_*`. Cách tổ chức này giúp hệ thống phục vụ đồng thời hai nhu cầu:

- Phân tích dữ liệu lịch sử về thị trường lao động thông qua batch dashboard.
- Theo dõi dữ liệu mới và xu hướng realtime thông qua realtime dashboard.

Nhờ Elasticsearch, Kibana có thể truy vấn trực tiếp các index đã được xử lý và hiển thị kết quả dưới dạng biểu đồ, bảng thống kê và dashboard tương tác.

6.2 Trực quan hóa dữ liệu sử dụng Kibana Dashboard

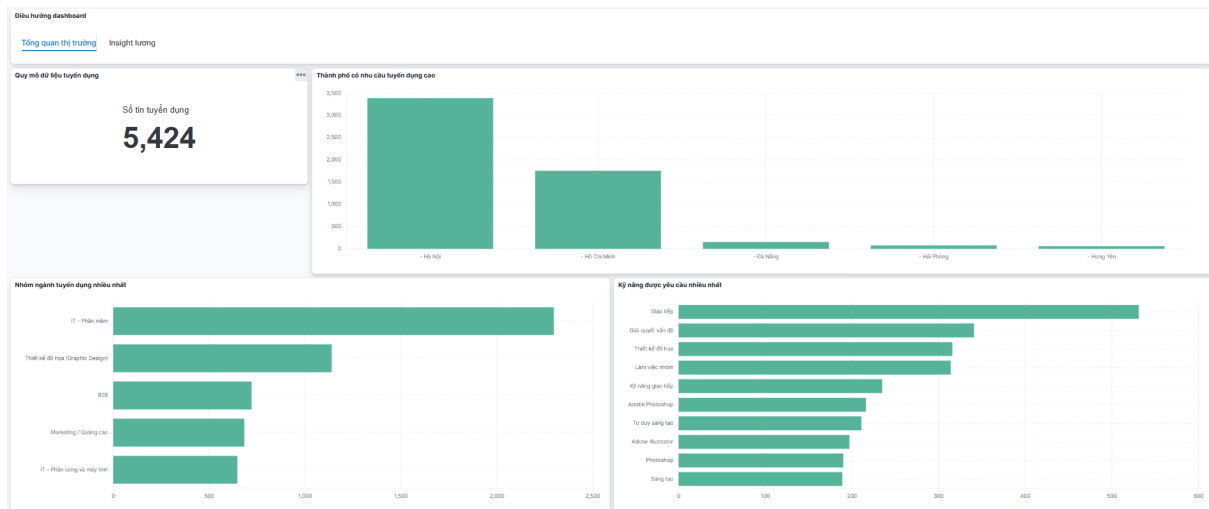
Kibana được sử dụng để xây dựng dashboard trực quan hóa dữ liệu thị trường lao động. Công cụ này kết nối trực tiếp tới Elasticsearch, sử dụng các index pattern tương ứng với batch index và realtime indexes để tạo biểu đồ phân tích.

Trong project, Kibana được triển khai trong namespace `search`. Các dashboard và saved objects được lưu trong thư mục `infra/kibana/saved_objects/`. Việc export dashboard ra file giúp nhóm có thể tái sử dụng hoặc import lại dashboard khi dựng lại môi trường Minikube.

6.2.1 Dashboard cho batch layer

Dashboard batch sử dụng index `gold-jobs-flat`. Đây là index chứa dữ liệu việc làm đã được xử lý qua toàn bộ batch pipeline, bao gồm các bước làm sạch, chuẩn hóa, trích xuất thông tin và bổ sung các trường phục vụ phân tích. Do dữ liệu trong index này đã ở lớp Gold, dashboard batch phù hợp cho các phân tích tổng quan về thị trường tuyển dụng theo địa điểm, nhóm ngành, kỹ năng và mức lương.

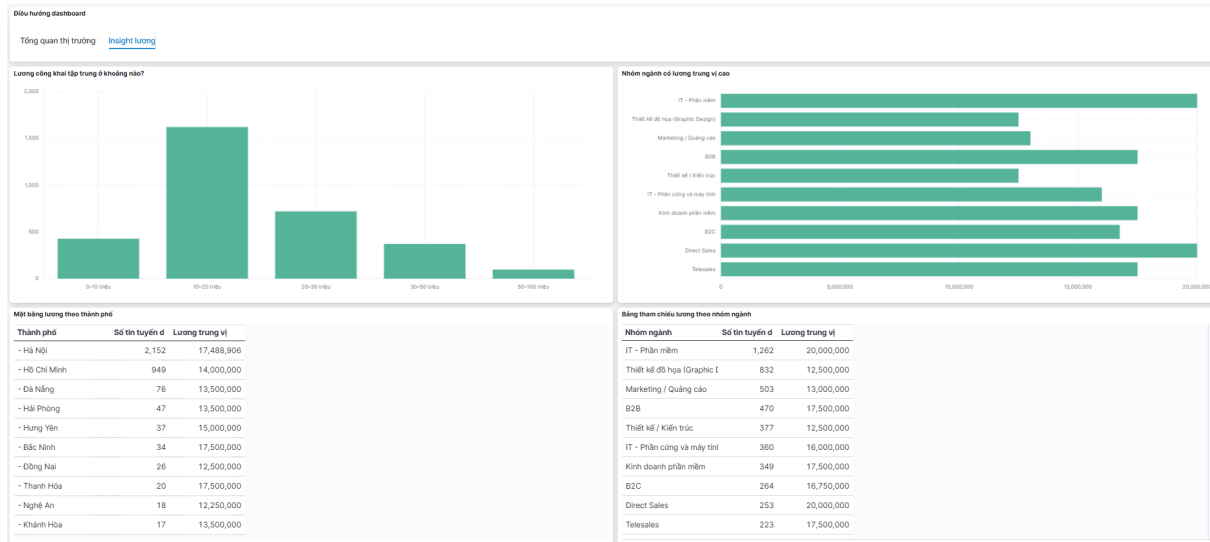
Trong hệ thống, dashboard batch được tổ chức thành hai nhóm chính: *Tổng quan thị trường* và *Insight lương*. Nhóm tổng quan thị trường tập trung vào các câu hỏi như: hệ thống đang có bao nhiêu tin tuyển dụng, nhu cầu tuyển dụng tập trung ở thành phố nào, nhóm ngành nào có số lượng tin lớn và các kỹ năng nào được nhà tuyển dụng nhắc đến nhiều nhất.



Hình 6.2: Dashboard tổng quan thị trường tuyển dụng, thể hiện quy mô dữ liệu, phân bố nhu cầu tuyển dụng theo thành phố, nhóm ngành và kỹ năng được nhà tuyển dụng yêu cầu nhiều nhất.

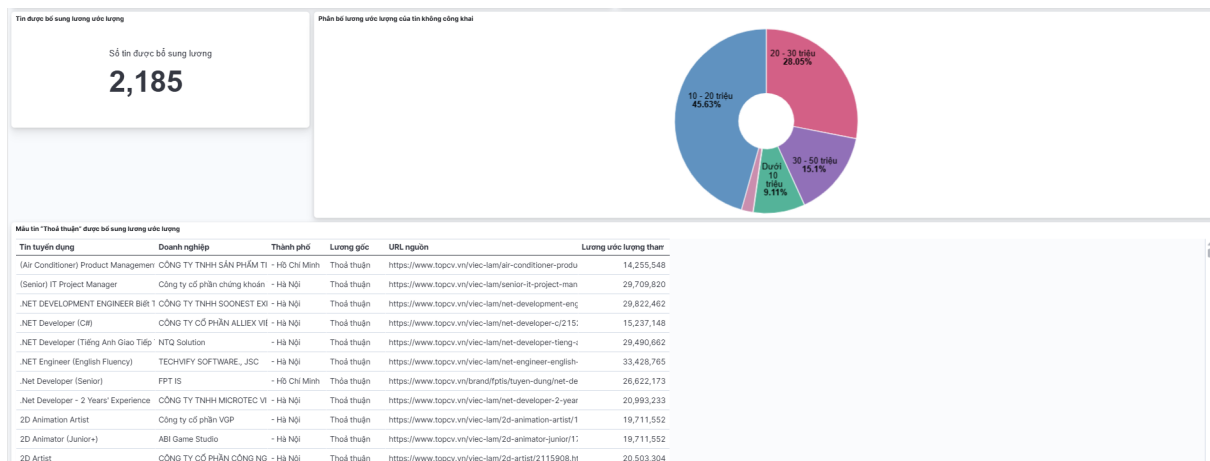
Từ dashboard ở Hình 6.2, có thể thấy dữ liệu tuyển dụng tập trung chủ yếu tại các thành phố lớn như Hà Nội và Thành phố Hồ Chí Minh. Nhóm ngành *IT - Phần mềm* chiếm số lượng tin tuyển dụng nổi bật, cho thấy nhu cầu nhân lực công nghệ thông tin là nhóm nhu cầu chính trong tập dữ liệu. Ngoài ra, các kỹ năng như giao tiếp, giải quyết vấn đề, làm việc nhóm và thiết kế đồ họa xuất hiện nhiều trong mô tả tuyển dụng, phản ánh xu hướng nhà tuyển dụng không chỉ quan tâm đến kỹ năng chuyên môn mà còn chú trọng các kỹ năng mềm.

Bên cạnh phần tổng quan, dashboard batch còn có nhóm *Insight lương* để phân tích mặt bằng lương từ các tin tuyển dụng có lương công khai. Các biểu đồ trong nhóm này tập trung vào phân bố khoảng lương phổ biến, lương trung vị theo nhóm ngành và mặt bằng lương theo thành phố. Ngoài ra, kết quả từ task `train_salary_model` cũng được sử dụng để bổ sung mức lương ước lượng tham khảo cho các tin tuyển dụng có lương gốc là “Thoả thuận”, giúp dữ liệu Gold đầy đủ hơn cho quá trình phân tích.



Hình 6.3: Dashboard phân tích lương từ các tin tuyển dụng có lương công khai, thể hiện phân bố khoảng lương phổ biến, nhóm ngành có lương trung vị cao và mật bảng lương theo thành phố.

Hình 6.3 cho thấy phần lớn các tin tuyển dụng có lương công khai tập trung trong khoảng 10–20 triệu đồng. Các nhóm ngành như *IT - Phần mềm*, *Direct Sales*, *B2B* và *Kinh doanh phần mềm* có mật bảng lương trung vị tương đối cao. Khi phân tích theo địa điểm, Hà Nội và Thành phố Hồ Chí Minh là hai khu vực có số lượng tin tuyển dụng lớn nhất, đồng thời cũng là hai khu vực quan trọng trong phân tích mật bảng lương.



Hình 6.4: Minh họa kết quả làm giàu dữ liệu lương bằng mô hình học máy, trong đó các tin tuyển dụng có lương gốc là “Thoả thuận” được bổ sung mức lương ước lượng tham khảo ở lớp Gold.

Hình 6.4 minh họa cách kết quả của task `train_salary_model` được sử dụng trong dashboard. Với các tin tuyển dụng không công khai lương, hệ thống bổ sung mức lương ước lượng tham khảo để tăng độ đầy đủ của dữ liệu phân tích. Biểu đồ phân bố cho thấy các tin được bổ sung lương chủ yếu nằm trong khoảng 10–20 triệu và 20–30 triệu đồng.

Bảng bên dưới cung cấp một số ví dụ cụ thể gồm tiêu đề tin tuyển dụng, doanh nghiệp, thành phố, lương gốc, URL nguồn và mức lương ước lượng tham khảo.

6.2.2 Dashboard cho speed layer

Dashboard realtime sử dụng các index có tiền tố `realtime_`. Các index này được cập nhật bởi Spark Structured Streaming trong quá trình xử lý dữ liệu từ Kafka.

Các nhóm biểu đồ realtime gồm:

- Số lượng tin tuyển dụng mới theo cửa sổ 10 phút.
- Top kỹ năng xuất hiện nhiều nhất theo từng giờ.
- Phân bố mức lương realtime theo từng khoảng lương.
- Danh sách các tin tuyển dụng mới được ghi vào `realtime_jobs_v1`.
- Trạng thái cập nhật dữ liệu gần thời gian thực của speed layer.

Realtime dashboard giúp theo dõi luồng dữ liệu mới từ TopCV sau khi đi qua Kafka và Spark Structured Streaming. Nhờ các bảng aggregate đã được tính toán trước, dashboard có thể hiển thị nhanh các xu hướng mới như số lượng job tăng theo thời gian, kỹ năng nổi bật trong giờ gần nhất hoặc sự thay đổi của phân bố mức lương.

6.2.3 Quy trình import và sử dụng dashboard

Để sử dụng dashboard, nhóm triển khai Elasticsearch và Kibana trong namespace `search`, sau đó import các saved objects đã chuẩn bị sẵn trong thư mục `infra/kibana/saved_objects/`. Sau khi import, Kibana sẽ có các data view tương ứng với batch index và realtime indexes.

Quy trình sử dụng dashboard gồm các bước chính:

1. Chạy batch pipeline để tạo dữ liệu Gold và ghi vào index `gold-jobs-flat`.
2. Khởi động speed layer để Spark Structured Streaming ghi dữ liệu vào các index `realtime_*`.
3. Truy cập Kibana thông qua service hoặc port-forward.
4. Import saved objects từ thư mục `infra/kibana/saved_objects/`.
5. Mở dashboard batch hoặc realtime để kiểm tra kết quả trực quan hóa.

Nhờ Kibana, kết quả xử lý dữ liệu không chỉ dừng lại ở các file lưu trữ trên HDFS mà còn được trình bày dưới dạng dashboard trực quan. Điều này giúp người dùng dễ dàng quan sát xu hướng thị trường lao động, so sánh mức lương, phân tích kỹ năng phổ biến và theo dõi dữ liệu mới trong speed layer.

Chương 7

BÀI HỌC KINH NGHIỆM

Quá trình xây dựng hệ thống phân tích thị trường việc làm theo kiến trúc Lambda Architecture không chỉ tạo ra pipeline dữ liệu hoàn chỉnh từ thu thập, xử lý, lưu trữ đến trực quan hóa, mà còn cung cấp nhiều kinh nghiệm thực tế về thiết kế hệ thống dữ liệu lớn. Các bài học dưới đây được tổng hợp từ những thành phần đã triển khai trong repo, bao gồm crawler TopCV, HDFS Raw/Bronze/Silver/Gold, Kafka, Spark Batch, Spark Structured Streaming, Elasticsearch, Kibana, Airflow và Kubernetes/Minikube.

7.1 Bài học 1: Crawler cần có cơ chế fallback khi cấu trúc website thay đổi

7.1.1 Mô tả vấn đề

Dữ liệu việc làm được thu thập từ TopCV, trong đó thông tin của một job có thể nằm ở nhiều vị trí khác nhau như JSON-LD, các thẻ HTML hoặc chỉ xuất hiện trong phần text của trang. Thách thức lớn là cấu trúc HTML của website không cố định: một số job có đầy đủ dữ liệu có cấu trúc, nhưng một số job lại thiếu trường hoặc được hiển thị theo layout khác.

Nếu crawler chỉ phụ thuộc vào một selector cố định, hệ thống rất dễ mất dữ liệu khi website thay đổi giao diện hoặc khi từng tin tuyển dụng có cấu trúc khác nhau. Điều này ảnh hưởng trực tiếp đến chất lượng dữ liệu đầu vào cho Kafka, HDFS và các tầng xử lý sau.

7.1.2 Cách tiếp cận đã thử

Cách đầu tiên là chỉ lấy dữ liệu bằng CSS selector từ HTML. Cách này dễ triển khai và code đơn giản, nhưng dễ hỏng khi website thay đổi class, layout hoặc cách trình bày nội dung.

Cách thứ hai là chỉ lấy toàn bộ text của trang rồi xử lý bằng regex. Cách này ít phụ thuộc vào HTML hơn, nhưng độ chính xác thấp, dễ lấy nhầm thông tin và khó tách chính xác từng trường nghiệp vụ.

Cách thứ ba là kết hợp nhiều nguồn trích xuất theo thứ tự ưu tiên. Cách này tăng khả năng lấy được dữ liệu trong nhiều tình huống khác nhau, nhưng parser phức tạp hơn và cần kiểm tra nhiều trường hợp lỗi.

7.1.3 Giải pháp cuối cùng

Hệ thống sử dụng chiến lược parse nhiều tầng. Trước hết, crawler ưu tiên lấy dữ liệu từ JSON-LD dạng `JobPosting` nếu trang có cung cấp. Nếu JSON-LD thiếu trường, crawler tiếp tục lấy dữ liệu từ các selector HTML. Nếu HTML cũng không đủ, crawler fallback sang `pageText` và regex để tìm các thông tin còn thiếu.

Các trường quan trọng như tiêu đề việc làm, tên công ty, lương, địa điểm, kinh nghiệm, deadline, mô tả công việc, yêu cầu và quyền lợi đều được lấy theo hướng ưu tiên nguồn tin cậy hơn, sau đó fallback sang nguồn tổng quát hơn. Ngoài dữ liệu chính, crawler còn lưu thêm các thông tin hỗ trợ như `pageText`, `json_ld`, `sectionsByHeading` và `quality_flags` để phục vụ kiểm tra lại khi parser bị lỗi.

7.1.4 Điểm rút ra

Khi crawl dữ liệu từ website thật, không nên giả định cấu trúc trang luôn ổn định. Parser nên được thiết kế theo hướng nhiều tầng: structured data, HTML selector và text/regex fallback.

Việc lưu thêm dữ liệu thô và metadata giúp dễ debug, dễ cải thiện parser và giảm rủi ro mất dữ liệu khi website thay đổi.

7.2 Bài học 2: Chống anti-bot cân cân bằng giữa tốc độ và độ ổn định

7.2.1 Mô tả vấn đề

Website tuyển dụng có thể áp dụng các cơ chế anti-bot để hạn chế crawler gửi quá nhiều request trong thời gian ngắn. Nếu crawler gửi request liên tục với tần suất cao, hệ thống có thể bị block, nhận HTML rỗng, bị chuyển sang trang xác minh hoặc không lấy được danh sách job.

Khi đó, pipeline phía sau có thể nhận dữ liệu thiếu, dữ liệu sai hoặc crawler dừng sớm vì hiểu nhầm rằng không còn job mới.

7.2.2 Cách tiếp cận đã thử

Cách đầu tiên là crawl với tốc độ cao nhất có thể. Cách này lấy dữ liệu nhanh khi chưa bị chặn, nhưng dễ bị website phát hiện và block, làm quá trình crawl thiếu ổn định.

Cách thứ hai là mỗi lần request tạo session/cookie mới. Cách này có thể vượt qua một số lỗi session, nhưng hành vi không tự nhiên, dễ bị nghi ngờ hơn và khó duy trì trạng thái ổn định.

Cách thứ ba là giả lập hành vi truy cập ổn định, thêm delay, cooldown và retry có kiểm soát. Cách này giảm nguy cơ bị block và tăng độ ổn định khi crawl lâu, đổi lại tốc độ crawl chậm hơn so với gửi request liên tục.

7.2.3 Giải pháp cuối cùng

Crawler sử dụng session giả lập trình duyệt để request tới website. Cookie và user-agent được tái sử dụng để duy trì phiên truy cập ổn định hơn. Giữa các request danh sách và request chi tiết có thời gian nghỉ ngẫu nhiên, tránh tạo pattern request quá đều.

Sau một số lượng request nhất định, crawler có cơ chế cooldown để giảm tải và tránh bị website đánh dấu là bot. Khi gặp dấu hiệu bị block hoặc request lỗi, crawler không kết luận ngay rằng dữ liệu không tồn tại, mà thực hiện retry, recover session/cookie hoặc ghi nhận link lỗi để xử lý lại.

7.2.4 Điểm rút ra

Trong crawl dữ liệu thực tế, “nhanh nhất” không phải lúc nào cũng là “tốt nhất”. Cần đánh đổi giữa tốc độ crawl và khả năng chạy ổn định trong thời gian dài.

Một crawler tốt cần có delay ngẫu nhiên, retry có giới hạn, cooldown, phát hiện block và log lỗi rõ ràng. Không nên coi response rỗng hoặc thiếu HTML là dữ liệu hợp lệ, vì đó có thể là dấu hiệu bị anti-bot.

7.3 Bài học 3: Tránh trùng lặp dữ liệu phải xét cả định danh và sự thay đổi nội dung

7.3.1 Mô tả vấn đề

Một tin tuyển dụng có thể xuất hiện nhiều lần trong các lần crawl khác nhau, đặc biệt trong speed layer khi crawler chạy định kỳ. Nếu không xử lý trùng lặp, cùng một job có thể bị đẩy nhiều lần vào Kafka, HDFS và Elasticsearch, làm tăng dung lượng lưu trữ và gây sai lệch khi phân tích.

Tuy nhiên, nếu bỏ trùng quá cứng theo URL hoặc `job_id`, hệ thống có thể bỏ sót trường hợp job cũ nhưng vừa được cập nhật nội dung như lương, deadline, yêu cầu hoặc địa điểm.

7.3.2 Cách tiếp cận đã thử

Cách đầu tiên là không xử lý trùng lặp. Cách này không bỏ sót dữ liệu, nhưng sinh nhiều bản ghi lặp và gây nhiễu cho downstream.

Cách thứ hai là bỏ qua mọi job đã từng thấy theo URL hoặc `job_id`. Cách này giảm dữ liệu trùng, nhưng có thể làm mất bản cập nhật mới của cùng một job.

Cách thứ ba là kết hợp `job_id`, thời gian cập nhật và `hash_content`. Cách này vừa giảm trùng lặp, vừa phát hiện được trường hợp nội dung job thay đổi, nhưng cần thêm metadata và cache để so sánh.

7.3.3 Giải pháp cuối cùng

Mỗi job được chuẩn hóa URL và tạo `job_id` từ nguồn dữ liệu và URL đã chuẩn hóa. Crawler tạo thêm `hash_content` từ các trường nội dung quan trọng của job để nhận biết nội dung có thay đổi hay không.

Với speed layer, hệ thống dùng cache để ghi nhớ những `job_id` đã xử lý gần đây, từ đó tránh phát lặp cùng một job vào Kafka. Nếu job đã tồn tại trong cache nhưng thời gian cập nhật trên trang danh sách mới hơn lần đã xử lý trước, crawler vẫn có thể crawl lại để kiểm tra thay đổi nội dung.

Với batch layer, hệ thống không dùng cache speed quá chặt, vì batch cần ưu tiên độ đầy đủ và không được bỏ sót những job cũ vừa được cập nhật.

7.3.4 Điểm rút ra

Tránh trùng lặp không chỉ là kiểm tra “URL này đã tồn tại chưa”. Cần phân biệt ba trường hợp: job hoàn toàn mới, job cũ bị crawl lặp và job cũ nhưng nội dung đã thay đổi.

Nên kết hợp `job_id`, `updated_time` và `hash_content` để cân bằng giữa chống trùng lặp và không bỏ sót bản cập nhật. Speed layer nên chống trùng mạnh hơn, còn batch layer nên ưu tiên thu thập đầy đủ hơn.

7.4 Bài học 4: Chất lượng dữ liệu cần được kiểm soát ngay từ tầng raw

7.4.1 Mô tả vấn đề

Dữ liệu crawl từ website thường không đồng nhất: có job thiếu lương, thiếu kinh nghiệm, thiếu mô tả, deadline không rõ ràng hoặc địa điểm được viết theo nhiều định dạng khác nhau. Nếu dữ liệu lỗi được đẩy thẳng vào các tầng xử lý sau mà không có đánh dấu chất lượng, Spark batch, streaming job hoặc Elasticsearch có thể nhận dữ liệu thiếu và tạo ra kết quả phân tích sai.

Việc debug cũng khó hơn nếu không biết record nào thiếu trường, record nào parse từ JSON-LD, record nào fallback từ text.

7.4.2 Cách tiếp cận đã thử

Cách đầu tiên là chỉ lưu các trường nghiệp vụ đã parse được. Cách này giúp dữ liệu gọn, dễ đọc, nhưng mất thông tin để kiểm tra lỗi và khó đánh giá độ tin cậy của record.

Cách thứ hai là lưu toàn bộ HTML gốc của trang. Cách này đầy đủ và có thể parse lại bất cứ lúc nào, nhưng dung lượng lớn và khó xử lý trong pipeline big data.

Cách thứ ba là lưu dữ liệu chính kèm metadata và `quality_flags`. Cách này vẫn giữ được khả năng kiểm tra chất lượng mà không làm dữ liệu quá nặng, nhưng schema raw phức tạp hơn.

7.4.3 Giải pháp cuối cùng

Mỗi record raw không chỉ chứa thông tin việc làm, mà còn có metadata phục vụ truy vết như `source`, `source_url`, `normalized_source_url`, `job_id`, `hash_content`, `ingest_ts`, `event_ts` và `crawl_version`.

Crawler sinh thêm `quality_flags` để đánh dấu tình trạng dữ liệu, ví dụ có lấy được JSON-LD hay không, có lấy được page text hay không, có tiêu đề, công ty, lương, địa điểm, kinh nghiệm, mô tả, yêu cầu, quyền lợi hay không và deadline có parse được hay không.

Dữ liệu raw được giữ lại ở JSONL/HDFS trước khi đưa qua các tầng làm sạch. Các tầng sau có thể dựa vào metadata và quality flags để validate, lọc lỗi, đưa record lỗi vào dead-letter queue hoặc xử lý bổ sung.

7.4.4 Điểm rút ra

Chất lượng dữ liệu không nên chỉ kiểm tra ở cuối pipeline, mà cần được kiểm soát ngay từ lúc crawl. Raw record nên chứa cả dữ liệu nghiệp vụ, metadata và cờ chất lượng.

`quality_flags` giúp hệ thống phát hiện sớm lỗi parser, đánh giá độ tin cậy của dữ liệu và hỗ trợ debug khi downstream có kết quả bất thường. Trong hệ thống big data, lưu raw data có kiểm soát là nền tảng để replay, audit và cải thiện pipeline về sau.

7.5 Bài học 5: Xử lý dữ liệu với Spark cần tối ưu theo luồng dữ liệu và giới hạn tài nguyên thực tế

7.5.1 Mô tả vấn đề

Spark được sử dụng cho các job Raw to Bronze, Bronze to Silver, Silver to Gold, huấn luyện mô hình dự báo lương và ghi dữ liệu sang Elasticsearch. Trong môi trường Minikube/local, tài nguyên CPU và RAM có giới hạn nên các job Spark dễ gặp vấn đề về thời gian chạy, shuffle lớn, thiếu bộ nhớ hoặc ghi đè nhầm partition.

Tác động của các vấn đề này là pipeline batch có thể chạy chậm, thất bại giữa chừng hoặc tạo ra dữ liệu không nhất quán giữa các lớp HDFS.

7.5.2 Cách tiếp cận đã thử

Cách đầu tiên là chạy toàn bộ dữ liệu theo chế độ full load để đơn giản hóa logic. Cách này dễ kiểm soát nhưng tốn thời gian và tài nguyên khi dữ liệu tăng. Cách thứ hai là chạy incremental theo ngày crawl, giảm khối lượng xử lý nhưng cần quản lý partition và trạng thái dữ liệu cẩn thận hơn.

Về mặt Spark configuration, có thể tăng số executor/cores để chạy nhanh hơn, nhưng trên Minikube điều này dễ làm các pod cạnh tranh tài nguyên với Kafka, HDFS, Elasticsearch và Airflow.

7.5.3 Giải pháp cuối cùng

Dự án thiết kế batch pipeline theo các lớp dữ liệu rõ ràng và hỗ trợ cả full load lẫn incremental. Dữ liệu được partition theo thời gian crawl/ngày xử lý, sử dụng định dạng Parquet và nén Snappy. Spark được cấu hình các tham số như `spark.sql.parquet.compression.codec=snappy` và `spark.sql.sources.partition-OverwriteMode=dynamic` để giảm dung lượng lưu trữ và tránh ghi đè toàn bộ dữ liệu lịch sử.

Các job được chia nhỏ theo từng bước ETL thay vì gom vào một job lớn. Airflow điều phối tuần tự giúp dễ retry từng bước, dễ quan sát log và giảm phạm vi lỗi.

7.5.4 Điểm rút ra

Tối ưu Spark không chỉ là tăng tài nguyên, mà là giảm dữ liệu phải đọc, giảm shuffle, chọn partition phù hợp và tách pipeline thành các bước có thể kiểm soát. Với môi trường nhỏ như Minikube, tối ưu theo chi phí tài nguyên quan trọng hơn tối ưu tuyệt đối về tốc độ.

7.6 Bài học 6: Xử lý luồng cần thiết kế rõ về checkpoint, cửa sổ thời gian và trạng thái

7.6.1 Mô tả vấn đề

Speed layer sử dụng Spark Structured Streaming đọc dữ liệu từ Kafka, chuẩn hóa realtime, ghi dữ liệu sạch ra `jobs_clean`, ghi lỗi ra `jobs_dead_letter` và ghi kết quả realtime sang Elasticsearch. Các aggregation như số lượng job theo 10 phút, top kỹ năng theo giờ và phân bố lương theo giờ đều cần trạng thái theo thời gian.

Thách thức là dữ liệu có thể đến trễ, trùng lặp, job streaming có thể bị restart và state của aggregation có thể tăng dần nếu không có watermark hoặc checkpoint hợp lý.

7.6.2 Cách tiếp cận đã thử

Một cách tiếp cận đơn giản là xử lý từng message độc lập và ghi thẳng sang Elasticsearch. Cách này phù hợp với index `realtime_jobs_v1` nhưng không đủ cho aggregation theo cửa sổ thời gian. Cách khác là tính aggregation thủ công bên ngoài Spark, nhưng sẽ làm mất lợi ích của Structured Streaming về state management và checkpoint.

7.6.3 Giải pháp cuối cùng

Dự án sử dụng Spark Structured Streaming với checkpoint directory riêng cho từng query. Các tham số như `STARTING_OFFSETS`, `TRIGGER_SECONDS`, `CHECKPOINT_DIR` và các cờ `ENABLE_*` được đưa vào runtime config để điều khiển linh hoạt. Các luồng aggregation dùng window theo thời gian, watermark và checkpoint để phục hồi offset Kafka cũng như trạng thái khi restart.

Việc ghi realtime được tách theo từng index Elasticsearch: bản ghi sạch, job count 10 phút, skill count hourly, top skills hourly và salary bins hourly. Điều này giúp giảm coupling giữa các loại dữ liệu realtime.

7.6.4 Điểm rút ra

Trong streaming, exactly-once không chỉ phụ thuộc vào Spark mà còn phụ thuộc vào sink, checkpoint và khóa ghi idempotent. Cần thiết kế document id, checkpoint path và chính sách retry ngay từ đầu. Với aggregation có state, watermark là bắt buộc để cân bằng giữa độ chính xác và kích thước state.

7.7 Bài học 7: Lưu trữ dữ liệu cần phân tầng theo giá trị sử dụng và vòng đời dữ liệu

7.7.1 Mô tả vấn đề

Hệ thống có nhiều loại dữ liệu: JSONL thô từ crawler, dữ liệu Bronze/Silver/Gold trên HDFS, dữ liệu realtime trong Kafka, index phân tích trên Elasticsearch và model Spark ML trên HDFS. Mỗi loại có yêu cầu khác nhau về độ bền, tốc độ truy vấn, khả năng ghi và chi phí lưu trữ.

Nếu dùng cùng một định dạng và cùng một nơi lưu trữ cho tất cả dữ liệu, hệ thống sẽ khó tối ưu. Ví dụ JSONL phù hợp để lưu Raw nhưng không phù hợp cho truy vấn phân tích; Elasticsearch phù hợp cho dashboard nhưng không phải nơi lưu lịch sử đầy đủ thay cho HDFS.

7.7.2 Cách tiếp cận đã thử

Có thể chỉ lưu dữ liệu cuối cùng ở Gold và Elasticsearch để đơn giản hóa vận hành. Tuy nhiên cách này làm mất khả năng tái xử lý khi logic nghiệp vụ thay đổi. Ngược lại, lưu mọi thứ ở mọi nơi sẽ làm tăng dung lượng và phức tạp đồng bộ.

7.7.3 Giải pháp cuối cùng

Dự án áp dụng mô hình nhiều lớp: Raw lưu JSONL giữ nguyên nguồn gốc; Bronze chuẩn hóa kỹ thuật; Silver chuẩn hóa nghiệp vụ; Gold tối ưu cho phân tích; Elasticsearch phục vụ tìm kiếm, filter và dashboard. Parquet/Snappy được dùng cho dữ liệu phân tích trên HDFS, còn Elasticsearch index được thiết kế riêng cho batch và realtime.

Dữ liệu nóng như `realtime_*` được ghi vào Elasticsearch để truy vấn nhanh. Dữ liệu lạnh/lịch sử được giữ trên HDFS để phục vụ tái xử lý và kiểm toán.

7.7.4 Điểm rút ra

Không có một hệ lưu trữ tối ưu cho mọi mục đích. Cần phân biệt rõ data lake, message broker và search engine. Định dạng lưu trữ, partition và nén phải được chọn theo cách dữ liệu được đọc lại trong tương lai, không chỉ theo cách dữ liệu được ghi vào hiện tại.

7.8 Bài học 8: Tích hợp hệ thống phân tán cần chuẩn hóa service discovery và cơ chế lỗi

7.8.1 Mô tả vấn đề

Pipeline kết nối nhiều thành phần: crawler, Kafka, Spark, HDFS, Elasticsearch, Kibana và Airflow. Khi triển khai trên Kubernetes, mỗi thành phần nằm trong namespace riêng và giao tiếp thông qua service DNS nội bộ. Sai hostname, sai namespace hoặc service chưa sẵn sàng đều có thể làm pipeline thất bại.

Tắc động là lỗi thường không nằm ở code xử lý dữ liệu mà nằm ở cấu hình môi trường, thứ tự khởi động service hoặc timeout giữa các thành phần.

7.8.2 Cách tiếp cận đã thử

Một cách đơn giản là hard-code endpoint trong code. Cách này chạy nhanh ở local nhưng khó chuyển sang Kubernetes. Cách khác là đặt tất cả dịch vụ trong cùng namespace để giảm độ phức tạp service discovery, nhưng làm giảm khả năng quản lý và phân quyền.

7.8.3 Giải pháp cuối cùng

Dự án tách namespace theo vai trò như `hdfs`, `kafka`, `spark`, `search`, `airflow` và sử dụng DNS nội bộ Kubernetes như `elasticsearch.search.svc`. Các endpoint quan

trọng được đưa vào biến môi trường hoặc file cấu hình. Airflow có quyền tạo và theo dõi Kubernetes Job trong namespace `spark`, đồng thời kiểm tra tài nguyên Kafka trong namespace `kafka`.

Với dữ liệu lỗi, speed layer ghi bản ghi không hợp lệ vào `jobs_dead_letter`, kèm lý do lỗi, offset, partition và timestamp để truy vết.

7.8.4 Điểm rút ra

Trong hệ thống phân tán, cần coi cấu hình kết nối là một phần của thiết kế kiến trúc. Service discovery nên dựa trên DNS/service name ổn định, không hard-code IP. Với các kết nối quan trọng, cần bổ sung retry, timeout, health check và về lâu dài có thể áp dụng circuit breaker ở các API/service serving.

7.9 Bài học 9: Tối ưu hiệu năng phải bắt đầu từ việc xác định đúng điểm nghẽn

7.9.1 Mô tả vấn đề

Hiệu năng của hệ thống có thể bị giới hạn bởi nhiều điểm: crawler chậm do network, Kafka backlog, Spark shuffle, HDFS I/O, Elasticsearch bulk indexing hoặc dashboard Kibana truy vấn quá rộng. Nếu tối ưu sai điểm nghẽn, hệ thống vẫn chậm nhưng lại tăng chi phí tài nguyên.

7.9.2 Cách tiếp cận đã thử

Có thể tăng tài nguyên cho Spark hoặc Elasticsearch để cải thiện tốc độ. Tuy nhiên trên Minikube, tăng tài nguyên một thành phần có thể làm thiếu tài nguyên cho thành phần khác. Một hướng khác là giảm dữ liệu đầu vào bằng filter sớm, nhưng nếu filter quá sớm sẽ làm mất dữ liệu cần cho kiểm toán.

7.9.3 Giải pháp cuối cùng

Dự án tối ưu theo từng lớp: crawler sinh JSONL có schema rõ ràng; Spark batch đọc Parquet thay vì JSON ở các lớp sau; Elasticsearch chỉ nhận dữ liệu Gold hoặc realtime đã chuẩn hóa; Kibana dùng index riêng cho từng loại biểu đồ. Các aggregation realtime được tách thành nhiều module như `jobs_per_10m.py`, `top_skills_hourly.py` và `salary_bins_realtime.py`, giúp bật/tắt từng nhóm theo nhu cầu.

7.9.4 Điểm rút ra

Tối ưu hiệu năng cần dựa trên đo đạc: thời gian job, số record, kích thước partition, Kafka lag, thời gian ghi Elasticsearch và độ trễ dashboard. Cache chỉ nên dùng khi dữ liệu được đọc lại nhiều lần. Phân bổ tài nguyên phải tính đến toàn bộ cụm, không chỉ một job đơn lẻ.

7.10 Bài học 10: Giám sát và gỡ lỗi cần log có cấu trúc, metrics và khả năng truy vết theo pipeline

7.10.1 Mô tả vấn đề

Khi pipeline gồm nhiều tầng, một lỗi ở dashboard có thể bắt nguồn từ crawler, schema Raw, job Silver, model dự báo lương hoặc bước ghi Elasticsearch. Nếu log rời rạc và không có metadata, việc xác định nguyên nhân gốc rất mất thời gian.

7.10.2 Cách tiếp cận đã thử

Cách thủ công là xem log từng pod bằng `kubectl logs` và kiểm tra dữ liệu ở từng thư mục HDFS. Cách này phù hợp khi hệ thống nhỏ nhưng không đủ khi số job và số index tăng. Một cách khác là chỉ dựa vào trạng thái Airflow task, nhưng trạng thái task không cho biết chất lượng dữ liệu bên trong.

7.10.3 Giải pháp cuối cùng

Dự án đưa logging dùng chung vào `configs/logger.py`, có tài liệu migration shared logger, sử dụng Airflow để quan sát trạng thái job và dùng Kafka dead-letter topic để lưu lỗi bản ghi streaming. Các schema test trong `tests/` kiểm tra cấu trúc output realtime như `jobs_per_10m`, `salary_bins_realtime` và `top_skills_hourly`. Kibana cũng đóng vai trò quan sát dữ liệu đầu ra cuối cùng.

7.10.4 Điểm rút ra

Giám sát tốt cần kết hợp ba lớp: log kỹ thuật, metrics vận hành và kiểm tra chất lượng dữ liệu. Mỗi bản ghi nên có định danh truy vết như `job_id`, `timestamp`, `source` và trạng thái xử lý. Khi có lỗi, cần phân tích nguyên nhân gốc theo chuỗi Raw → Bronze → Silver → Gold → Elasticsearch, thay vì chỉ sửa ở tầng hiển thị.

7.11 Bài học 11: Mở rộng hệ thống cần lập kế hoạch ngang, dọc và chi phí tài nguyên

7.11.1 Mô tả vấn đề

Hệ thống ban đầu chạy trên Minikube, phù hợp cho phát triển và kiểm thử tích hợp. Tuy nhiên khi số nguồn crawl, số bản ghi và số dashboard tăng, nhu cầu mở rộng sẽ xuất hiện ở nhiều tầng: crawler, Kafka partitions, Spark executors, HDFS storage, Elasticsearch shards và Airflow workers.

7.11.2 Cách tiếp cận đã thử

Scale dọc bằng cách tăng CPU/RAM cho node Minikube giúp cải thiện nhanh nhưng có giới hạn vật lý. Scale ngang bằng cách tăng replica/partition phù hợp hơn cho môi trường production, nhưng yêu cầu cấu hình lại Kafka, Spark, Elasticsearch và chính sách resource request/limit.

7.11.3 Giải pháp cuối cùng

Trong phạm vi dự án, hệ thống được container hóa và tách namespace để chuẩn bị cho scale độc lập. Các script triển khai, manifest Kubernetes và cấu hình Airflow/Spark được tổ chức trong `infra/` và `scripts/`. Các job được tách theo chức năng để có thể tăng tài nguyên cho đúng thành phần khi cần.

7.11.4 Điểm rút ra

Khả năng mở rộng phải được thiết kế từ cấu trúc deployment, không chờ đến khi quá tải mới xử lý. Cần có kế hoạch tài nguyên cho từng tầng, bao gồm số partition Kafka, số executor Spark, dung lượng HDFS, shard Elasticsearch và lịch chạy Airflow. Auto-scaling chỉ hiệu quả khi hệ thống đã có metrics đáng tin cậy và workload được chia nhỏ hợp lý.

7.12 Bài học 12: Chất lượng dữ liệu và kiểm thử phải được đặt ngang hàng với code xử lý

7.12.1 Mô tả vấn đề

Dữ liệu tuyển dụng có nhiều trường bán cấu trúc: lương có thể là khoảng, thỏa thuận hoặc thiếu; kinh nghiệm có thể là số năm, số tháng hoặc không yêu cầu; kỹ năng có thể nằm trong mô tả tự do. Nếu không kiểm tra chất lượng, pipeline vẫn chạy thành công nhưng kết quả phân tích sai.

7.12.2 Cách tiếp cận đã thử

Một cách là kiểm tra thủ công một số file JSONL và dashboard. Cách này nhanh nhưng không ổn định. Cách khác là validate schema nghiêm ngặt ngay tại ingestion, nhưng dữ liệu web thay đổi thường xuyên nên validate quá cứng có thể làm mất nhiều bản ghi.

7.12.3 Giải pháp cuối cùng

Dự án dùng `quality_flags` ở Raw để ghi nhận chất lượng đầu vào, chuẩn hóa dần qua Bronze/Silver/Gold và bổ sung test tự động trong `tests/`. Các test kiểm tra producer, fake crawler và schema của các aggregation realtime. Bản ghi lỗi realtime được đưa vào `jobs_dead_letter` thay vì làm dừng toàn bộ stream.

7.12.4 Điểm rút ra

Kiểm thử pipeline dữ liệu cần bao gồm unit test cho hàm chuẩn hóa, integration test cho Kafka/Spark/Elasticsearch và performance test cho job lớn. Data quality nên được đo bằng chỉ số cụ thể như tỷ lệ thiếu lương, tỷ lệ thiếu kỹ năng, số bản ghi lỗi schema và số bản ghi trùng. Không nên đánh đồng “job chạy thành công” với “dữ liệu đúng”.

7.13 Bài học 13: Bảo mật và quản trị dữ liệu cần được chuẩn bị ngay cả trong môi trường học thuật

7.13.1 Mô tả vấn đề

Mặc dù dữ liệu tuyển dụng chủ yếu là dữ liệu công khai, hệ thống vẫn có nhiều điểm cần quản trị: quyền truy cập HDFS, Kafka topic, Elasticsearch index, Airflow UI, Kibana dashboard, biến môi trường và log vận hành. Nếu không kiểm soát, người dùng có thể vô tình xóa dữ liệu, ghi sai topic hoặc lộ endpoint nội bộ.

7.13.2 Cách tiếp cận đã thử

Trong môi trường local, có thể đơn giản hóa bảo mật để dễ triển khai. Tuy nhiên cách này không phản ánh yêu cầu production. Một hướng khác là bật đầy đủ authentication/authorization cho mọi dịch vụ, nhưng sẽ làm phức tạp đáng kể quá trình phát triển trên Minikube.

7.13.3 Giải pháp cuối cùng

Dự án tổ chức cấu hình qua `configs/.env.example`, tách namespace Kubernetes, tách vai trò Airflow và Spark job, đồng thời không hard-code thông tin nhạy cảm trực tiếp trong code xử lý. Các dashboard Kibana và Elasticsearch index được đặt tên rõ ràng theo batch/realtime để hạn chế nhầm lẫn khi vận hành.

7.13.4 Điểm rút ra

Bảo mật nên được triển khai theo từng mức: quản lý secret, phân quyền namespace/service account, giới hạn quyền ghi/xóa index, bật audit log và mã hóa kết nối khi chuyển sang môi trường thật. Với dữ liệu tuyển dụng, cũng cần tuân thủ điều khoản sử dụng của nguồn dữ liệu, hạn chế crawl quá mức và không lưu thông tin không cần thiết.

7.14 Bài học 14: Chịu lỗi cần kết hợp phục hồi tự động, checkpoint và khả năng tái xử lý

7.14.1 Mô tả vấn đề

Pipeline dữ liệu lớn luôn có khả năng gặp lỗi: crawler bị gián đoạn, Kafka broker restart, Spark job fail, checkpoint hỏng, HDFS tạm thời không sẵn sàng hoặc Elasticsearch ghi lỗi. Nếu không có cơ chế chịu lỗi, một lỗi nhỏ có thể làm mất dữ liệu hoặc buộc chạy lại toàn bộ pipeline.

7.14.2 Cách tiếp cận đã thử

Một cách là chạy lại toàn bộ pipeline từ đầu khi có lỗi. Cách này đơn giản nhưng tốn thời gian. Cách khác là retry từng bước, nhưng nếu job không idempotent có thể tạo dữ liệu trùng hoặc ghi sai kết quả.

7.14.3 Giải pháp cuối cùng

Dự án thiết kế pipeline theo hướng có thể tái xử lý: Raw được lưu bền vững trên HDFS, các job batch tách theo lớp dữ liệu, Airflow điều phối và retry theo task, Spark Structured Streaming dùng checkpoint để phục hồi offset/state, Kafka dead-letter lưu bản ghi lỗi, còn Elasticsearch index có thể được rebuild từ Gold khi cần.

7.14.4 Điểm rút ra

Chịu lỗi không chỉ là retry. Cần có dữ liệu nguồn bền vững, job idempotent, checkpoint rõ ràng, backup cấu hình và quy trình khôi phục thảm họa. Trong tương lai, hệ thống nên bổ sung replication cho storage, backup định kỳ cho Elasticsearch/Kibana saved objects và runbook khôi phục cho từng loại sự cố.

TÀI LIỆU THAM KHẢO

- [1] Nathan Marz and James Warren, *Big Data: Principles and best practices of scalable realtime data systems*. Manning Publications, 2015.
- [2] Databricks, “What is a medallion architecture?” <https://www.databricks.com/glossary/medallion-architecture>. Truy cập ngày 06/06/2026.
- [3] TopCV, “Nền tảng tuyển dụng và tìm kiếm việc làm TopCV.” <https://www.topcv.vn/>. Truy cập ngày 06/06/2026.
- [4] JSON Lines, “JSON Lines text file format.” <https://jsonlines.org/>. Truy cập ngày 06/06/2026.
- [5] Python Software Foundation, “Python Documentation.” <https://docs.python.org/3/>. Truy cập ngày 06/06/2026.
- [6] Leonard Richardson, “Beautiful Soup Documentation.” <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>. Truy cập ngày 06/06/2026.
- [7] Python Requests, “Requests: HTTP for Humans.” <https://requests.readthedocs.io/>. Truy cập ngày 06/06/2026.
- [8] Apache Hadoop, “Apache Hadoop Documentation.” <https://hadoop.apache.org/docs/>. Truy cập ngày 06/06/2026.
- [9] Apache Hadoop, “HDFS Architecture Guide.” <https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html>. Truy cập ngày 06/06/2026.
- [10] Apache Parquet, “Apache Parquet Documentation.” <https://parquet.apache.org/docs/>. Truy cập ngày 06/06/2026.
- [11] Apache Spark, “Apache Spark Documentation.” <https://spark.apache.org/docs/latest/>. Truy cập ngày 06/06/2026.
- [12] Apache Spark, “PySpark API Reference.” <https://spark.apache.org/docs/latest/api/python/>. Truy cập ngày 06/06/2026.
- [13] Apache Spark, “Spark SQL, DataFrames and Datasets Guide.” <https://spark.apache.org/docs/latest/sql-programming-guide.html>. Truy cập ngày 06/06/2026.
- [14] Apache Spark, “Structured Streaming Programming Guide.” <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>. Truy cập ngày 06/06/2026.

- [15] Apache Spark, “MLlib: Main Guide.” <https://spark.apache.org/docs/latest/ml-guide.html>. Truy cập ngày 06/06/2026.
- [16] Apache Spark, “Gradient-Boosted Trees.” <https://spark.apache.org/docs/latest/ml-classification-regression.html#gradient-boosted-tree-regression>. Truy cập ngày 06/06/2026.
- [17] Apache Kafka, “Apache Kafka Documentation.” <https://kafka.apache.org/documentation/>. Truy cập ngày 06/06/2026.
- [18] Apache Kafka, “Kafka Design.” <https://kafka.apache.org/documentation/#design>. Truy cập ngày 06/06/2026.
- [19] Strimzi, “Strimzi Documentation.” <https://strimzi.io/documentation/>. Truy cập ngày 06/06/2026.
- [20] Elastic, “Elasticsearch Guide.” <https://www.elastic.co/guide/en/elasticsearch/reference/current/index.html>. Truy cập ngày 06/06/2026.
- [21] Elastic, “Elasticsearch for Apache Hadoop.” <https://www.elastic.co/guide/en/elasticsearch/hadoop/current/index.html>. Truy cập ngày 06/06/2026.
- [22] Elastic, “Kibana Guide.” <https://www.elastic.co/guide/en/kibana/current/index.html>. Truy cập ngày 06/06/2026.
- [23] Apache Airflow, “Apache Airflow Documentation.” <https://airflow.apache.org/docs/>. Truy cập ngày 06/06/2026.
- [24] Apache Airflow, “DAGs.” <https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/dags.html>. Truy cập ngày 06/06/2026.
- [25] Kubernetes, “Kubernetes Documentation.” <https://kubernetes.io/docs/home/>. Truy cập ngày 06/06/2026.
- [26] Kubernetes, “Jobs.” <https://kubernetes.io/docs/concepts/workloads/controllers/job/>. Truy cập ngày 06/06/2026.
- [27] Kubernetes, “CronJobs.” <https://kubernetes.io/docs/concepts/workloads/controllers/cron-jobs/>. Truy cập ngày 06/06/2026.
- [28] Minikube, “Minikube Documentation.” <https://minikube.sigs.k8s.io/docs/>. Truy cập ngày 06/06/2026.
- [29] Docker, “Docker Documentation.” <https://docs.docker.com/>. Truy cập ngày 06/06/2026.
- [30] Prometheus, “Prometheus Documentation.” <https://prometheus.io/docs/introduction/overview/>. Truy cập ngày 06/06/2026.
- [31] Nhóm 9, “Big Data Job Market Repository.” <https://github.com/a3kc74/bigdata-job-market>. Truy cập ngày 06/06/2026.